

Ingeniería y Calidad de Software

Resumen de estudio para el parcial de **Aprobación Directa**, que toma el temario completo: las nueve unidades del programa. Sigue el estilo del resumen de Regularización: está pensado para leerse de corrido, cada capítulo abre explicando por qué el tema existe antes de entrar en las definiciones, y las tablas aparecen sólo donde hay algo que comparar. El orden es temático, no el del examen. Los capítulos 24 y 25 están pensados para consultarlos durante el parcial. Para el detalle completo de cada tema y los ejercicios resueltos está la wiki ([ISW.md](#)).

Versión extendida (101 págs.). La versión para estudiar es `resumen-ad.md`, condensada a unas 60 páginas con la misma proporción que el resumen de Regularización.

CONTENIDO

- | | |
|---|--|
| 1 El parcial y su temario | 14 Los requerimientos y su desarrollo |
| 2 Qué significa calidad en software | 15 La gestión de requerimientos y de los cambios |
| 3 El software como objeto de ingeniería | 16 Verificación y validación |
| 4 CMMI: qué es y para qué sirve | 17 La organización de las pruebas |
| 5 Anatomía de un área de proceso | 18 Las técnicas dinámicas |
| 6 Los niveles de madurez | 19 Las técnicas estáticas: revisiones |
| 7 La gestión de procesos de la organización | 20 La gestión de configuración (CM) |
| 8 Activos de proceso y productos de trabajo | 21 El aseguramiento de la calidad (PPQA) |
| 9 El proceso de desarrollo: SPEM y RUP | 22 La medición y el análisis (MA) |
| 10 El proyecto y su planificación | 23 Las pericias informáticas |
| 11 La estimación del esfuerzo | 24 Cómo reconocer el área de proceso en un enunciado |
| 12 La gestión de riesgos (RSKM) | 25 Las preguntas BP |
| 13 Los ciclos de vida | 26 Confusiones frecuentes |

1. El parcial y su temario

El parcial de Aprobación Directa es **presencial**, de **opción múltiple** —entre 15 y 20 preguntas, según la cátedra, aunque la cantidad es orientativa— y **a libro abierto, sólo en papel**: no se permite ningún dispositivo electrónico. Se aprueba con el **60 % del total de puntos**. Para promocionar la materia hay que aprobar los dos parciales, Regularización y Aprobación Directa, y entregar los trabajos prácticos de clase. Si se desaprueba éste, queda el **Globalizador** de Aprobación Directa.

A diferencia del parcial de Regularización, acá no hay recorte de páginas: entra **todo el programa**.

UNIDAD	TEMA	CAPÍTULOS
1 — Modelos de calidad	Calidad, ingeniería del software, CMMI, niveles de madurez	2 a 6
2 — Gestión de procesos	OPE, OPD, OT, activos de proceso, SPEM y RUP	7 a 9
3 — Gestión de proyectos	PP, PMC, estimación y puntos función, riesgos, ciclos de vida	10 a 13
4 — Requerimientos	RD, REQM y gestión de solicitudes de cambio	14 y 15

UNIDAD	TEMA	CAPÍTULOS
5 — Verificación y validación	VER, VAL, pruebas, técnicas dinámicas y estáticas	16 a 19
6 — Gestión de configuración	CM: elementos, líneas base, versionado, cambios	20
7 — Aseguramiento de calidad	PPQA: auditorías, no conformidades, escalamiento	21
8 — Medición y análisis	MA: métricas de proceso, proyecto y producto	22
9 — Pericias informáticas	Evidencia digital, cadena de custodia, protocolo	23

Lo que muestran los parciales de 2024 y 2025. Aunque entra todo, el peso estuvo en las unidades que **no** estaban en el parcial de Regularización: **configuración, aseguramiento de la calidad, requerimientos y medición**, más alguna pregunta de planificación y de verificación y validación. Riesgos y pericias no aparecieron, pero entran.

Las preguntas tienen tres formas que se repiten. La primera es la pregunta **BP**: se describe cómo trabaja una software factory y hay que decir si esa práctica **puede generar problemas de calidad** y por qué; el capítulo 25 explica cómo resolverlas. La segunda pide reconocer **qué área de proceso —y a veces qué práctica— corresponde a una actividad** descripta; para eso está el capítulo 24. La tercera son ejercicios cortos con números: calcular cuántos años sobrevive un esquema de versión, cuántas releases mínimas tiene un proyecto, o qué casos de prueba salen de una partición.

ATENCIÓN En las preguntas de selección múltiple, **marcar una opción incorrecta además de la correcta deja la pregunta en cero**. Pasó en el parcial de 2024. Ante la duda, es mejor marcar sólo lo que se puede justificar.

2. Qué significa calidad en software

La palabra calidad arrastra un problema: todos creen saber qué quiere decir, y cada uno entiende algo distinto. En la materia se trabaja con una idea central —**calidad es idoneidad de uso**— y varias definiciones que la precisan desde ángulos diferentes.

La definición **clásica**, de Juran, la plantea en dos mitades: las características del producto que satisfacen las necesidades del cliente, y la **inexistencia de deficiencias**. Un producto puede tener todas las funciones pedidas y aun así fallar la segunda mitad. **Deming** la mira desde el otro lado: calidad es cualquier cosa que **aumente la satisfacción del cliente**.

CMMI la define como la capacidad de un conjunto de características inherentes de un producto, componente o **proceso** de satisfacer **por completo** los requisitos del cliente. Lo importante acá es que incluye al proceso: para CMMI, un proceso también tiene calidad.

ISO 8402 —y casi con las mismas palabras **ANSI**— habla del conjunto de propiedades y características que le confieren aptitud para satisfacer necesidades **explícitas o implícitas**. La palabra que aporta es *implícitas*: hay expectativas que el cliente nunca escribió y que igual espera que se cumplan. **ISO 9000:2000**, en cambio, la vuelve una cuestión de grado: el **nivel** al que un conjunto de **características inherentes** satisface los **requisitos**.

ATENCIÓN En una opción múltiple, cada definición se reconoce por la palabra que aporta: *inexistencia de deficiencias* (Juran), *satisfacción del cliente* (Deming), *proceso y por completo* (CMMI), *explícitas o implícitas* (ISO 8402 y ANSI), *nivel de características inherentes* (ISO 9000).

2.1 De qué depende la calidad

Cuatro elementos la determinan: los **procesos y buenas prácticas** que se siguen, las **herramientas** disponibles, las **personas** que hacen el trabajo, y las **medidas y métricas** con las que se controla. Ninguno alcanza por sí solo. Un equipo excelente con un proceso caótico produce resultados irrepetibles, y un proceso impecable no compensa a un equipo sin formación.

2.2 Tres tipos de calidad de producto

No toda la calidad se ve desde afuera, y esa distinción explica por qué a veces un sistema que "anda bien" es un problema.

La **calidad interna** es la que **el cliente no percibe**: facilidad de mantenimiento, claridad del código, respeto de los estándares, reusabilidad. No la ve, pero la paga: es la que determina cuánto cuesta cada cambio futuro.

La **calidad externa** es la visible: que el producto cumpla los requerimientos acordados.

La **calidad de uso** es la que algunos llaman *calidad futura*. Es la que no se tuvo en cuenta y aparece recién cuando el software se usa de verdad: el sistema pasa todas las pruebas, se entrega, y a la media hora de operación real se nota el problema que nadie había previsto.

Un resumen de alumnos lo lee con la norma **ISO 9126**: la interna se mide **antes de probar**, sin el código funcionando; la externa, con **las pruebas**, en un entorno que no es el del cliente; la de uso es la **satisfacción del cliente** con el software en su entorno real.

2.3 Los tres niveles de gestión de la calidad

La calidad se puede gestionar en tres escalas distintas, y cada una tiene sus herramientas.

A nivel de **producto** se gestiona el desarrollo con pruebas ejecutadas **en paralelo a cada etapa**, para detectar y corregir los defectos cerca de donde se producen. A nivel de **proyecto** se gestiona controlando todas las fases y áreas de gestión de ese proyecto en particular, implantando metodologías y mejores prácticas. A nivel de **proceso** se gestionan las áreas de proceso de **toda la organización** mediante una metodología, lo que da información sobre los procesos para controlarlos y mejorarlos.

Ese tercer nivel es el que ocupa la mayor parte de la materia, y es donde juega CMMI. La apuesta de fondo es que **la calidad de un producto está muy influenciada por la calidad del proceso empleado para desarrollarlo y mantenerlo**: si el proceso es bueno, los productos buenos dejan de ser casualidad.

Conviene fijar desde ya la diferencia entre **proceso** y **proyecto**, porque atraviesa toda la materia. Los dos son conjuntos de tareas organizadas para cumplir un objetivo, pero el proceso es **genérico, repetitivo y reiterativo**, no tiene fechas y **produce siempre el mismo producto**; el proyecto es **único**, tiene inicio y fin, trabaja bajo **restricciones** de tiempo, dinero y recursos, y es en el fondo una **instanciación de procesos** (el capítulo 10 lo define con precisión). Por eso la calidad de un proyecto depende tanto de los **procesos elegidos** para llevarlo a cabo.

2.4 Aseguramiento y control de la calidad

Dentro de la gestión de la calidad conviven dos actitudes que se confunden seguido: **prevenir** los defectos o **detectarlos**. La guía de INTECO las separa así:

	QA – ASEGURAMIENTO DE LA CALIDAD	QC – CONTROL DE LA CALIDAD
Naturaleza	Preventivo y proactivo	Reactivo
Se orienta al	Proceso	Producto o servicio
Responsabilidad	De la organización	Del equipo de control
Qué hace	Identifica las debilidades de los procesos y los mejora; evalúa si el control funciona	Verifica que los atributos especificados estén presentes en el producto
Ejemplos	Auditorías de proceso, definición de procesos, selección de herramientas, formación	Revisiones, inspecciones, ejecución de pruebas

El ejemplo de clase lo deja claro. Una fábrica produce mil tazas, al final controla que pesen menos de 150 gramos y descarta las diez que no cumplen: eso es **control**. Si salen quinientas defectuosas y, en vez de descartarlas, se revisa el proceso productivo para que no vuelva a pasar, eso es **aseguramiento**. Las tareas de asesoramiento de QA van en esa línea: procesos que **añadan valor, prevención, mejora y reutilización de la experiencia**.

El área **PPQA** de CMMI (capítulo 21) es aseguramiento en este sentido: no busca fallas en el producto, controla que procesos y productos respeten los estándares definidos. Las pruebas, en cambio, son control, y tienen que **empezar pronto y seguir durante todo el ciclo de vida**.

ATENCIÓN La misma presentación asocia la **verificación** con QA (revisiones e inspecciones, antes y durante el desarrollo) y la **validación** con QC (pruebas de caja negra con el desarrollo casi terminado). Eso **no coincide con CMMI**, donde la prueba de sistema es VER y la validación puede empezar con los requerimientos, y ni siquiera coincide con su propia tabla, que pone revisiones e inspecciones como QC. Para una pregunta de concepto sobre INTECO, usá su tabla; para ubicar una actividad en un área, usá CMMI (capítulo 16).

2.5 Medir para poder mejorar

La presentación cierra con la medición, resumida en una cadena: **si no se puede medir, no se puede controlar; si no se puede controlar, no se puede mejorar**. Una buena métrica se relaciona con los objetivos, está bien definida, es sencilla y ayuda a **entender el pasado, controlar el presente y predecir el futuro**; las que reflejan los factores críticos de éxito son los **KPI**, que se siguen en **cuadros de mando**. El vocabulario de la medición se precisa en el capítulo 22.

Un dato de la misma fuente justifica buena parte de la materia: la mayoría de los defectos **se introducen en los requisitos y el diseño**, pero la mayoría **se detectan en las pruebas de aceptación y en producción**, que es cuando más caro sale corregirlos (capítulo 16).

3. El software como objeto de ingeniería

Antes de hablar de procesos conviene entender por qué el software necesita una ingeniería propia y no le sirven las de otras disciplinas.

Software = programas + datos + documentos. Es un elemento **lógico**, no físico.

Cada componente tiene su papel. Los **programas** son las instrucciones que dan la funcionalidad y el rendimiento. Los **datos** son los que permiten manejar y probar los programas, junto con sus estructuras. Los **documentos** describen la operación y el uso, y también los necesita quien mantiene el software. La percepción común —y equivocada— es que el software son sólo los programas.

ATENCIÓN La definición **IEEE Std. 610** agrega un cuarto componente: "programas, **procedimientos**, documentación y datos asociados", y las anotaciones de clase la repiten. Pero en los parciales de años anteriores la respuesta esperada fue la de INTECO: **programas, datos y documentos**.

Esa naturaleza lógica trae cuatro consecuencias que condicionan todo lo demás.

El software se desarrolla, no se fabrica. No hay línea de producción: cada producto se construye para los requisitos únicos de un cliente. No se pueden amortizar costos repitiendo unidades, porque no hay unidades que repetir.

El recurso principal son las personas, y no son intercambiables con el tiempo. Agregar gente no acelera un proyecto de forma lineal: el desarrollo requiere coordinación y comunicación, y cada integrante nuevo no es productivo de inmediato y además consume tiempo de los que ya están. Por eso es falso el principio de que "las personas y el tiempo son intercambiables".

El software no se estropea, pero se deteriora. Un engranaje se desgasta por uso; el software no. Al principio de su vida falla por los defectos que no se detectaron, y a medida que se corrigen las fallas bajan. Lo que lo degrada después son los **cambios**: cada modificación de mantenimiento tiene probabilidad de introducir defectos nuevos, y con los años el producto acumula complejidad y fragilidad.

La reutilización está lejos de su potencial. El hardware se arma con componentes estándar, y eso le da menor costo, plazos más cortos y un mantenimiento más fácil. El software, en cambio, se construye normalmente **a medida y desde cero**: cada producto responde a requisitos únicos, y por eso cuesta identificar qué partes servirían para otro.

Hay dos grandes familias: el software **de aplicaciones**, que da servicio al negocio, y el **de sistemas**, que opera el propio sistema informático (sistemas operativos, compiladores). Una aplicación se caracteriza por el **contenido** de la información que maneja y por su **determinismo**: un cálculo de ingeniería es determinado; un sistema operativo multiusuario, en el que no se puede predecir el orden ni el momento en que llegan los datos, es indeterminado.

3.1 Qué es la ingeniería del software

El término nace en la **conferencia de la OTAN de 1968**, convocada para reflexionar sobre la **crisis del software**. De las definiciones que circulan, la que se usa es la de IEEE:

Ingeniería del software (IEEE, 1993): aplicación de un enfoque **sistemático, disciplinado y cuantificable** al desarrollo, operación y mantenimiento del software.

Las tres palabras importan. *Sistemático* excluye improvisar; *disciplinado* excluye abandonar el método cuando aprieta el tiempo; *cuantificable* excluye opinar sin medir. Las otras definiciones apuntan a lo mismo desde otro ángulo; la de Bauer (1972), por ejemplo, habla de obtener software **rentable y fiable** que funcione en máquinas reales.

Su **objetivo primario** es construir un producto de **alta calidad** de manera **oportuna**: entender el problema, diseñar una solución, implementarla correctamente, probarla y gestionar todo eso, incluidos el **aseguramiento de la calidad** y la **gestión de la configuración**. Y no es una metodología rígida —"no es una religión y no hay verdades absolutas"—: procesos, métodos y herramientas se adaptan al producto, a la gente y al negocio.

Se suele representar como una **tecnología multicapa**: sobre una base de **compromiso con la calidad** se apoyan los **procesos**, sobre ellos los **métodos**, y encima las **herramientas**. El orden no es decorativo: comprar herramientas sin proceso debajo no produce calidad.

La capa de **proceso** es el fundamento, "la unión que mantiene juntas las capas": el marco que le permite al jefe de proyecto controlar la gestión y las actividades técnicas. Un proceso definido responde quién se comunica con quién, cómo se coordinan las actividades que dependen entre sí, quién es responsable de qué, y quién produce cada producto de trabajo y cómo se evalúa; además, **especifica los puntos de control de calidad**. Todo equipo tiene un proceso, aunque muchas veces sea **ad hoc, invisible y caótico**. Los **métodos** son el "cómo" técnico —análisis, diseño, codificación, pruebas, mantenimiento—, y las **herramientas** automatizan el soporte a las otras dos capas; cuando se usan, la documentación pasa a ser parte del trabajo y no una tarea agregada.

3.2 Las etapas y el mantenimiento

Las etapas clásicas son: análisis de requisitos, especificación, diseño y arquitectura, programación, **prueba** y mantenimiento. La etapa que comprueba que el software realiza correctamente las tareas indicadas en la especificación es la de **prueba**, y es buena práctica que pruebe **alguien distinto** de quien programó. El resultado del análisis se plasma en la **Especificación de Requisitos**, normalizada por la **IEEE Std. 830-1998**; la especificación pesa sobre todo en las **interfaces externas**, que deben permanecer estables.

El mantenimiento no es una sola cosa. Distinguir los cuatro tipos es una de las preguntas recurrentes de la materia:

TIPO	QUÉ PERSIGUE	SEÑAL TÍPICA EN UN ENUNCIADO
Correc-tivo	Corregir errores detectados en el producto	"Se detectaron errores", "falla"
Evoluti-vo	Incorporaciones, modificaciones y eliminaciones necesarias para cubrir la expansión o el cambio en las necesidades del usuario	Funcionalidad nueva, un módulo nuevo, ajustes que pide el cliente
Adapta-tivo	Modificaciones que responden a cambios del entorno donde opera el sistema: hardware, software de base, gestores de base de datos, comunicaciones	Cambio de plataforma o de arquitectura, distribución geográfica
Perfec-tivo	Mejorar la calidad interna del sistema, sin cambio funcional visible	Refactorizar, optimizar

ATENCIÓN **Perfectivo no es sinónimo de evolutivo.** El evolutivo cambia lo que el sistema **hace**; el perfectivo mejora **cómo está hecho**. Y que un cambio lo pida el cliente no lo vuelve evolutivo: según la resolución de un final, repartir un sistema entre sucursales de otra ciudad, que la tecnología actual no soporta, es **adaptativo**, porque lo que cambia es el entorno.

3.3 Los principios de la disciplina

La guía de INTECO enumera una lista de principios que conviene haber leído al menos una vez, porque las opciones de examen salen de ahí: haz de la calidad la razón de trabajar; una buena gestión es más importante que una buena tecnología; las personas y el tiempo **no** son intercambiables; selecciona el ciclo de vida adecuado; las técnicas son anteriores a las herramientas; primero hazlo correcto, luego hazlo rápido; probar, probar y probar;

la entropía del software es creciente; el **compromiso del cliente es el factor más crítico** en la calidad; haz que los errores los encuentre un colaborador y no un cliente; **itera en todas las fases excepto en la codificación**; la gestión de errores y solicitudes de cambio es esencial; si mides lo que haces puedes aprender a hacerlo mejor; y no puedes cambiar todo de una vez.

Dos aparecen una y otra vez como correctos: **haz de la calidad la razón de trabajar y probar, probar y probar**. Y los distractores típicos son principios dados vuelta, que enuncian exactamente lo contrario de lo que sostiene la disciplina: que las personas y el tiempo son intercambiables, que conviene hacerlo rápido primero y correcto después, y que hay que forzar el mismo modelo de ciclo de vida en todos los proyectos.

4. CMMI: qué es y para qué sirve

CMMI son las siglas de *Capability Maturity Model Integration*: modelo de madurez de capacidades integrado, desarrollado por el **SEI**. Es un modelo de **mejora de procesos** para el desarrollo de productos y servicios, con buenas prácticas que cubren el ciclo de vida del producto desde la concepción hasta la entrega y el mantenimiento. No es una metodología ni un manual de procedimientos. Es una **guía de buenas prácticas** que no dice *cómo* hacer las cosas, sino **qué** hay que lograr. La versión que usa la cátedra es la **CMMI-DEV v1.2**, en castellano.

Esa distinción es la clave para entender el modelo entero. CMMI define objetivos; cada organización decide con qué prácticas los alcanza, y puede usar prácticas propias distintas de las que el modelo sugiere, siempre que cumpla el objetivo.

Trabajar con un modelo probado da tres cosas que una organización sola tarda años en construir: la **experiencia acumulada** de otras empresas, un **lenguaje y una visión común** dentro de la organización, y un **punto de partida** para no inventar desde cero. Un modelo es además una **visión simplificada** de la situación real, más fácil de controlar y de mejorar, y por eso permite **predecir** mejor el comportamiento y el rendimiento de la empresa. Los resultados que se le atribuyen son menos defectos, menos tiempo de entrega, menor costo, más satisfacción del cliente y más beneficios.

Hay algo que CMMI **no** hace: agregar actividades técnicas. Una organización sin ningún nivel también analiza, diseña, programa y prueba. Lo que cambia con la madurez es si esas actividades están **definidas, institucionalizadas, medidas y mejoradas**. Por eso el modelo habla de procesos y no de tecnologías: dos empresas con el mismo lenguaje de programación y las mismas herramientas pueden estar en niveles muy distintos.

CMMI tampoco es el único modelo. La guía de INTECO lo ubica entre los de **mejora de proceso** —junto con ISO/IEC 15504, SwTQM, ITMark, MoProsoft y TPI/TMAP, este último sólo para el proceso de pruebas— y los separa de los de **mejora de producto**, como ISO 9126 o XP. CMMI-DEV v1.2 se evalúa a través del SEI y sirve tanto para empresas grandes como para PYMES.

4.1 Área de proceso

Área de proceso: grupo de prácticas relacionadas que, implementadas conjuntamente, satisfacen un conjunto de objetivos importantes para la mejora en esa área.

CMMI define **22 áreas de proceso**, repartidas en los niveles de madurez 2 a 5 y agrupadas en **cuatro categorías**: **gestión de procesos**, **gestión de proyectos**, **ingeniería** y **soporte**. Cada una cubre un aspecto del trabajo: planificar proyectos, gestionar requerimientos, verificar productos, formar gente. Dentro de cada categoría

hay áreas **básicas**, que conviene implementar primero porque son la base de las demás, y áreas **avanzadas**. La lista completa, con nivel y categoría, está en el capítulo 6.

5. Anatomía de un área de proceso

Toda área de proceso está construida con las mismas piezas, y esas piezas tienen distinto peso. Entender cuál es obligatoria y cuál no es probablemente el concepto más preguntado de la materia.

CATEGORÍA	QUÉ ES	QUÉ INCLUYE
Requeridos	Lo que la organización debe lograr para satisfacer el área. Es la base de las evaluaciones	Metas específicas (SG) y metas genéricas (GG)
Esperados	Lo que puede implementar para lograr lo requerido. Se admiten alternativas propias	Prácticas específicas (SP) y prácticas genéricas (GP)
Informativos	Material que ayuda a entender cómo aproximarse a lo requerido y lo esperado	Subprácticas, productos de trabajo típicos, ampliaciones, elaboraciones de prácticas genéricas, títulos y notas, ejemplos, referencias, declaración de propósito, notas introductorias, áreas relacionadas

ATENCIÓN Lo **requerido son sólo las metas**. Las prácticas —aunque el modelo las liste y las explique— son **esperadas**, no obligatorias. Una organización puede sustituir una práctica por otra si con ella alcanza la misma meta.

Lo que se evalúa, entonces, es el logro de las metas. Si una organización alcanza todas las metas de un área sin aplicar al pie de la letra las prácticas recomendadas, cumple el área.

Los componentes informativos merecen tres aclaraciones, porque aparecen en las opciones. Los **productos de trabajo típicos** son ejemplos de lo que produce una práctica; se llaman "típicos" porque suele haber otros igual de eficaces que el modelo no enumera. Una **ampliación** es una nota o un ejemplo que sólo vale para una disciplina particular: ingeniería de sistemas, de hardware o de software. Y las **subprácticas** son informativas aunque describan pasos muy concretos: no se exigen, pero en los ejercicios de discriminación son las que justifican la respuesta (capítulo 24).

ATENCIÓN Pregunta típica: "¿cuáles son componentes **requeridos** de un área de proceso?" → **metas específicas y metas genéricas**. Las subprácticas son informativas, y las herramientas ni siquiera son componentes del modelo.

5.1 Qué significa "genérico"

Un componente es **genérico** cuando la misma declaración se aplica a **múltiples áreas de proceso**. Las metas y prácticas específicas son propias de un área; las genéricas se repiten en todas.

El papel de las genéricas es tratar la **institucionalización**: que el proceso no dependa de la buena voluntad de quien lo ejecuta, sino que esté incorporado a la manera de trabajar de la organización. Esa es la respuesta cuando preguntan qué componentes tratan la institucionalización.

Un ejemplo lo vuelve concreto. La **GP 2.9 Evaluar objetivamente la adherencia** está en todas las áreas: en cada una, alguien que no ejecuta el proceso controla que se esté siguiendo, incluso cuando el proyecto está atrasado. Esa práctica genérica se implementa a través de PPQA (capítulo 21). Del mismo modo, la **GP 2.6 Ges-**

tionar configuraciones pone bajo control los productos de cada área usando el sistema de gestión de configuración (capítulo 20).

En la representación por etapas se usan sólo dos metas genéricas: la **GG 2** en el nivel 2, y la **GG 3** del nivel 3 al 5.

5.2 La numeración

Las metas se numeran secuencialmente: **SG1**, **SG2**, **GG1**. Las prácticas llevan dos números, **SP x.y**, donde **x** es el número de la meta a la que pertenecen e **y** el número de secuencia dentro de esa meta. Así, **SP 2.3** es la tercera práctica de la segunda meta específica. Las genéricas siguen la misma regla: **GP 2.6** es la sexta práctica de la meta genérica 2.

Saber leer la numeración sirve para descartar opciones: si un área tiene dos metas específicas, una opción que diga **SP 3.1** es imposible.

6. Los niveles de madurez

CMMI organiza la mejora en cinco escalones. Cada uno describe un estado de la organización, no una lista de tareas cumplidas.

Nivel de madurez: meseta evolutiva definida para la mejora de procesos. Reúne las prácticas específicas y genéricas de un **conjunto predefinido de áreas de proceso**, y se alcanza logrando sus metas. Cada nivel madura un subconjunto de procesos y prepara a la organización para el siguiente.

NIVEL	NOMBRE	CÓMO SE RECONOCE
1	Inicial	Procesos ad hoc y caóticos: no hay un entorno estable que los sostenga. El éxito depende de la heroicidad del personal. Suelen entregar productos que funcionan, pero exceden el presupuesto y no cumplen el calendario ; se comprometen de más, abandonan los procesos en las crisis y no pueden repetir sus éxitos
2	Gestionado	Los proyectos se planifican y ejecutan según políticas , se monitorizan, controlan y revisan , y se evalúa la adherencia a sus descripciones de proceso. La dirección ve el estado en los hitos . La disciplina se mantiene bajo estrés . El orden es de cada proyecto : los procedimientos pueden ser distintos en cada uno
3	Definido	Existe un conjunto de procesos estándar de la organización y cada proyecto adapta el suyo con guías de adaptación . Los procesos se describen con más rigor: propósito, entradas, criterios de entrada , actividades, roles, medidas, verificación, salidas, criterios de salida . El orden es de la organización . El rendimiento es predecible sólo cualitativamente
4	Gestionado cuantitativamente	Organización y proyectos fijan objetivos cuantitativos de calidad y de rendimiento. Las medidas se analizan estadísticamente y se guardan en el repositorio de medición . Se identifican y corrigen las causas especiales de variación. El rendimiento es predecible cuantitativamente
5	En optimización	Mejora continua basada en la comprensión cuantitativa de las causas comunes de variación: mejoras incrementales e innovadoras , de proceso y de tecnología, con objetivos cuantitativos de mejora

Las dos clases de variación que separan los niveles 4 y 5 tienen definición propia en el glosario de CMMI:

Causa especial: la que es propia de una **circunstancia transitoria** y no forma parte inherente del proceso. **Causa común:** la variación que producen las **interacciones normales y esperadas** entre los componentes del proceso.

Un servidor que llega diez días tarde y atrasa un proyecto es una causa especial. Una técnica de estimación que subestima siempre los requerimientos con reglas de negocio complejas es una causa común: está metida en el proceso, y se va a repetir en cada proyecto hasta que se cambie la técnica.

6.1 Las dos comparaciones que hay que poder explicar

Nivel 2 frente a nivel 3: el alcance de los estándares. En nivel 2 cada proyecto puede tener sus propios procedimientos; lo que se exige es que los tenga, los siga y los sostenga. En nivel 3 los procedimientos **se adaptan desde el conjunto estándar de la organización**, y por lo tanto son consistentes entre proyectos salvo por las diferencias que las guías de adaptación permitan. Además, en nivel 3 los procesos se gestionan más proactivamente, usando la comprensión de las interrelaciones entre actividades y medidas detalladas.

Nivel 4 frente a nivel 5: el tipo de variación que se trata. El nivel 4 ataca las **causas especiales** —las anomalías puntuales— y logra **predictibilidad estadística**. El nivel 5 ataca las **causas comunes**, las que están incorporadas al proceso mismo, y para eliminarlas **cambia el proceso**.

6.2 Cómo se avanza: las reglas del modelo escalonado

De estas reglas salen casi todos los ejercicios de niveles.

Primera: para estar en un nivel hay que satisfacer todas las áreas de ese nivel y de los anteriores. Si falta una sola área, no se está en ese nivel. Una organización que cumple perfectamente las áreas de nivel 3 pero incumple una de nivel 2 no está en nivel 3 ni en nivel 2: está en **nivel 1**, que es el nivel por defecto porque no exige nada.

Segunda: los niveles son acumulativos. Alcanzar un nivel superior no exime de las metas de los anteriores. Una organización nivel 5 sigue planificando y monitorizando proyectos todos los días. Si dejara de hacerlo, se caería hasta nivel 1.

Tercera: se puede instanciar un área de nivel superior al propio si el contexto de negocio lo justifica, pero se corre el riesgo de intentar prácticas **sin la base institucional que las soporte**. Eso funciona hasta que aparece el estrés, que es justamente cuando más se las necesita. A veces conviene: para pasar del nivel 1 al 2 se suele crear un **grupo de procesos**, que pertenece a OPF (nivel 3); el nivel 2 no lo exige, pero ayuda a lograrlo.

Cuarta: saltar niveles es, en general, contraproducente, porque cada nivel es la base del siguiente. El libro da dos ejemplos: un proceso definido de nivel 3 corre **gran riesgo** si las prácticas de gestión de nivel 2 son deficientes —un calendario mal planificado, cambios a la línea base de requisitos sin control—, y recoger antes de tiempo los datos detallados del nivel 4 produce **datos que no se pueden interpretar**, porque las definiciones de procesos y de medidas todavía no son consistentes.

De la primera regla sale el cálculo que piden los ejercicios cuando a una organización le falta una sola área o una sola práctica: si es de **nivel 2**, la organización queda en **nivel 1**; si es de **nivel 3**, queda **como máximo en nivel 2**, y sólo si cumple todas las de nivel 2.

La misma lógica se pregunta al revés: qué se encuentra en una organización de cierto nivel. En una software factory de **nivel 2**, cada proyecto tiene su cronograma, su plan y sus puntos de control (PP y PMC). Lo que **no** se puede dar por supuesto es un proceso de prueba definido para toda la organización ni estándares organizacionales, que son de OPD, nivel 3. Y la afirmación "el aseguramiento de la calidad nace en el nivel 3" es falsa: PPQA es de nivel 2.

6.3 Madurez y capacidad

CMMI admite dos formas de mirar el progreso.

La **madurez**, que es la representación **por etapas**, mide a la **organización entera** contra los conjuntos fijos de áreas de cada nivel. Es la que se usa normalmente para certificar y la que usan todos los ejercicios de la materia.

La **capacidad**, que es la representación **continua**, mide el nivel alcanzado en **un área de proceso puntual**, con independencia del resto. Sirve para que la organización elija en qué área quiere crecer primero, según lo que le duela al negocio.

	REPRESENTACIÓN CONTINUA (CAPACIDAD)	REPRESENTACIÓN POR ETAPAS (MADUREZ)
Qué mide	Un área de proceso por vez	La organización , contra conjuntos fijos de áreas
Camino de mejora	Lo elige la organización: qué áreas y hasta qué nivel (perfil objetivo)	Predeterminado por el modelo
Escala	Niveles de capacidad 0 a 5 : Incompleto · Realizado · Gestionado · Definido · Gestionado cuantitativamente · En optimización	Niveles de madurez 1 a 5
Punto de partida	Incompleto	Inicial

Para alcanzar el nivel de capacidad 1 en un área hay que lograr **todas sus metas específicas**; del nivel 2 en adelante se suma la **institucionalización**, que aportan las metas y prácticas genéricas. Los niveles de madurez 2 a 5 llevan **los mismos nombres** que los de capacidad 2 a 5, a propósito: son conceptos complementarios.

ATENCIÓN Algunos resúmenes dan niveles de capacidad **0 a 3**: eso corresponde a CMMI v1.3 (conocimiento general). En la versión de la cátedra, la **v1.2**, son **0 a 5**. Y los ejercicios del parcial razonan **siempre por etapas**.

6.4 Las 22 áreas de proceso

NIVEL	SIGLA	ÁREA DE PROCESO	CATEGORÍA
2	PP	Planificación de proyecto	Gestión de proyectos
2	PMC	Monitorización y control del proyecto	Gestión de proyectos
2	SAM	Gestión de acuerdos con proveedores	Gestión de proyectos
2	REQM	Gestión de requerimientos	Ingeniería
2	MA	Medición y análisis	Soporte
2	PPQA	Aseguramiento de la calidad de proceso y de producto	Soporte
2	CM	Gestión de configuración	Soporte
3	OPF	Enfoque en procesos de la organización	Gestión de procesos
3	OPD	Definición de procesos de la organización	Gestión de procesos
3	OT	Formación organizativa	Gestión de procesos
3	IPM	Gestión integrada del proyecto	Gestión de proyectos
3	RSKM	Gestión de riesgos	Gestión de proyectos

NIVEL	SIGLA	ÁREA DE PROCESO	CATEGORÍA
3	RD	Desarrollo de requerimientos	Ingeniería
3	TS	Solución técnica	Ingeniería
3	PI	Integración de producto	Ingeniería
3	VER	Verificación	Ingeniería
3	VAL	Validación	Ingeniería
3	DAR	Análisis de decisiones y resolución	Soporte
4	OPP	Rendimiento del proceso de la organización	Gestión de procesos
4	QPM	Gestión cuantitativa de proyecto	Gestión de proyectos
5	OID	Innovación y despliegue en la organización	Gestión de procesos
5	CAR	Análisis causal y resolución	Soporte

En negrita, las trece que este resumen desarrolla; del resto alcanza con el nivel y la categoría. Es lo que hace falta para resolver un caso como "cumple todo el nivel 3 pero le falta MA", que deja a la organización en nivel 1 porque MA es de nivel 2. La tabla muestra también la lógica de 6.1: las siete de nivel 2 ordenan **cada proyecto**, y las de gestión de procesos aparecen recién en el nivel 3, cuando el orden pasa a ser **de la organización** (OPF, OPD y OT son las básicas de esa categoría; OPP y OID, las avanzadas).

ATENCIÓN La traducción castellana del CMMI que usa la cátedra tiene erratas que conviene conocer. En las carátulas de las áreas, **OT** figura como "nivel de madurez 4", **OID** como nivel 3 y **RSKM** como nivel 2; las tablas del mismo libro dan **OT = 3**, **OID = 5** y **RSKM = 3**, que es lo correcto (para OT lo confirma además la presentación de la cátedra). Y la tabla de áreas pone "gestión de proyectos" como categoría de OPF, OPD, OT, OPP y OID, que son de **gestión de procesos**, y le asigna la sigla RD a "Gestión de requerimientos", que es REQM.

6.5 Nivel 1 y nivel 5 frente al mismo proyecto

Un ejercicio de clase muestra qué cambia de verdad entre niveles. Dos software factories, una de nivel 1 y otra de nivel 5, reciben el mismo encargo y sufren el mismo imprevisto: el servidor llega diez días tarde y el cliente pide cambiar una regla de negocio. La de **nivel 1** estima a ojo porque nunca midió nada, hace horas extra y se saltea las pruebas de sistema —**abandona el proceso en la crisis** y se salva, si se salva, por heroicidad—, acuerda el cambio por teléfono y, al cierre, lo aprendido se va con la gente. La de **nivel 5** estima con su repositorio de medición, ya tenía el retraso del servidor registrado como riesgo con su contingencia, hace entrar el cambio por gestión de requerimientos y de configuración y, al cierre, busca las **causas comunes** de los desvíos. **El proceso no se abandona: se ejecuta.**

Lo que se olvida son las similitudes. Las dos **pueden entregar software que funciona** —el modelo dice que las de nivel 1 a menudo lo hacen, sólo que tarde, caro y sin poder repetirlo—, hacen las mismas actividades técnicas, pueden tener gente excelente, enfrentan los mismos riesgos y **las dos pueden fallar**: el nivel 5 no garantiza cero defectos. La diferencia está en **si la organización sabe por qué el proyecto salió como salió y si el próximo va a salir mejor.**

7. La gestión de procesos de la organización

Tres áreas de nivel 3 se ocupan de los procesos de la organización como un todo, y son las que más se confunden entre sí porque sus nombres se parecen. Son las **básicas** de la categoría de gestión de procesos (OPF y OID son las avanzadas). La forma más rápida de separarlas es por el verbo que las define.

OPF diagnostica y despliega. OPD define y guarda. OT capacita.

Las tres trabajan **para toda la organización**, no para un proyecto: cuando un enunciado plantea un problema de un proyecto puntual, difícilmente la respuesta sea una de ellas (capítulo 24). Lo que hay que dominar de cada una es el **propósito** y sus **metas y prácticas específicas**, que se listan a continuación con el texto oficial. Las subprácticas no se toman como tales, pero son las que justifican las respuestas en los ejercicios de discriminación.

7.1 OPF – Enfoque en procesos de la organización

Propósito: planificar, implementar y desplegar las mejoras de procesos de la organización, basadas en una comprensión completa de las **fortalezas y debilidades actuales** de los procesos y de los activos de proceso.

Es el área del **diagnóstico y la mejora**. No es un proyecto: busca elevar la madurez de toda la organización trabajando sobre sus activos de proceso. Evalúa dónde está parada la organización —incluso comparándose con otras—, arma los planes de acción para corregir las debilidades que encuentra, y después despliega los cambios e incorpora lo aprendido. Sus tres metas siguen exactamente ese recorrido:

- **SG 1 Determinar las oportunidades de mejora de procesos.** SP 1.1 Establecer las necesidades de procesos de la organización · SP 1.2 **Evaluar los procesos** de la organización · SP 1.3 Identificar las mejoras de procesos.
- **SG 2 Planificar e implementar las mejoras de procesos.** SP 2.1 **Establecer planes de acción de procesos** · SP 2.2 Implementar los planes de acción de procesos.
- **SG 3 Desplegar los activos de proceso e incorporar las lecciones aprendidas.** SP 3.1 Desplegar los activos de proceso · SP 3.2 Desplegar los procesos estándar · SP 3.3 Monitorizar la implementación · SP 3.4 **Incorporar las experiencias relativas al proceso en los activos de proceso** de la organización.

De dónde salen las mejoras. De la medición de los procesos, de las **lecciones aprendidas**, de las evaluaciones de procesos y de productos, de la **comparación con otras organizaciones** (benchmarking) y de recomendaciones de otras iniciativas. Una guía de resolución de alumnos distingue la **revisión activa**, en la que OPF recorre los proyectos y observa cómo trabajan, de la **revisión pasiva**, que parte de las lecciones aprendidas de cada proyecto y de los **informes de no conformidad** de PPQA.

Una cadena de planes. El **plan de mejora de procesos** da el marco general; el **plan de evaluación** fija alcance, recursos y el **modelo de referencia** contra el que se evalúa; de la evaluación sale el **plan de acción de procesos**, que dice cómo se atacan las debilidades encontradas; si la mejora se prueba primero en un grupo acotado hay un **plan piloto**; y el **plan de despliegue** dice cuándo y cómo llega a toda la organización. El modelo advierte que la aceptación que se gana durante una evaluación **se deteriora rápido si no la sigue un plan de acción**.

ATENCIÓN SG 2 frente a SG 3 se pregunta siempre. La SG 2 trabaja con mejoras **todavía no aprobadas**: se planifican y se **prueban en proyectos piloto**. La SG 3 **despliega** a toda la organización una mejora **ya aceptada**. Buscá en el enunciado si se habla de una **propuesta** ("qué proyectos se precisan para verificar la efectividad de la propuesta" → SP 2.2) o de una **modificación ya introducida** ("qué proyectos en curso deben incorporar la reciente modificación" → SP 3.2).

Su lugar entre PPQA y OPD. PPQA **evalúa e informa**: emite no conformidades y tendencias de calidad. OPF **analiza** esos informes y propone la mejora; si varios informes dicen que los proyectos completan mal la plantilla de casos de prueba, OPF puede proponer una guía de llenado. OPD **deja asentada** la mejora en el activo. El **sector de SQA** de una empresa suele hacer las dos primeras cosas a la vez, así que no hay que confundir el departamento con el área de proceso.

7.2 OPD – Definición de procesos de la organización

Propósito: establecer y mantener un conjunto **usable** de **activos de proceso** de la organización y de **estándares del entorno de trabajo**.

Es el área que **define y guarda**. Todo lo que la organización tiene para que sus proyectos lo reutilicen —el proceso estándar, los modelos de ciclo de vida, las reglas para adaptarlos, el repositorio de mediciones, la biblioteca de documentos— lo establece OPD. Tiene una sola meta que entra en la materia, y cada práctica crea uno de esos activos:

- **SG 1 Establecer los activos de proceso de la organización.** SP 1.1 Establecer los **procesos estándar** · SP 1.2 Establecer las descripciones de los **modelos de ciclo de vida** · SP 1.3 Establecer los **criterios y las guías de adaptación** · SP 1.4 Establecer el **repositorio de medición** de la organización · SP 1.5 Establecer la **biblioteca de activos de proceso** · SP 1.6 Establecer los **estándares del entorno de trabajo**.

La segunda meta, *facilitar la gestión IPPD*, pertenece a la extensión IPPD y no entra.

Los activos se organizan en una jerarquía: el **conjunto de procesos estándar** contiene **procesos estándar**, que se descomponen en **elementos de proceso** —la unidad atómica—, conectados según una **arquitectura de proceso**. Un elemento de proceso bien descrito incluye roles, estándares, entradas y salidas, medidas a recoger y, sobre todo, **criterios de entrada y de salida**.

Las demás prácticas tienen cada una un rasgo que la delata en un enunciado.

SP 1.2, modelos de ciclo de vida. La organización aprueba los ciclos de vida que sus proyectos pueden usar. Un resumen de alumnos da el ejemplo de tres modelos aprobados: cascada para contratos fijos, incremental para desarrollos en evolución, ágil para mantenimiento y mejora continua. La frase que aparece en los enunciados es la definición misma de ciclo de vida: las fases por las que pasa un sistema **desde que nace la idea hasta que el software es retirado o reemplazado**.

SP 1.3, criterios y guías de adaptación. Dicen cómo se arma el proceso de un proyecto a partir del estándar: qué es obligatorio, qué opciones hay y con qué criterio se elige, y cómo se documenta la adaptación. Tienen que equilibrar **flexibilidad**, para adaptarse a cada contexto, con **consistencia**, para que se respeten los estándares de la organización. Es la práctica de las **excepciones**: escribir el procedimiento para pedir, con motivos justificados, que un proyecto no incluya testing automatizado es OPD SP 1.3.

SP 1.4 y SP 1.5, repositorio y biblioteca. Son dos cosas distintas y se preguntan por separado:

	REPOSITORIO DE MEDICIÓN (SP 1.4)	BIBLIOTECA DE ACTIVOS DE PROCESO (SP 1.5)
Qué guarda	Medidas de producto y de proceso relacionadas con los procesos estándar, más la información para interpretarlas	Documentos para el personal y los proyectos: políticas, descripciones de procesos, procedimientos, plantillas, checklists, material de formación, lecciones aprendidas, modelos de estimación
Para qué sirve	Estimar y comparar el desempeño entre proyectos	Compartir las mejores prácticas, dar soporte al aprendizaje y a la mejora
Cómo aparece en un enunciado	"La estructura de la base de datos donde se registrarán métricas de todos los proyectos"	"El sitio de Intranet donde se consultan los documentos del proceso"

ATENCIÓN Cuando un enunciado menciona un **repositorio de medidas ligado a los procesos estándar de la organización**, la respuesta es **OPD** (SP 1.4), no MA. MA es de nivel 2 y especifica qué se mide y cómo se recogen los datos; el repositorio de la organización es un **activo**, y los activos son de OPD. Unas notas de práctica lo resumen así: "OPD establece estándares, no métricas"; si junta medidas, es para conocer el desempeño de los procesos.

SP 1.6, estándares del entorno de trabajo. Fijan el hardware, el software y las herramientas comunes —por ejemplo, que todos los desarrolladores usen el mismo sistema operativo, el mismo editor y el mismo servidor de versiones— y el proceso para **solicitar y aprobar excepciones**. Permiten aprovechar formación y mantenimiento comunes y ahorrar por volumen de compra.

ATENCIÓN No toda revisión entre pares es VER. Varias prácticas de OPD tienen como subpráctica **revisar entre pares** el activo que se está definiendo: el proceso estándar, los ciclos de vida, las guías de adaptación, las definiciones de medidas. Si tres jefes de proyecto se reúnen a buscar inconsistencias en un **procedimiento aún no publicado** que redactó otro, la respuesta es **OPD SP 1.1**: se revisa un activo de la organización, no un producto de un proyecto.

7.3 OT – Formación organizativa

Propósito: desarrollar las **habilidades y el conocimiento** de las personas para que puedan realizar sus **roles** eficaz y eficientemente.

Primero se construye la capacidad de formar y después se forma. Un punto que suele preguntarse es el reparto de responsabilidades: la organización cubre las necesidades de formación **comunes** a los proyectos, y cada proyecto cubre las **específicas** suyas. Un resumen de alumnos lo ilustra: la seguridad de la información o el uso de las herramientas de trabajo de todos son necesidades de la organización; un framework puntual que usa un solo proyecto, del proyecto. Que la organización cubra una necesidad particular de un proyecto es posible, pero tiene que **acordarse** con él. Las necesidades **estratégicas** miran entre **dos y cinco años** hacia adelante, para introducir tecnologías nuevas o cambios organizativos importantes; el **plan táctico** baja eso a cursos concretos.

- **SG 1 Establecer una capacidad de formación organizativa.** SP 1.1 Establecer las necesidades de formación **estratégicas** · SP 1.2 Determinar qué necesidades de formación son **responsabilidad de la organización** · SP 1.3 Establecer un **plan táctico** de formación · SP 1.4 Establecer la capacidad de formación.
- **SG 2 Proporcionar la formación necesaria.** SP 2.1 Impartir la formación · SP 2.2 Establecer los **registros** de formación · SP 2.3 Evaluar la **eficacia** de la formación.

Las habilidades que cubre son de tres tipos: **técnicas** (usar equipos, herramientas, datos y procesos), **de la organización** (actuar según la estructura, el rol y los métodos de la empresa) y **de contexto** (autogestión, comunicación, relaciones interpersonales).

ATENCIÓN OT no es PP SP 2.5. OT define capacidades y forma para **la organización**: los roles del proceso estándar, las técnicas que usan todos. Elegir qué cursos tienen que hacer los programadores **del proyecto X** antes de empezar es **PP SP 2.5** Planificar el conocimiento y las habilidades necesarios. En cambio, conseguir un instructor, o mandar a alguien a formarse para que después capacite a otros, sí es OT (SP 1.4), aunque lo aprovechen los proyectos.

7.4 Las prácticas que se cruzan entre OPF y OPD

En los exámenes aparece la misma lista de prácticas preguntada dos veces, una pidiendo las de OPF y otra las de OPD. La regla para no mezclarlas: si la práctica **crea** un activo, es OPD; si **evalúa, planifica, despliega o actualiza** lo que ya existe, es OPF.

PRÁCTICA	ÁREA
Establecer el repositorio de medición de la organización	OPD
Establecer las descripciones de los modelos de ciclo de vida	OPD
Establecer los criterios y las guías de adaptación	OPD
Establecer la biblioteca de activos de proceso	OPD
Evaluar los procesos de la organización	OPF
Establecer planes de acción de procesos	OPF
Desplegar los activos de proceso	OPF
Incorporar las experiencias relativas al proceso en los activos de proceso	OPF

8. Activos de proceso y productos de trabajo

Esta distinción es corta de explicar y sorprendentemente fácil de errar, porque los dos conceptos se refieren a documentos que a veces tienen nombres casi idénticos.

Un **activo de proceso** es un artefacto relativo a la **descripción, implementación y mejora** de los procesos —políticas, descripciones de procesos, plantillas, medidas, herramientas de soporte— que la organización mantiene **para ser usado en las tareas de los proyectos**. Un **producto de trabajo** es el **resultado útil** de un proceso —un documento, un fichero, una especificación, una parte del producto— obtenido en una tarea **de un proyecto**.

Se llaman activos porque son **inversiones**: la organización espera que le den valor hoy y en el futuro. Un producto de trabajo, en cambio, no tiene por qué formar parte del producto final; el que sí se entrega al cliente o al usuario se llama, a secas, **producto**.

La pregunta que resuelve cualquier caso es: *¿esto existe para ser usado en muchos proyectos, o salió de uno?*

Las señales de **activo** son las palabras plantilla, guía, directriz, modelo, estándar, política y checklist. También son activos los procesos estándar y los modelos de ciclo de vida documentados, las guías de adaptación, las lecciones aprendidas, los datos históricos de desempeño de los proyectos y los resultados de las evaluaciones. Y lo

son las **herramientas que la organización adquiere** en lugar de desarrollar, junto con su documentación: el manual de usuario del compilador Java es un activo, porque no es producto de ningún proyecto.

Las señales de **producto de trabajo** son las referencias a un proyecto concreto: "del cliente", "del sistema desarrollado". Una minuta, el código fuente, una regla de negocio, la descripción del proceso de negocio del cliente.

El par que fija el criterio es este: la **plantilla de manual de instalación** es un activo —es el molde, y pertenece a la organización—, mientras que el **manual de instalación del sistema desarrollado para el cliente** es un producto de trabajo —es lo moldeado, y pertenece al proyecto.

Unas notas de práctica lo dicen en una línea: **no es lo mismo decir "caso de uso" que "plantilla de caso de uso"**. El caso de uso es un producto de trabajo: lo verifica **VER** contra lo especificado, lo valida **VAL** con el cliente, y **PPQA** controla que respete el estándar. La plantilla es un activo: la define y guarda **OPD**, la mejora y la despliega **OPF**, y **VER nunca la controla**, porque VER trabaja sobre productos de trabajo, no sobre activos de la organización.

ATENCIÓN Pregunta trampa: "¿qué área se encarga de verificar y almacenar los **productos de trabajo** a nivel organizacional?" La respuesta es **NINGUNA**. "Verificar", "almacenar" y "nivel organizacional" empujan hacia OPD, pero los productos de trabajo salen de cada proyecto; si la pregunta dijera **activos**, sí sería OPD.

Dos detalles más. Que algo sea un activo no impide ponerlo además bajo gestión de configuración: el compilador con que se construyó una versión es un elemento de configuración si hace falta para reconstruirla (capítulo 20). Y cuando un activo cambia, el cambio se documenta, para **comunicarlo** y para poder entender después cómo influyó en el rendimiento de los procesos.

9. El proceso de desarrollo: SPEM y RUP

Los capítulos anteriores hablaron de procesos en abstracto: que hay que definirlos, guardarlos y mejorarlos. Este capítulo baja a cómo se **escribe** un proceso concreto: con qué elementos se describe, quién hace qué, y qué material de apoyo lo acompaña.

9.1 Por qué gestionar procesos

Las organizaciones concentran su mejora en tres **dimensiones críticas**: las **personas**, los **métodos y procedimientos**, y las **herramientas y el equipamiento**. Los procesos son lo que sostiene a las tres: permiten alinear el modo de operar de la organización, incorporar el conocimiento sobre cómo hacer mejor las cosas, aprovechar mejor los recursos y entender las tendencias de la propia actividad.

Las áreas de **gestión de procesos** de CMMI contienen las actividades **transversales a los proyectos**: definir, planificar, desplegar, implementar, monitorizar, controlar, evaluar, medir y mejorar los procesos, en un ciclo que vuelve a empezar. Se dividen en **básicas** —OPF, OPD y OT, las del capítulo 7— y **avanzadas** —OPP y OID—.

9.2 SPEM

SPEM (*Software Process Engineering Meta-Model*): estándar de la OMG —el mismo consorcio que mantiene UML— que establece los elementos clave para representar métodos, ciclos de vida, técnicas, roles, actividades, procesos, metodologías y plantillas de la ingeniería del software.

Es un **meta-modelo**: no describe un proceso en particular, sino el vocabulario con el que se describe cualquier proceso. Por eso su alcance se limita a los **elementos mínimos** necesarios, sin características de un dominio o disciplina en particular, y sirve para procesos de distintos estilos, culturas, niveles de formalismo y ciclos de vida. No es un lenguaje de modelado de procesos en general: está orientado al software.

Lo que aporta es facilitar la **comprensión y la comunicación** entre personas, facilitar la **reutilización**, dar soporte a la **mejora** y a la **gestión** de procesos, guiar su **automatización** y dar soporte a su **ejecución automática**.

ATENCIÓN Un resumen de alumnos llama a SPEM "lenguaje universal de modelado de procesos". Según la presentación de la cátedra, "SPEM es un lenguaje de modelado de procesos en general" es **falso**.

La idea central es que todo proceso se representa respondiendo tres preguntas:

PREGUNTA	ELEMENTO	QUÉ REPRESENTA
¿Quién?	Rol	Quién hace el trabajo
¿Qué?	Producto de trabajo	Las entradas que usan las tareas y las salidas que producen
¿Cómo?	Tarea	El esfuerzo a realizar

Además, SPEM define cuatro **niveles de detalle** para representar ese esfuerzo, de mayor a menor. El **delivery process** es un proceso completo, tan complejo como se necesite, que sirve de base para un tipo de proyecto. El **capability pattern** es un fragmento de proceso **reutilizable** más de una vez dentro de un delivery process. La **actividad** es el elemento central, que organiza roles, productos de trabajo y tareas. Y la **tarea** es la **unidad elemental** de trabajo del modelo, la porción más pequeña.

Un ejemplo recorre los cuatro niveles: el ciclo de vida típico de RUP contiene la disciplina *Entorno*, que contiene la actividad *Preparar el entorno para el proyecto*, que contiene la tarea *Personalizar el proceso de desarrollo para el proyecto*.

9.3 El Proceso Unificado y RUP

Un resumen de alumnos describe el **Proceso Unificado** (UP) como una estructura genérica de tres piezas. Las **disciplinas** son el contenido del método: todo lo que hay que hacer. Las **fases** reparten el esfuerzo en el tiempo. Y las **iteraciones** son tramos con fecha de inicio y de fin **inamovibles** y objetivos propios; al terminar cada una se evalúa si se alcanzaron y se realimenta la siguiente. En **cada iteración** se hacen actividades de **todas** las disciplinas, pero el esfuerzo que recibe cada disciplina depende de la fase en la que se está.

RUP (*Rational Unified Process*) es el UP de Rational, hoy de IBM: un proceso **iterativo e incremental, adaptable y no prescriptivo**, construido sobre SPEM, que trae roles, actividades, plantillas y guías prearmadas. Según la guía de INTECO, cada fase termina en un **hito** que indica que se logró su objetivo, y cada iteración termina con un prototipo, una versión ejecutable del sistema.

9.4 Los elementos del proceso en RUP

Fase. El ciclo de vida se descompone en fases, y cada fase es un **período de tiempo entre dos objetivos importantes**. RUP tiene cuatro: **Concepción** (o Inicial), **Elaboración**, **Construcción** y **Transición**.

Disciplina. Una **categorización de tareas** según la similitud de sus preocupaciones y la cooperación del esfuerzo. RUP tiene nueve: modelado de negocio, requisitos, análisis y diseño, implementación, prueba, despliegue,

configuración y gestión de cambios, gestión de proyectos, y entorno. La guía de INTECO agrupa las seis primeras como disciplinas **de ingeniería** y las tres últimas como disciplinas **de soporte**.

Las fases y las disciplinas son dos ejes distintos: las fases **ordenan el tiempo**, las disciplinas **agrupan el tipo de trabajo**. Por eso se cruzan, y a lo largo de las fases se trabaja en varias disciplinas a la vez.

Actividad. Agrupa lógicamente elementos de proceso relacionados y permite anidarlos; puede contener referencias a tareas, roles y productos de trabajo. **Una disciplina tiene una o más actividades, y una actividad tiene una o más tareas.**

Tarea. Describe una **unidad de trabajo**. La llevan a cabo **roles específicos**, dura entre **unas horas y unos días**, suele afectar a uno o pocos productos de trabajo y puede desglosarse en **pasos**. Una tarea bien descripta reúne el **rol responsable**, los **productos de trabajo de entrada y de salida**, y las guías que la apoyan: **directrices, plantillas y listas de comprobación**.

Ejemplo: la tarea *Desarrollar la visión* la ejecuta el **analista de sistemas**; toma como entrada las **solicitudes del interesado** y produce como salida la **visión**. Sus pasos son acordar el problema, identificar a los interesados, definir los límites del sistema, identificar las restricciones, formular el problema y definir las características del sistema. La apoyan la directriz *Entrevista*, la plantilla *Visión* y la lista de comprobación *Visión*.

Rol. Un conjunto de **habilidades, competencias y responsabilidades** relacionadas: analista de sistemas, arquitecto de software, diseñador, revisor técnico.

Producto de trabajo. Un **resultado significativo del proceso**: los roles los usan para realizar tareas y los producen al realizarlas. Hay tres tipos:

TIPO	QUÉ ES
Artefacto	Un producto tangible , no trivial: un documento, un modelo, el código
Resultado	Un producto intangible : un estado o una consecuencia del trabajo
Entregable	Un empaquetado de otros productos de trabajo, que se entrega a una parte interna o externa

9.5 Las guías

Una **guía** es todo contenido cuyo objetivo principal es **explicar otros elementos** del proceso. La presentación de la cátedra desarrolla ocho tipos, y los resúmenes de alumnos agregan un noveno, los **materiales de soporte**:

GUÍA	QUÉ ES
Plantilla	La estructura de un producto de trabajo: secciones, formato y cómo completarlas
Directriz	Indicaciones sobre cómo hacer algo concreto; se aplica a tareas y productos de trabajo
Lista de comprobación	Elementos que deben completarse o verificarse ; se usa en revisiones e inspecciones
Ejemplo	Una instancia parcialmente completa de un producto, para mostrar cómo queda
Concepto	Una idea fundamental , más general que una directriz
Guía de herramientas	Cómo usar una herramienta específica para producir parte de un producto
Documentación	Documentos publicados fuera de RUP a los que el proceso hace referencia
Informe	Un resultado generado automáticamente por una herramienta a partir de otros productos

Las guías se conectan con el capítulo anterior: una plantilla, una directriz o una lista de comprobación de la organización son **activos**, y el documento que se produce al usarlas en un proyecto es un **producto de trabajo**.

9.6 Las fases y las disciplinas por dentro

Para el parcial de Regularización quedaban afuera los objetivos de cada fase y los propósitos de cada disciplina; el AD toma el temario completo, así que conviene tenerlos a mano.

FASE	OBJETIVOS
Concepción (Inicial)	Fijar el ámbito y los límites del producto —incluido lo que no debe contener— y los criterios de aceptación · identificar los casos de uso más importantes · proponer una arquitectura posible · estimar el coste global y la planificación · estimar los riesgos · preparar el entorno de soporte
Elaboración	Lograr que arquitectura, requisitos y planes sean estables y que los riesgos estén mitigados, para poder fijar coste y fecha · tratar los riesgos arquitectónicamente significativos · demostrar que la arquitectura soporta los requisitos a un costo y un plazo razonables
Construcción	Completar el análisis, diseño, desarrollo y prueba de toda la funcionalidad · minimizar costes y lograr la calidad adecuada · obtener versiones útiles (alfa, beta) · decidir si software, sitios y usuarios están listos para el despliegue
Transición	Prueba beta contra las expectativas del usuario, con operación en paralelo junto al sistema anterior · convertir las bases de datos · formar a los usuarios · desplegar · corregir defectos y ajustar rendimiento y usabilidad · lograr que el usuario sea autosuficiente

DISCIPLINA	PROPÓSITO
Modelado de negocio	Entender la organización y sus problemas actuales, identificar mejoras y obtener los requisitos que salen de ahí
Requisitos	Acordar con el cliente qué debe hacer el sistema y qué no ; definir sus límites ; dar la base para planificar las iteraciones y estimar
Análisis y diseño	Transformar los requisitos en un diseño y hacer evolucionar la arquitectura
Implementación	Organizar el código en subsistemas, implementarlo, probar los componentes como unidades e integrarlos
Prueba	Buscar y documentar defectos; comprobar que el software funciona según lo diseñado y que los requisitos se implementaron
Despliegue	Poner el producto a disposición de los usuarios
Configuración y gestión de cambios	Controlar los productos de trabajo y evitar los problemas de actualización simultánea, notificación limitada y versiones múltiples
Gestión de proyectos	Dar el marco para gestionar el proyecto y sus riesgos
Entorno	Proveer al equipo los procesos y las herramientas de trabajo

Las cuatro fases permiten presentar RUP a alto nivel como si fuera una cascada, pero la clave está en las **iteraciones** de cada fase: cada una es un ciclo de desarrollo completo que termina en una **versión ejecutable**, así que el usuario ve resultados desde temprano y no recién al final. Las iteraciones se agrupan por tiempo más que por funcionalidad, y analistas y arquitectos trabajan **una iteración por delante** de desarrolladores y testers. Los ciclos de vida en general se tratan en el capítulo 13.

10. El proyecto y su planificación

Todo proyecto de software arranca con una promesa: un producto, para una fecha, por un costo. Cumplirla es difícil porque el terreno se mueve —el cliente pide algo nuevo, alguien se enferma, una estimación resulta optimista—, y la gestión de proyectos existe para que esos imprevistos se manejen con método y no a pulmón.

CMMI le dedica dos áreas de **nivel 2** que son las dos caras de la misma tarea: **PP** arma el plan y **PMC** lo sigue y lo corrige. La guía práctica de gestión de proyectos de INTECO detalla los procesos que componen esa gestión.

10.1 Qué es un proyecto

Proyecto: conjunto de actividades coordinadas y controladas, con **inicio y fin definidos**, que crea un producto o servicio **único** conforme a requisitos específicos, dentro de límites de tiempo, coste y recursos. Se desarrolla **en pasos**, en lo que se llama elaboración gradual.

De la definición salen las características que le reconoce la guía: puede ser **largo** y estar sujeto a influencias externas e internas, tiene **restricciones de coste y recursos**, conlleva **riesgo e incertidumbre**, crea **entregables únicos**, se define en forma general al comienzo y se precisa a medida que avanza, y tiene **duración limitada**: termina cuando se **logran los objetivos** o cuando se **cancela**, porque no pueden alcanzarse o porque la necesidad dejó de existir.

Tres precisiones. Un proyecto **siempre tiene fin**: lo que no termina no es un proyecto sino una operación. Los proyectos de software **no son sólo de desarrollo** (10.2). Y tener un diagrama de Gantt **no es tener un plan de proyecto**: el Gantt es una vista del cronograma, mientras que el plan incluye alcance, estimaciones, recursos, riesgos, calidad y comunicación.

La contracara del proyecto es el **proceso**, que es **repetitivo y reiterativo y produce siempre el mismo producto**: es genérico, ordena etapas sin fechas y no tiene incertidumbre. El proyecto es concreto —una instancia-ción de procesos—, tiene fechas, es único y trabaja con incertidumbre y restricciones. Un producto, además, pasa por muchos proyectos: cada actualización durante su operación es una idea nueva y, por lo tanto, otro proyecto.

Gestión de proyectos: aplicación de **conocimientos, habilidades, herramientas y técnicas** a las actividades del proyecto para satisfacer sus requisitos.

Gestionar **no elimina** los problemas, los riesgos ni las sorpresas: cambia cómo se manejan cuando aparecen, porque hay un proceso para las contingencias. Por eso el tiempo dedicado a gestionar **nunca es una pérdida**. Si muchas organizaciones igual no lo hacen, es por la **inversión inicial**, por **falta de compromiso y conocimiento**, o por la **aversión al control** del propio equipo.

Entre los participantes conviene distinguir al **patrocinador**, que aporta los **recursos financieros** —en dinero o en especie—, del **cliente o usuario**, que es quien va a usar el producto. Completan la lista el director del proyecto, el equipo, el equipo de dirección y los **influyentes**, que no están directamente relacionados con el proyecto pero pueden influir a favor o en contra.

10.2 Tipos de proyecto y de mantenimiento

La clase distingue tres tipos de proyecto: **desarrollo, mantenimiento y despliegue o implantación**, que algunos apuntes de práctica llaman **operación**. La clasificación cambia qué tareas entran en la EDT y cómo se estima: un mantenimiento correctivo de dos semanas no se planifica como un desarrollo nuevo.

Los cuatro tipos de mantenimiento se vieron en el capítulo 3; los ejercicios de gestión piden reconocerlos en un enunciado:

TIPO	QUÉ CUBRE	SEÑAL EN EL ENUNCIADO
Correctivo	Corregir errores	"Se detectaron errores", "falla"
Evolutivo	Altas, bajas y modificaciones por la expansión o el cambio de las necesidades del usuario	Funcionalidad nueva, un módulo nuevo , ajustes pedidos por el cliente
Adaptativo	Cambios en el entorno donde opera: hardware, software de base, gestor de base de datos, comunicaciones	Cambio de plataforma o de arquitectura , distribución geográfica
Perfectivo	Mejorar la calidad interna	Refactorizar u optimizar sin cambiar la funcionalidad

ATENCIÓN Perfectivo y evolutivo no son sinónimos: el evolutivo cambia **lo que el sistema hace**; el perfectivo mejora **cómo está hecho**. Hay además un caso discutido, **traducir el sistema a otro idioma**: dos resoluciones de finales lo clasifican como **adaptativo**, pero también se puede defender como **evolutivo**, porque el adaptativo, por definición, toca el entorno técnico. Si aparece, conviene justificar la elección.

10.3 Los procesos de gestión de proyectos

La guía práctica de INTECO organiza la gestión en **diez grupos de procesos**. Son **buenas prácticas** —hay acuerdo general en que aumentan las posibilidades de éxito—, pero **no todos tienen que estar presentes** en cada proyecto: las características del proyecto y de la organización deciden cuáles se incluyen. Conviene saber a qué grupo pertenece cada proceso, porque los ejercicios juegan con procesos que suenan a un grupo y pertenecen a otro.

GRUPO	PROCESOS
Coordinación	Iniciar el proyecto · Desarrollar el plan · Gestionar la ejecución · Supervisar el trabajo · Control integrado de cambios · Cerrar el proyecto
Alcance	Definir el alcance · Definir las actividades · Verificar y controlar el alcance
Tiempo	Establecer la secuencia de actividades · Estimar la duración · Desarrollar el cronograma · Controlar el cronograma
Costes	Estimar los costes · Elaborar los presupuestos · Controlar los costes
Calidad	Planificar la calidad · Realizar el aseguramiento de calidad · Realizar el control de calidad
Recursos	Planificar los recursos · Controlar los recursos
Personal	Definir el equipo del proyecto · Gestionar el equipo
Comunicación	Planificar las comunicaciones · Gestionar la información y los interesados
Riesgos	Planificar la gestión · Identificar · Analizar · Planificar la respuesta · Controlar
Adquisiciones	Planificar las adquisiciones · Planificar la contratación · Solicitar respuesta a proveedores · Seleccionar proveedores · Administrar el contrato · Cerrar el contrato

Lo que se pregunta son los detalles. **Definir las actividades** pertenece a **Alcance**, no a Tiempo, aunque suene a cronograma; el grupo de Alcance busca que el proyecto incluya **todo el trabajo requerido y sólo ese**, y su

enunciado del alcance ya trae criterios de aceptación, riesgos iniciales e hitos. *Iniciar el proyecto* es su **definición, autorización y apertura formal**, y *cerrarlo* vale tanto para el proyecto **completado** como para el **cancelado**. El *control integrado de cambios* mantiene al día plan, alcance y entregables **aprobandos o rechazando** cambios.

En Tiempo, *estimar la duración* parte del **esfuerzo** y de la **cantidad de recursos** aplicados: no son lo mismo (10.6). *Desarrollar el cronograma* produce la **línea base** contra la que se mide el avance, y debe identificar explícitamente el **camino crítico** —el de mayor duración en la red de actividades—, las actividades críticas y los **hitos**. En Costes, *elaborar los presupuestos* debe incluir las **reservas para contingencias de gestión**, previstas para cambios no planificados (capítulo 12). En Calidad, el **aseguramiento** es un conjunto de actividades **planificadas y sistemáticas** que mira el proceso, y el **control supervisa los resultados** contra las normas.

Definir el equipo del proyecto consiste en determinar **roles y responsabilidades**, para que cada actividad tenga un **propietario no ambiguo**; no incluye estimar el esfuerzo por rol ni controlar el cronograma. *Planificar las comunicaciones* responde **quién** necesita **qué** información, **cuándo**, **cómo** y **por quién**. El grupo de Riesgos busca **aumentar** la probabilidad y el impacto de los eventos **positivos** y **disminuir** los de los adversos. Y en Adquisiciones los proveedores se eligen con **criterios ponderados** definidos de antemano, entre los que la guía cuenta la **permanencia del proveedor**.

Estos diez grupos son una agrupación **propia de INTECO**: PMBOK tiene **nueve áreas de conocimiento** y grupos de procesos de inicio, planificación, ejecución, control y cierre. La guía menciona además **ISO 10006**, que define los procesos pero no las técnicas; **MÉTRICA V3**, que trata la gestión de proyectos como una interfaz; y **PRINCE2**, orientado a productos.

10.4 PP – Planificación de proyecto

Su propósito es **establecer y mantener los planes** que definen las actividades del proyecto: desarrollar el plan, interactuar con las partes interesadas, **obtener el compromiso** con él y mantenerlo. Arranca **con los requerimientos**, que definen producto y proyecto. Y **el plan necesitará corregirse**: cambian los requerimientos y los compromisos, y las estimaciones resultan inexactas. Eso no es un fracaso de la planificación; PP cubre también la **replanificación**.

META	PRÁCTICA	CÓMO SE RECONOCE
SG 1 Establecer estimaciones	SP 1.1 Estimar el alcance del proyecto	EDT de alto nivel, paquetes de trabajo
	SP 1.2 Establecer las estimaciones de los atributos de los productos de trabajo y de las tareas	Tamaño y complejidad: puntos función, líneas de código, cantidad de requerimientos
	SP 1.3 Definir el ciclo de vida del proyecto	Fases del proyecto: por ejemplo, encuadrar las tareas en Inicio, Elaboración, Construcción y Transición
	SP 1.4 Determinar las estimaciones de esfuerzo y de coste	Horas y coste a partir del tamaño, con datos históricos o juicio de expertos
SG 2 Desarrollar un plan de proyecto	SP 2.1 Establecer el presupuesto y el calendario	Cronograma, hitos, dependencias entre tareas, criterios de acción correctiva
	SP 2.2 Identificar los riesgos del proyecto	Identificar y analizar riesgos y acordarlos con los interesados

META	PRÁCTICA	CÓMO SE RECONOCE
	SP 2.3 Planificar la gestión de los datos	Qué datos del proyecto se registran (autor y aprobador de cada documento, tiempo insumido en corregir cada defecto), con qué formato y dónde
	SP 2.4 Planificar los recursos del proyecto	Personal, equipamiento e instalaciones; cantidad de personas y dedicación
	SP 2.5 Planificar el conocimiento y las habilidades necesarios	Formación para este proyecto
	SP 2.6 Planificar la involucración de las partes interesadas	Qué hace cada interesado y cuándo, cliente incluido
	SP 2.7 Establecer el plan de proyecto	El plan global que integra todo
SG 3 Obtener el compromiso con el plan	SP 3.1 Revisar los planes que afectan al proyecto	Planes de otras áreas (configuración, calidad) compatibles con el global
	SP 3.2 Reconciliar los niveles de trabajo y de recursos	Renegociar cuando lo estimado no coincide con lo disponible
	SP 3.3 Obtener el compromiso con el plan	Compromisos internos y externos documentados

ATENCIÓN La planificación **incluye** estimar las tareas, determinar los recursos e identificar los riesgos. **No incluye** el pago a los recursos ni la especificación detallada de la arquitectura del software, que corresponde a **TS** (Solución técnica).

ATENCIÓN Fijate el **alcance** del enunciado. La formación o el registro de datos planificados **para un proyecto puntual** son PP (SP 2.5 o SP 2.3), no OT ni MA, que son de alcance organizacional. Y ojo con algunos resúmenes de alumnos: numeran "SP 2.2 Planificar la gestión de los datos", pero en el CMMI de la cátedra la SP 2.2 es identificar los riesgos y la de datos es la **SP 2.3**.

La SP 2.6 decidió una pregunta BP del AD de 2025: la plantilla de plan de proyecto de una software factory pedía incluir **sólo los roles de la software factory**, más responsabilidades de OT, OPF y OPD. La correcta fue que **sí** puede generar problemas de calidad, porque los miembros del proyecto por parte del cliente no sabrían qué tienen que hacer: el plan debe tener los roles de **todas** las partes interesadas relevantes, y el cliente lo es. Sin eso, además, PMC no tiene contra qué monitorizar su participación. Lo de OT, OPF y OPD era un segundo error: son áreas organizacionales, no roles de un proyecto.

10.5 PMC – Monitorización y control del proyecto

Su propósito es comprender el progreso del proyecto para tomar **acciones correctivas apropiadas** cuando el rendimiento se desvía significativamente del plan. Se comparan la calidad de los productos de trabajo, el esfuerzo, el coste y el calendario **reales contra el plan**, en los **hitos o niveles de control** definidos en la EDT. Ante un desvío, se **replanifica**, se establecen **nuevos acuerdos** o se agregan **actividades de mitigación** al plan actual.

Desvío significativo: aquel que, si se deja sin resolver, **impide al proyecto cumplir sus objetivos**. Por eso es falso que cualquier desvío deba resolverse: el modelo pide criterio, no reacción automática.

META	PRÁCTICA	CÓMO SE RECONOCE
SG 1 Monitorizar el proyecto frente al plan	SP 1.1 Monitorizar los parámetros de planificación del proyecto	Real contra plan en calendario, coste, esfuerzo, tamaño, recursos y habilidades
	SP 1.2 Monitorizar los compromisos	Compromisos incumplidos o en riesgo
	SP 1.3 Monitorizar los riesgos del proyecto	Revisar los riesgos contra el plan y comunicar su estado
	SP 1.4 Monitorizar la gestión de los datos	Contra el plan de gestión de datos
	SP 1.5 Monitorizar la involucración de las partes interesadas	¿Asisten, responden, participan como estaba previsto?
	SP 1.6 Llevar a cabo revisiones de progreso	Revisiones periódicas del estado
	SP 1.7 Llevar a cabo revisiones de hitos	Revisiones en los hitos
SG 2 Gestionar las acciones correctivas hasta su cierre	SP 2.1 Analizar los problemas	Recoger y analizar problemas; determinar qué hacer
	SP 2.2 Llevar a cabo las acciones correctivas	Acordar y ejecutar: replanificar, renegociar, sumar recursos
	SP 2.3 Gestionar las acciones correctivas	Seguirlas hasta el cierre y evaluar si funcionaron; lecciones aprendidas

Dentro de la segunda meta, el tiempo verbal decide la práctica: "analizando la situación se concluyó que la causa es..." es **SP 2.1**; "en este momento se determinó reemplazar al usuario clave", **SP 2.2**; "el cambio se hizo hace dos semanas y ahora se evalúa si resolvió el problema", **SP 2.3**.

Los **puntos de control se definen al planificar** y se usan al ejecutar. En cada uno se arma un **informe de avance** con los **desvíos** de cronograma o presupuesto, las tareas realizadas y las previstas hasta el próximo control. Según apuntes de clase, una tarea atrasada primero se intenta **compensar** —con otra tarea, con horas extra— y, si el desfasaje se descontrola, se **replanifica**.

ATENCIÓN Para distinguir PP de PMC en un enunciado, fíjate dónde está la reunión respecto del momento narrado. Si la reunión está **en el futuro** —"para informarlo en la reunión donde se juntará por primera vez todo el equipo"— todavía se está planificando: es **PP**. Si la reunión **ya ocurrió** —"en la última reunión de avance...", "respecto de lo acordado en la reunión de lanzamiento"— se está monitorizando: es **PMC**. Todo cambio durante el proyecto es seguimiento.

10.6 El plan en los ejercicios: EDT, cronograma, hitos y puntos de control

Los ejercicios de planificación de los finales combinan EDT, cronograma, puntos de control y riesgos. En un parcial de opción múltiple no se dibuja nada, pero los criterios se preguntan sueltos. La secuencia es siempre la misma: se arma la **EDT**, se identifican las **tareas típicas** del ciclo de vida elegido y se establecen las **precedencias** entre ellas.

EDT (estructura de desglose del trabajo, o WBS): descomposición del proyecto en **paquetes de trabajo** manejables, que sirve de base para asignar esfuerzo, calendario y responsables.

Establecerla es PP SP 1.1. Es una lista indentada, o un árbol, con las **etapas** como sustantivos y las **tareas** como verbos en infinitivo y concretas: "Analizar módulo A", no "Análisis". La EDT **lista**; el orden y las dependencias van en el cronograma. Su primer nivel se elige según lo que se quiere mostrar —fases, iteraciones, módulos—, y las **adquisiciones también son tareas**. Según CMMI suele estar **orientada al producto**, con un **identificador único** por paquete, y debería dejar ver las tareas de mitigación de riesgos y las de los planes de soporte, como configuración y calidad.

Al leer una red de tareas, una **predecesora directa** es la inmediatamente anterior: si A precede a B y B precede a C, A **no** es predecesora directa de C, aunque tenga que ocurrir antes. Y los **módulos independientes arrancan en paralelo**: si un producto tiene tres módulos independientes, las tareas que inician el proyecto son tres, una de análisis por módulo.

El **camino crítico** es la cadena más larga del cronograma y fija la duración del proyecto; las demás tareas tienen **holgura**. Los recursos de un rol son el **máximo de tareas de ese rol que se superponen**: dos análisis en paralelo piden dos analistas. Aprovechar la holgura, dividir tareas o sumar y quitar recursos ajustan la duración o la carga semanal, no el esfuerzo total.

ATENCIÓN Esfuerzo y duración no son lo mismo y no se suman igual. El **esfuerzo** se mide en horas-persona o días-persona y **se suma**. La **duración** se mide en días, semanas o meses de calendario y **no se suma**: sale del camino crítico. Dos tareas de 4 y 5 semanas suman siempre 9 semanas de esfuerzo, pero duran 9 si van en secuencia y 5 si van en paralelo. Cuando un ejercicio pide esfuerzo, hay que leer la columna de esfuerzo y convertir: 1 día-persona = 8 horas.

Por eso, si piden el esfuerzo de un plan **con y sin restricciones** de precedencia, la respuesta es la misma. La única excepción es que el orden cambie el trabajo mismo, como cuando analizar un módulo sale más barato si otro ya está analizado.

Hito: evento que requiere entradas específicas o una toma de decisiones, o en el que se planea una entrega relevante.

Un hito bien redactado es **un producto terminado o una aprobación** y dice qué habilita: "Especificación de requisitos aprobada" (recién entonces se asignan recursos), "Sistema instalado y aceptado" (empieza la garantía). Los **puntos de control**, en cambio, se ubican **en función de los riesgos**: más controles donde es más probable que algo salga mal, como el análisis de un requisito poco claro; sin riesgos explícitos, según la duración, y en un proyecto mediano, semanalmente. Conviene controlar una etapa **ya avanzada y antes de que termine**, con margen para corregir, y decir **qué se controla y con qué medida**: porcentaje de casos de uso terminados, horas gastadas contra presupuesto, entorno de desarrollo instalado.

11. La estimación del esfuerzo

Estimar es decidir cuánto va a costar algo que todavía no existe. La materia presenta cuatro métodos, que no compiten entre sí: se usan en momentos distintos y con información distinta. En todos, la clase propone estimar primero un proyecto **ideal**, sin contingencias, y después ajustarlo.

El **valor esperado**, o **técnica de tres puntos**, estima cada tarea en tres escenarios —optimista, normal y pesimista— y los combina en un único valor. La ponderación más difundida le da peso cuatro al escenario normal, $(0 + 4N + P) / 6$ (conocimiento general: la clase sólo enumera los tres escenarios). Es rápida y sirve cuando la incertidumbre está acotada.

Delphi es una estimación **grupal e independiente de expertos**: cada uno vuelca su perspectiva en números sin ver la del resto. Es **iterativa**, y en cada vuelta se suma información buscando la **convergencia**. Sirve cuando no hay datos históricos pero sí gente con experiencia, y reaparece como técnica para identificar riesgos (capítulo 12).

Los **puntos de función** miden el tamaño del software desde una perspectiva funcional, independiente de la tecnología, a partir de la lista de requerimientos (11.3). Son una medida **indirecta del tamaño**, no del esfuerzo: el esfuerzo se deriva después. Los **puntos de historia**, por último, son la estimación relativa propia de las metodologías ágiles (conocimiento general: la clase los nombra pero no los desarrolla).

Un consejo que la cátedra repite: **tomar nota de las decisiones tomadas**, sobre todo en la etapa de estimación. Es lo que después permite explicar un desvío en PMC en lugar de improvisar una justificación.

11.1 Del tamaño al esfuerzo

Los puntos función, las líneas de código o la cantidad de requerimientos miden **tamaño**. Para llegar a horas hace falta un **indicador de productividad** —puntos función por mes, horas por punto función— sacado de **proyectos similares**; si la productividad está expresada por mes, también hacen falta las horas hábiles de cada mes.

ATENCIÓN **Tamaño no es esfuerzo**, y en CMMI son prácticas distintas. "Se cuentan las entradas, salidas, consultas y ficheros y se obtienen 320 PF" es **PP SP 1.2** (atributos de los productos de trabajo). "Con la productividad histórica de 12 PF por mes, el desarrollo lleva 26,7 meses-persona" es **PP SP 1.4** (esfuerzo y coste).

El histórico sirve sólo si se parece al proyecto. Un final planteaba tres proyectos anteriores con productividades de 7, 8 y 10 puntos función, donde los dos últimos eran similares entre sí y el primero no. La resolución usa **dos** indicadores: 9, el promedio de los similares, para los proyectos parecidos a ellos, y 7 para los parecidos al primero. El promedio general, 8,3, existe pero no se recomienda, porque mezcla proyectos que no se parecen.

11.2 La distribución 40-20-40

Es una regla de reparto del esfuerzo total de un proyecto: **40 % análisis y diseño** —de los cuales 10 a 15 puntos van al análisis y 25 a 30 al diseño—, **20 % codificación** y **40 % pruebas**.

La lectura que importa no es el número exacto sino su consecuencia: **codificar es apenas la quinta parte del proyecto**, y se prueba tanto como se analiza y diseña junto.

11.3 Análisis de puntos función

El APF mide el tamaño del software **entregado al usuario** desde una perspectiva funcional, independiente de la tecnología, y se puede aplicar en cualquier fase del ciclo de vida. Lo definió Allan Albrecht; la guía de la cátedra dice que en 1970.

Como los puntos función de un sistema no cambian con el lenguaje, el método ni la plataforma —la **única variable es el esfuerzo** para entregarlos—, sirven para **comparar productividad** entre herramientas, lenguajes u organizaciones, que es su uso más importante. También permiten **seguir los cambios de alcance**: si los puntos función medidos al cerrar requisitos, diseño y codificación van creciendo, hubo cambios de alcance.

La guía describe el método en **dos etapas**: primero se identifican y clasifican las funciones; después se pondera cada una según su complejidad y, al final, se ajusta el total con las características del entorno.

Identificar y clasificar. Se fija el **límite del sistema** desde el punto de vista del usuario, que determina qué queda adentro y qué es externo, y se reconocen cinco componentes: **entradas** (datos que cruzan el límite hacia adentro y pueden actualizar un fichero interno), **salidas** (datos que lo cruzan hacia afuera: informes, ficheros enviados a otras aplicaciones), **consultas** (combinación de entrada y salida para obtener datos), **ficheros lógicos internos** o FLI (datos que residen dentro de la aplicación y son actualizados por las entradas) y **ficheros de interfaz externos** o FIE (datos que residen fuera y son mantenidos por otra aplicación). Las tres primeras son **funciones de transacción**; los ficheros, **funciones de datos**.

Ponderar. La complejidad se establece por la diversidad de atributos en tipo y cantidad: en las transacciones se cruzan los **ficheros referenciados** con los **tipos de datos**; en los ficheros, los **tipos de elementos de registro** con los tipos de datos. La guía da estas tablas como ejemplo:

TRANSACCIONES: FICHEROS REFERENCIADOS	1 A 5 TIPOS DE DATOS	6 A 19	20 O MÁS
0 o 1	Baja	Baja	Media
2	Baja	Media	Alta
3 o más	Media	Alta	Alta

FICHEROS: ELEMENTOS DE REGISTRO	1 A 19 TIPOS DE DATOS	20 A 50	51 O MÁS
1	Baja	Baja	Media
2 a 5	Baja	Media	Alta
6 o más	Media	Alta	Alta

Cada función se multiplica por el peso de su complejidad, y la suma da los **PFD**, puntos función sin ajustar:

COMPONENTE	BAJA	MEDIA	ALTA
Entradas	3	4	6
Salidas	4	5	7
Consultas	3	4	6
Ficheros internos (FLI)	7	10	15
Ficheros externos (FIE)	5	7	10

Ajustar. El factor de ajuste incorpora características no funcionales del entorno. Se califican **14 características** de 0 (ninguna influencia) a 5 (influencia fuerte): comunicaciones de datos, procesamiento distribuido, rendimiento, utilización masiva, tasa de transacción, entrada de datos on-line, eficiencia para el usuario, actualización on-line, procesamiento complejo, reutilización, facilidad de instalación, facilidad de operación, puestos múltiples y facilidad de cambio. La suma de los 14 valores es el **TDI**, que va de 0 a 70:

$$\begin{aligned} \text{Factor de ajuste} &= (65 + \text{TDI}) / 100 && (\text{o sea, } 0,65 + \text{TDI}/100) \\ \text{PF ajustados} &= \text{PFD} \times \text{Factor de ajuste} \end{aligned}$$

Con el TDI entre 0 y 70, el factor va de **0,65 a 1,35**: el ajuste puede mover el tamaño hasta un 35 % para cada lado. Por ejemplo, 200 PFD con un TDI de 42 dan un factor de 1,07 y **214 PF**.

11.4 Puntos función de una mejora

La guía avanzada de puntos función se dedica sobre todo a medir **mejoras**, es decir, cambios sobre un sistema que ya existe. Hacen falta el **APF de la parte afectada** —sin los puntos función de lo que existe no se puede medir el cambio—, su documentación, una **propuesta de mejora** sin ambigüedades y un **plan de pruebas**. El método es **objetivo**, porque no depende de quién lo aplique, y **repetible**, porque da lo mismo en aplicaciones sucesivas.

Cada función afectada mide su PFD multiplicado por un **factor de impacto (FI)**. Una función **añadida** tiene FI = 1; una **eliminada**, 0,4 (borrar una función de 6 PF cuenta 2,4); una **modificada** toma su PFD posterior al cambio y un FI que depende del porcentaje de cambio, en el que cuentan todos los tipos tocados:

$$\% \text{ de cambio} = (\text{tipos añadidos} + \text{borrados} + \text{modificados}) \times 100 / \text{tipos originales}$$

En las funciones de datos alcanza con el porcentaje de tipos de datos: hasta 33 %, FI = 0,25; hasta 67 %, 0,50; hasta 100 %, 0,75; más de 100 %, 1. Si el fichero además cambia de tipo, de FLI a FIE o al revés, el FI es 0,4, y se toma el mayor. En las transacciones se cruzan dos porcentajes:

FICHEROS REFERENCIADOS CAMBIADOS	TIPOS DE DATOS HASTA 67 %	HASTA 100 %	MÁS DE 100 %
Hasta 33 %	0,25	0,50	0,75
Hasta 67 %	0,50	0,75	1,00
Hasta 100 %	0,75	1,00	1,25
Más de 100 %	1,00	1,25	1,50

El ejemplo de la guía: un informe de 5 PF muestra 16 tipos de datos; se añaden 3, se modifican 3 y se eliminan 2, o sea 8 de 16, un **50 %**. Usa 2 ficheros referenciados y cambian los dos: **100 %**. La tabla da FI = 0,75, y la mejora de esa función mide $5 \times 0,75 = \mathbf{3,75 \text{ PF}}$. Si un FI da 1 o más, quizás convenga tratar el caso como eliminar la función y crear una nueva.

Los **PFDM** —puntos función sin ajustar de la mejora— se ajustan con **dos** factores distintos:

$$\begin{aligned} \text{PFM} = & (\text{PFDM añadidos} + \text{PFDM modificados}) \times \text{FA después} \\ & + \text{PFDM borrados} \times \text{FA antes} \end{aligned}$$

ATENCIÓN El factor de ajuste de **después** de la mejora se aplica a lo **añadido y modificado**; el de **antes**, a lo **eliminado**.

Las pruebas se miden aparte, con los **PFP**: el APF estándar de las funciones del sistema **mejorado** que intervienen directamente en una prueba, sin importar qué les hizo la mejora. El rango a probar puede ser mucho mayor que el de la mejora. Con las dos medidas y la productividad histórica de cada una sale el esfuerzo:

$$\text{Esfuerzo de la mejora} = \text{PFM} \times (\text{horas por PFM}) + \text{PFP} \times (\text{horas por PFP})$$

Del glosario de la guía, que también entra, conviene retener las siglas —**PFD** (sin ajustar), **PFDM** (sin ajustar de la mejora), **PFM** (de la mejora) y **PFP** (de las pruebas)— y tres definiciones: **fichero referenciado**, un FLI leído o modificado por una transacción, o un FIE leído por ella; **factor de impacto**, el grado de cambio de una función; y **propuesta de mejora**, la petición formal con detalle suficiente para comprender el alcance y el impacto del cambio.

12. La gestión de riesgos (RSKM)

Todo proyecto convive con cosas que pueden salir mal: el único que conoce la base de datos renuncia, el proveedor entrega tarde, el cliente descubre a mitad de camino que necesitaba otra cosa. En nivel 2 el proyecto ya **identifica** riesgos al planificar (PP) y los **monitoriza** durante la ejecución (PMC), pero en general **reacciona** cuando se materializan. La gestión de riesgos es el paso siguiente: prepararse, prevenir y mitigar de forma **sistémica**, porque detectar un riesgo temprano es más fácil, más barato y menos dañino que corregir sus efectos en una fase tardía.

Propósito de RSKM: identificar los problemas potenciales **antes de que ocurran**, para que las actividades de tratamiento de riesgos puedan planificarse e invocarse según sea necesario a lo largo de la vida del producto o del proyecto, para mitigar los impactos adversos en el logro de los objetivos.

Es un proceso **continuo** que mira hacia adelante, considera fuentes **internas y externas** de riesgos de coste, calendario y rendimiento, y necesita un entorno de **divulgación libre**: se identifican riesgos, no culpables, y los datos de riesgos no sirven para evaluar a las personas.

ATENCIÓN RSKM es de **nivel 3**, de la categoría de gestión de proyectos. La carátula del área en la traducción castellana del CMMI dice "nivel de madurez 2": es una errata del mismo tipo que la de OT (capítulo 6).

12.1 Qué es un riesgo

Riesgo de un proyecto: evento o condición **incierto** que, si ocurre, tiene un efecto **positivo o negativo** sobre al menos un objetivo del proyecto: tiempo, coste, alcance o calidad.

Tiene tres componentes: un **evento** definible, su **probabilidad** y su **consecuencia** o impacto. Y la definición trae una sorpresa que se pregunta: un riesgo **también puede ser una oportunidad**. Poder usar la aplicación antes de terminar el proyecto, gracias a un desarrollo incremental, es un riesgo con impacto positivo.

La guía de INTECO toma prestado vocabulario de la seguridad: un **activo** es cualquier recurso (software, hardware, datos, personas); una **vulnerabilidad**, una debilidad interna que puede activarse; una **amenaza**, la posibilidad de que se active, que sin vulnerabilidad no plantea riesgo; el **impacto**, la materialización del riesgo sobre el activo; y una **suposición**, algo que se da por cierto sin prueba y que se analiza porque puede convertirse en riesgo.

Los riesgos se clasifican en ejes que no se excluyen: **conocidos**, que se identifican, analizan y planifican, o **desconocidos**, que no se pueden gestionar proactivamente y se cubren con una reserva; **internos**, controlables por el equipo, o **externos**, como la regulación, la inflación o una huelga. Los apuntes de práctica agregan otras clasificaciones —de proyecto, de producto y de negocio; o por tipo: personal, organizativos, herramientas, requerimientos, estimación—, que no contradicen a éstas: son ejes distintos.

12.2 Metas y prácticas

CMMI divide la gestión de riesgos en tres partes, que son sus tres metas: prepararse, identificar y analizar, y mitigar.

META	PRÁCTICA	QUÉ SE HACE
SG 1 Preparar la gestión de riesgos	SP 1.1 Determinar las fuentes y las categorías de los riesgos	Listas de fuentes internas y externas —requerimientos incompletos, esfuerzos sin precedentes, tecnología no disponible, estimaciones irreales, personal, proveedores, comunicación con el cliente— y de categorías
	SP 1.2 Definir los parámetros de los riesgos	Probabilidad, consecuencia y umbrales que disparan las actividades; criterios comunes para comparar riesgos
	SP 1.3 Establecer una estrategia de gestión de riesgos	Alcance, métodos, técnicas de mitigación (prototipos, pilotos, simulación, diseños alternativos), intervalos de revisión. Suele quedar en un plan de gestión de riesgos
SG 2 Identificar y analizar los riesgos	SP 2.1 Identificar los riesgos	Lista de riesgos con contexto, condiciones y consecuencias, recorriendo cada elemento de la EDT, las lecciones aprendidas y los proyectos similares
	SP 2.2 Evaluar, categorizar y priorizar los riesgos	Lista priorizada . Probabilidad × consecuencia = exposición . Es lo que se llama "análisis de riesgos"
SG 3 Mitigar los riesgos	SP 3.1 Desarrollar los planes de mitigación de riesgos	Para los riesgos más importantes : opciones de tratamiento, planes de mitigación y de contingencia , un responsable por riesgo
	SP 3.2 Implementar los planes de mitigación de riesgo	Monitorizar el estado e invocar el tratamiento al superarse un umbral; seguir las acciones hasta el cierre

ATENCIÓN El orden es **identificar** → **analizar** → **priorizar**. Es falso que "el análisis de riesgos es previo a su identificación": un riesgo tiene que estar identificado y descripto antes de poder analizarse. Lo que sí va antes que todo es la **preparación**: fuentes, parámetros y estrategia. Y los riesgos se identifican **desde la planificación** del proyecto.

12.3 Umbrales, mitigación y contingencia

Los **umbrales** son lo que separa "anotar el riesgo" de "actuar". CMMI da ejemplos como que los costes superen en más de un 10 % el objetivo, o que índices como el CPI o el SPI caigan por debajo de 0,95. Con ellos, un riesgo puede quedar **aceptado**, cuando es demasiado bajo para justificar una mitigación formal o no hay forma viable de reducirlo —y entonces la razón **debe documentarse**—, o **vigilado**, cuando hay límites objetivos, verificables y documentados que, al superarse, activan el plan de mitigación o invocan el de contingencia. Normalmente los umbrales de mitigación se disparan **antes** que los de contingencia, y muchas veces sólo se tratan formalmente los riesgos de prioridad alta y media.

ATENCIÓN Por eso es **falso** que "cualquier desvío en un riesgo habilita actividades de tratamiento": se actúa al **superar el umbral**. Es la misma lógica del desvío significativo de PMC (10.5).

La **mitigación** actúa **antes** de que el riesgo ocurra, para reducir su probabilidad o su impacto. La **contingencia** responde **después**, cuando el riesgo ocurre a pesar de todo, para limitar el daño. Las dos son proactivas, porque las dos se **planifican** de antemano. Un ejemplo las separa: ante el riesgo de que renuncie el único DBA, capaci-

tar desde ahora a un segundo DBA es **mitigación**; dejar previsto que, si renuncia, se contrata a una consultora con la reserva del proyecto es **contingencia**.

Para decidir qué hacer con cada riesgo, las dos fuentes de la materia usan listas que **no coinciden**, y eso se presta a trampas:

	CMMI	GUÍA DE INTECO
Opciones	Evitar (cambiar o reducir requerimientos sin dejar de cumplir las necesidades) · controlar · transferir · monitorizar · aceptar	Ante amenazas : evitar (eliminar la causa) · transferir · mitigar . Ante oportunidades : explotar · compartir · mejorar . Para ambas: aceptar
Transferir	Reasignar requerimientos para reducir el riesgo	Trasladar impacto y responsabilidad a un tercero —seguros, garantías, contratos—: el riesgo no desaparece y casi siempre se paga una prima
Aceptar	Reconocer el riesgo sin tomar ninguna acción	Pasiva , sin acción, o activa , con una reserva y un plan de contingencia

ATENCIÓN Si una opción dice que aceptar un riesgo es no hacer nada, es correcta según CMMI e incompleta según INTECO. Si dice que transferir es contratar un seguro, habla en términos de INTECO; en CMMI, transferir es reasignar requerimientos.

12.4 El proceso según INTECO

La guía práctica de gestión de riesgos baja todo esto a seis actividades, que se actualizan durante todo el proyecto. En todas, la responsabilidad final es del **jefe de proyecto**, aunque delegue.

Desarrollar el plan de gestión de riesgos. Lo hace el jefe de proyecto y describe la estrategia, el alcance, **cómo** se va a identificar, analizar, responder y monitorizar, el presupuesto, el calendario y los roles. Ojo: el plan **no contiene los planes de respuesta ni trata riesgos concretos**; dice cómo se va a gestionar, no qué se hace con cada riesgo.

Identificar los riesgos. La salida es el **registro de riesgos**, con los riesgos, sus **disparadores** y las suposiciones, y conviene que participe todo el personal. Las técnicas son la tormenta de ideas, **Delphi** —expertos anónimos, en varias rondas, sin influencias indebidas—, las entrevistas, los diagramas de afinidad y de causa-efecto, el análisis de suposiciones y las **listas de control**, que nunca son exhaustivas.

Analizar los riesgos. El análisis **cualitativo** clasifica el impacto en bajo, medio, alto o muy alto, y la probabilidad en baja (menos del 35 %, se toma 0,15), media (35 a 65 %, 0,45), alta (65 a 85 %, 0,70) y muy alta (85 % o más, 0,90); los dos se cruzan en una **matriz probabilidad-impacto** que da la prioridad. El **cuantitativo** calcula el **valor esperado** = **impacto** × **probabilidad**, en dinero o en días.

Planificar la respuesta. Para cada riesgo se fijan una estrategia, las acciones y un **propietario**, que puede ser externo al equipo. Se determinan también los **riesgos residuales** —lo que queda después de la respuesta, que se analiza como cualquier otro riesgo— y la reserva.

Controlar y monitorizar. Vigilar disparadores y residuales, descubrir riesgos nuevos, ejecutar las respuestas y evaluar si funcionaron, en las reuniones con el equipo y con el cliente, en los hitos y dentro del control de cambios. **Cerrar**, por último, es registrar las **lecciones aprendidas**, que en realidad deben capturarse durante todo el proyecto.

La guía de **gestión de proyectos** (10.3), en cambio, usa **cinco** procesos para su grupo de riesgos, sin un cierre aparte.

12.5 Las reservas

Responder a un riesgo cuesta, y la guía avanzada distingue de dónde sale ese dinero. La **reserva de contingencia** es la suma de los valores esperados de los riesgos **aceptados** y de los **residuales**, es decir, de riesgos **conocidos**. La **reserva de gestión** cubre la incertidumbre de los riesgos **desconocidos**. Y el costo de las respuestas de evitar, transferir y mitigar **no va a ninguna reserva**: va al **presupuesto del proyecto**, dentro de la EDT, porque se sabe cuánto cuesta y cuándo se gasta.

Un riesgo con probabilidad alta (0,70) y un impacto de \$20.000 tiene un valor esperado de \$14.000. Si se acepta, ese monto suma a la reserva de contingencia. Si se mitiga, el costo de la mitigación va al presupuesto, y a la reserva sólo va el valor esperado del riesgo residual.

ATENCIÓN Los nombres se cruzan entre guías. La de gestión de proyectos pide incluir en el presupuesto "reservas para contingencias **de gestión**", para cambios no planificados, que se parecen más a la reserva **de gestión** de la guía de riesgos. Si una opción las mezcla, fíjate qué cubren: riesgos conocidos, contingencia; lo no planificado o desconocido, gestión.

12.6 Riesgos en PP, PMC y RSKM

Las tres áreas hablan de riesgos, y es uno de los cortes más finos al reconocer el área de un enunciado. El momento separa PP de PMC, igual que en el resto de la planificación; la preparación del método es de RSKM.

ACTIVIDAD DEL ENUNCIADO	ÁREA / PRÁCTICA
Identificar y analizar riesgos para armar el plan ; revisar con los interesados y obtener su acuerdo sobre los riesgos documentados	PP SP 2.2 Identificar los riesgos del proyecto
Revisar periódicamente los riesgos contra los del plan , actualizar su documentación y comunicar su estado	PMC SP 1.3 Monitorizar los riesgos del proyecto
Definir fuentes, categorías, parámetros o umbrales , la estrategia , los planes de mitigación o contingencia , o ejecutarlos al superarse un umbral	RSKM
Evaluar alternativas de mitigación con un proceso formal de decisión	DAR

Dos ítems de la práctica oficial para el AD muestran el corte. "Informar a los stakeholders que aumentaron las chances, respecto de lo planificado, de que el módulo de consultas no pueda desarrollarse para Firefox; aún es posible resolverlo" es **PMC SP 1.3**: se compara contra el plan y el riesgo todavía no ocurrió; si ya hubiera ocurrido, sería un problema, PMC SP 2.1. "Previo al inicio del desarrollo, revisar con el gerente de sistemas del cliente si están de acuerdo con el impacto documentado para los riesgos" es **PP SP 2.2**: es planificación, y el objeto son los riesgos documentados. La respuesta oficial no está publicada; ésta es la que sostiene el texto de cada práctica.

ATENCIÓN RSKM **casi nunca es la respuesta** en los ejercicios de áreas, porque se mezcla con PP y PMC; en esa práctica ni siquiera estaba entre las opciones. Ante la duda, mirá si el enunciado habla de **preparar o planificar la gestión** de riesgos en general (RSKM) o de **un riesgo concreto** del plan (PP o PMC).

12.7 Los riesgos en una pregunta de examen

Cuando hay que proponer o evaluar los riesgos de un caso, el criterio de los finales es que salgan **del enunciado** y estén justificados: si la empresa tiene veinte años de experiencia en el rubro, "falta de experiencia" no va,

salvo que incurriera en una tecnología nueva. Y conviene cubrir tipos distintos —internos, de producto y de proyecto— en lugar de listar sólo riesgos de proyecto.

En las tablas de verdadero o falso del tipo "¿es un riesgo posible en este proyecto?", tres reglas resuelven casi todo. Un riesgo con impacto **positivo** también es un riesgo. Uno de **baja probabilidad o bajo impacto** sigue siendo un riesgo posible. Y la afirmación es **falsa** sólo cuando el enunciado **lo descarta**: si hay una sola base de datos, no hay riesgo de interfaces entre bases; si el proveedor sólo prueba y la programación es interna, no hay riesgo de que el proveedor no pueda programar.

En formato BP, una software factory que **registra los riesgos recién cuando el problema aparece** y entonces decide qué hacer **sí** tiene un problema de calidad: es exactamente la postura reactiva que RSKM viene a superar. Sin identificación temprana ni umbrales no hay plan de mitigación ni de contingencia, y se improvisa en plena crisis.

13. Los ciclos de vida

Elegir un ciclo de vida es decidir **cuándo** se hace cada cosa, y esa decisión determina cuándo se detectan los defectos y cuánto cuesta corregirlos.

Ciclo de vida: conjunto de fases por las que pasa el sistema **desde que nace la idea inicial hasta que el software es retirado o reemplazado**.

La frase aparece textual en los enunciados, y conviene reconocerla. Un modelo de ciclo de vida fija el **orden de las fases**, los **criterios de transición** entre ellas y las **entradas y salidas** de cada una, y sirve de base para planificar y coordinar el proyecto. Los modelos se diferencian sobre todo en su estructura: si hay realimentación e iteración.

Quién lo decide depende del alcance. Elegir el ciclo de un proyecto es **PP SP 1.3** Definir el ciclo de vida del proyecto. Establecer las **opciones** de ciclos de vida de toda la software factory es **OPD SP 1.2**, y fijar **en base a qué características** se elige entre los aprobados es **OPD SP 1.3** (capítulo 7). Controlar que las fases de un proyecto respeten el modelo aprobado es **PPQA** (capítulo 21).

13.1 Los modelos

CICLO	CÓMO FUNCIONA	CUÁNDO CONVIENE	CONTRAS
Casca-da	Cada etapa empieza cuando termina la anterior; las pruebas van al final	Requisitos conocidos, claros y estables ; proyectos cortos o chicos ; mantenimiento correctivo corto	El cliente no ve nada hasta el final ; un requisito tardío obliga a volver atrás. Es el que menos sirve con requerimientos inestables
Modelo en V	Cascada en la que cada fase de desarrollo tiene su nivel de prueba asociado , planificado desde el comienzo	Proyectos chicos con requisitos claros, cuando se quiere detectar defectos temprano	Rígido como la cascada; sin prototipos ; sin camino claro ante problemas en las pruebas
Incremental	Secuencias lineales escalonadas; cada una entrega un incremento operativo , y el primero es el núcleo	El cliente necesita software operativo pronto ; se conoce el problema y se lo divide	Requiere experiencia para repartir los incrementos; fases rígidas dentro de cada uno
Iterativo	Varias pasadas sobre el producto; al final de cada una el cliente evalúa una versión mejorada	Requisitos poco claros o no estabilizados , que se refinan en cada iteración	Problemas de arquitectura por no tener todos los requisitos al inicio
Espiral	Vueltas de objetivos, análisis de riesgos , desarrollo y prueba, y planificación; las características evolucionan a través de prototipos	Proyectos largos, caros, complejos o de misión crítica ; el equipo no puede especificar por adelantado	Costoso, exige experiencia en riesgos, malo para proyectos chicos. Riesgo de entregar como final un prototipo que no está listo
Prototipado	Recolección de requisitos, diseño rápido de lo visible para el usuario, construcción, evaluación del cliente, refinamiento	El cliente define objetivos generales pero no los requisitos detallados ; dudas técnicas	Se invierte en algo desechable ; el cliente puede creer que el producto ya está; tentación de estirar el prototipo hasta producto

La desventaja crítica de la cascada es que concentra las pruebas al final, de modo que los defectos se detectan cerca de la implementación, que es donde más caro sale corregirlos. El **modelo en V** nace como respuesta: cada fase de desarrollo tiene su nivel de prueba, que se planifica y diseña en paralelo. Las pruebas de **aceptación** se apoyan en los requisitos de usuario, las de **sistema** en la especificación de requisitos de software, las de **integración** en el diseño, y las **unitarias** se hacen a medida que se genera el código. La revisión de los requisitos de usuario es la **validación temprana**; las pruebas de aceptación, la **validación tardía** (capítulo 16). La cascada tiene además una variante, la **Sashimi**, que solapa las fases para descubrir antes los problemas de implementación.

13.2 Incremental, iterativo y espiral

Los tres repiten, y por eso se confunden. En el **incremental**, cada incremento es un **pedazo distinto** del producto que se pone en funcionamiento: el incremento 1 cubre el requisito R1; el 2, R2. Cada incremento es una mini-cascada completa —análisis, diseño, desarrollo y pruebas—, y lo entregado no necesariamente se refina después. En el **iterativo**, cada iteración vuelve a pasar por los **mismos** requisitos para mejorarlos: la iteración 2 retoma R1 y R2, la 3 los refina otra vez; se reserva tiempo para revisar lo hecho y replanificar lo que sigue.

Entre incremental y espiral, la diferencia es la **incertidumbre**. El incremental **parte de que no la hay**: conozco el problema y lo divido. El espiral la **asume alta** y la gestiona con un análisis de riesgos explícito en cada vuelta.

ATENCIÓN Como cada incremento es una cascada completa, en un ciclo incremental **el análisis se hace en todos los incrementos**, y las **pruebas de sistema también**, no solamente en el último: cada incremento termina en un producto funcional. Las pruebas unitarias, por supuesto, también van en cada uno.

Una consecuencia que entró en el AD de 2024: como cada incremento **se entrega**, un incremental de cuatro incrementos tiene como mínimo **cuatro releases**, y una cascada, **uno**, porque todo lo anterior a la entrega final es interno (capítulo 20). En un iterativo, en cambio, según apuntes de clase una iteración no siempre genera una versión.

13.3 Cómo elegir y justificar

En los ejercicios el ciclo casi nunca se elige en abstracto: hay que **justificarlo con elementos del enunciado**. Según una pregunta del cuestionario de la cátedra, las variables que inciden en la elección son la **estabilidad de los requerimientos** y la **necesidad de poner algo en producción antes de terminar**; el presupuesto del cliente, la cantidad de recursos y el lenguaje, no. Los finales agregan el tamaño y el riesgo del proyecto.

LO QUE DICE EL ENUNCIADO	CICLO
Requisitos conocidos y claros que no van a cambiar; proyecto corto	Cascada , o incremental si conviene dividir en versiones
Mantenimiento correctivo de un par de semanas sobre requisitos ya implementados	Cascada : no vale la pena armar incrementos de tres días con su versionado y despliegue
El cliente quiere salir a producción o ver resultados lo antes posible	Incremental , o iterativo por casos de uso en un proceso unificado
Requisitos incompletos, poco claros o no estabilizados	Iterativo o prototipos ; la cascada es el menos adecuado
Riesgo de subestimar el esfuerzo	Prototipado : al construir la pantalla se entiende el requisito
Proyecto largo, caro, riesgoso , de misión crítica	Espiral

Dos de esas filas merecen explicación. Con requisitos incompletos, el problema de la cascada es doble: exige conocerlos todos al inicio, y el riesgo de no haberlos entendido **se materializa al final**, cuando el cliente ve el producto. Y contra la subestimación del esfuerzo el incremental no ayuda: si el último módulo estaba subestimado, te enterás recién al hacerlo. El ciclo elegido también condiciona el cronograma: en cascada ninguna tarea de una etapa arranca hasta que terminó **toda** la anterior, y eso define el camino crítico.

En formato BP, que cada proyecto elija el ciclo sin criterio —una cascada con requisitos poco claros— es un problema de **OPD SP 1.2**: los errores aparecen al final y no se aprovecha lo que ya funcionó en otros proyectos. Y un proyecto sin ciclo de vida definido (**PP SP 1.3**) no tiene secuencia de tareas: se codifica sin análisis ni diseño, o se omiten las pruebas.

Dos preguntas más que se tomaron. En una cascada, el **manual de usuario** se confecciona en la fase de **prueba** —la guía de V&V lo pone como salida de las pruebas de sistema—, a partir de las **interfaces** y los **casos de uso**, no del código ni de los informes de pruebas unitarias. Y el riesgo de **relegar las pruebas al final** es detectar los defectos cuando corregirlos es más caro, con más presupuesto y atraso; la alternativa es adelantar V&V con el modelo en V, las pruebas en cada incremento, la prueba de cada prototipo y las revisiones desde los requisitos (capítulo 19).

13.4 Ciclo de vida, metodología y norma

Un ciclo de vida es un modelo **genérico**. Una **metodología** es un conjunto integrado de técnicas y métodos que combina esos modelos y define artefactos, roles y actividades. Las **tradicionales** ponen el énfasis en la planificación y la documentación, con contrato prefijado y procesos muy controlados; las **ágiles**, en la adaptabilidad: grupos chicos, el cliente como parte del equipo, contrato flexible, pocos artefactos y preparación para el cambio.

La norma **ISO/IEC 12207** ofrece un marco común de procesos del ciclo de vida del software: cinco **principales** (adquisición, suministro, desarrollo, operación y mantenimiento), los de **apoyo** (documentación, gestión de configuración, verificación, validación, revisiones conjuntas, auditoría y solución de problemas) y cuatro **organizativos** (gestión, infraestructura, mejora y formación). La guía de la cátedra dice que los de apoyo son ocho pero enumera siete; el que falta es el aseguramiento de la calidad (conocimiento general).

14. Los requerimientos y su desarrollo

Según el NIST, que cita la guía de requisitos de la cátedra, los requisitos **incompletos, imprecisos o contradictorios** causan cerca del **70 % de los defectos** de una aplicación. El problema no suele ser que después no se puedan corregir, sino que, por falta de tiempo o de presupuesto, el equipo **se precipita o supone**, y el costo del producto se multiplica. Un buen relevamiento cuesta menos que reparar un producto deficiente o cancelar un proyecto.

De ahí salen dos ideas que sirven para justificar respuestas. Los requisitos tienen que ser **entendidos por todas las partes** —cliente y desarrollador— **antes de construir**. Y los requisitos **no se congelan**: el producto evoluciona y sus requisitos también, así que la evolución hay que aceptarla y gestionarla, no negarla.

Requisito: algo que el producto debe hacer o una característica que debe tener; una condición o capacidad que el sistema tiene que cumplir y que necesitan los involucrados en el negocio. Se escribe en forma **tecnológicamente neutra**: dice **qué**, no **cómo**.

La **ingeniería de requisitos** tiene dos mitades. El **desarrollo** produce los requisitos: entiende los de negocio, obtiene los de usuario y los traduce a requisitos de sistema. La **gestión** administra los que ya existen: sus cambios y su consistencia con el resto del proyecto. En CMMI son dos áreas de la categoría de Ingeniería: **RD**, de **nivel 3**, y **REQM**, de **nivel 2**. Este capítulo trata la primera mitad; el siguiente, la segunda. Una aclaración de vocabulario: la guía de INTECO dice **requisito** y la traducción del CMMI dice **requerimiento**; son lo mismo.

14.1 Tipos de requisitos

Por nivel de abstracción, la guía distingue requisitos **de negocio** —objetivos, visión, alcance y valor esperado: dan la dirección del proyecto—, **de usuario** —las tareas que el sistema ejecuta cuando el usuario opera con él— y **de sistema o software** —las funcionalidades y características que satisfacen a los anteriores, base de la arquitectura, el diseño y los planes de prueba—. A eso se suman las **restricciones**, que limitan las opciones del diseñador o del programador (conviene evitar que el cliente imponga restricciones innecesarias), y los requisitos **técnicos**, que agrega el diseñador por la tecnología elegida y que conviene separar de los de negocio.

La distinción que más se pregunta es la de funcionales y no funcionales:

	FUNCIONALES	NO FUNCIONALES
Qué son	Qué debe hacer el producto: acciones, comportamiento observable	Cualidades que debe tener: rapidez, fiabilidad, seguridad, usabilidad
La imagen de la guía	Hacen que el producto realice el trabajo	Le dan carácter al trabajo
Cuándo se definen	Primero	Normalmente después de la funcionalidad
Peso	La mayor parte de la especificación	Tan importantes como los funcionales ; a veces críticos

ATENCIÓN Un resumen de alumnos dice que los no funcionales "no son requeridos pero son deseables". La guía dice lo contrario: son **tan importantes como los funcionales** y a veces deciden la aceptación, como pasa con la usabilidad. "Los no funcionales son opcionales" es **falso**.

La guía agrupa los no funcionales en **de producto** (usabilidad, eficiencia, fiabilidad, portabilidad, seguridad, escalabilidad), **organizacionales** (entrega, implementación, estándares, recursos) y **externos** (interoperabilidad, legislación, privacidad). En algunos resúmenes de alumnos circula también **FURPS+**, la clasificación de RUP: la F es lo funcional, y la usabilidad, la fiabilidad, el rendimiento, el soporte y las restricciones del "+" son lo no funcional (conocimiento general; no está en la guía).

Lo que el cliente dice no viene ordenado: el analista lo clasifica en requisitos de negocio, casos de uso, **reglas de negocio**, requisitos funcionales, atributos de calidad, interfaces externas, restricciones, definiciones de datos —que van al diccionario de datos— e **ideas de solución**. Si el cliente describe una forma específica de interactuar con el sistema, eso es **una solución sugerida, no un requisito**. Y lo que no encaja en ninguna categoría puede ser un requisito **del proyecto que no es de software**, como capacitar a los usuarios.

14.2 El desarrollo de requisitos según la guía

La guía organiza el desarrollo en cuatro actividades: la **obtención**, que busca los requisitos; la **definición**, que los escribe; la **verificación**, que controla cada uno en una puerta de calidad; y la **revisión de la especificación**, que controla el conjunto y lo prioriza.

Obtención. Es identificar las necesidades del negocio **resolviendo las disparidades** entre los involucrados. Bien hecha, produce requisitos completos, consistentes y dentro del alcance; **identificados de forma única** y priorizados; viables; **claros y no ambiguos**; y **testeables**, es decir, comprobables, para poder verificarlos y validarlos después. El **analista de requisitos** trabaja como un traductor: observa el trabajo, lo interpreta —el experto es el usuario—, inventa mejores formas de hacerlo y lo registra. Además tiene que sacar a la luz los requisitos que el usuario **no sabe que tiene**, porque los tiene tan internalizados que los olvida o porque no conoce la tecnología: capturarlos ahora es mucho más barato que cuando aparecen con el producto en uso.

Las buenas prácticas de obtención son detectar si el **alcance está mal definido** —muy grande, se relevan requisitos de más; muy chico, quedan afuera necesidades importantes—, centrarse en el **qué**, hacer tangible lo relevado con modelos, escenarios y prototipos, y cuidar la cantidad de participantes: **demasiados** vuelven lento el proceso y **demasiado pocos** hacen que se pasen requisitos por alto o que se especifiquen los de una minoría. Las técnicas de recogida son las **entrevistas**, con preguntas **libres de contexto** para no condicionar; las **reuniones**; la **observación** del proceso de negocio; los **cuestionarios**, útiles cuando la gente es mucha o está lejos o

importa el **anonimato**; el **brainstorming**; los **casos de uso**, que no sirven para sistemas sin interacción con el usuario; y los **prototipos y escenarios**, para cuando el usuario no da detalle o el producto es muy innovador.

Definición. Si no se toma tiempo para definir, se pierde después. Las reglas de redacción son concretas: requisitos tecnológicamente neutros y **no ambiguos**, sin **pronombres** (se reemplazan por el sujeto), con cuidado en los **adjetivos y adverbios**, y **sin "debería"**, que da a entender que el requisito es opcional. Se escriben en **lenguaje de negocio**, porque los leen personas no técnicas, y se apoyan en un **glosario** que evoluciona con el proyecto. Para confirmar que todos entienden lo mismo, la guía propone elegir requisitos al azar y pedir a los agentes del negocio que los interpreten: si hay muchas discordancias, hay que reescribir la especificación.

Verificación: la puerta de calidad. Cada requisito, antes de entrar a la especificación, pasa por una **puerta de calidad** que custodian una o dos personas —típicamente el responsable del análisis y el **técnico de pruebas**—, que controlan su relevancia, su coherencia, su trazabilidad y su **capacidad de prueba**. En esta etapa el **usuario final agrega los criterios de aceptación** de cada requisito, lo que ayuda a eliminar ambigüedades. La puerta sirve además para prevenir las **fugas de requisitos**, los que aparecen en la especificación sin que nadie sepa de dónde vienen ni qué valor agregan.

Revisión de la especificación. La puerta controla requisitos **uno por uno**; la revisión controla **el conjunto**: que no falte ningún **tipo** de requisito —un producto financiero sin requisitos de seguridad tiene algo faltante—, que haya excepciones y escenarios alternativos suficientes, y que **no haya conflictos**. Dos requisitos están en conflicto si la solución de uno impide implementar el otro, y se detectan con una **matriz de conflictos** que cruza requisitos contra requisitos.

La **priorización** hace falta cuando las expectativas son altas y los plazos y recursos, limitados. El cliente prioriza por **valor**; el desarrollador informa **costo, dificultad y riesgo**, y con eso el cliente revisa si cada requisito es tan esencial como creía. Si el cliente no logra priorizar, decide el jefe de proyecto. La prioridad es un **atributo de cada requisito**: alta (esencial para la próxima versión), media (puede esperar a una posterior) o baja (una mejora, si sobran recursos).

Evolución. Es un error creer que hay que especificar todo antes de diseñar, salvo que la especificación sea la base de un contrato. Se puede arrancar con los casos de uso de **alta prioridad**, pasarlos a desarrollo y seguir relevando el resto; así los errores en los requisitos se detectan antes. El proceso es una **espiral**: requisitos en bruto, análisis y negociación, documentación, validación, y un punto de decisión para aceptar la especificación o volver a iterar.

La matriz de responsabilidades de la guía deja cuatro reglas: el **jefe de proyecto es responsable** de todo el proceso; el **cliente aprueba** los requisitos y su validación; el **CCB aprueba los cambios** (capítulo 15); y **SQA revisa**, pero no aprueba los requisitos del cliente.

ATENCIÓN La guía llama "verificación" a la puerta de calidad, donde el usuario agrega criterios de aceptación, y en otro pasaje formula la validación con la pregunta que en realidad es de la verificación. Para el examen conviene el criterio CMMI del capítulo 16: verificar es contrastar contra lo especificado; validar, contra lo que el usuario necesita en su entorno.

14.3 Prototipos

Un **prototipo** es un borrador o una **simulación de los requisitos**. Rinde más cuando los usuarios no están seguros de lo que necesitan o no saben expresarlo, cuando el sistema altera una operación básica del negocio, o cuando hay que comparar soluciones. Puede ser de **alta fidelidad**, hecho con herramientas, o de **baja fidelidad**, en papel, que es el más usado por rápido.

TIPO	PARA QUÉ	CUÁNDO
De concepto	Visión general de alto nivel: aspecto, alcance, topología	Al definir el concepto
De viabilidad	Probar que los componentes críticos de la arquitectura se integran	Al definir el concepto
Horizontal (de interfaz)	Clarificar alcance y requisitos: todas las pantallas, casi sin lógica	Temprano en el análisis
Vertical	Pocas funciones en profundidad: acceso a datos, seguridad, excepciones	Tarde en el análisis
Storyboard	La secuencia de pantallas en el orden en que se verán	Al definir las funciones

Tiene sus límites: la estructura se corrompe con los cambios, exige participación activa del usuario, deja cuestiones de diseño sin documentar, puede llevar a un **acuerdo prematuro sobre el diseño físico** y, sobre todo, **no reemplaza la especificación escrita**: la complementa.

14.4 RD – Desarrollo de requerimientos

Propósito de RD: producir y analizar los requerimientos **de cliente, de producto y de componente del producto**.

Los tres tipos existen porque el cliente habla en sus términos, a veces no técnicos: esos son los **requerimientos de cliente**. Para diseñar hay que refinarlos y traducirlos a términos técnicos, y así nacen los **requerimientos de producto y de componentes**. A ellos se suman los **requerimientos derivados**, que surgen de restricciones, de problemas implícitos que el cliente no declaró y de las decisiones de arquitectura y diseño.

META	PRÁCTICA	CÓMO SE RECONOCE
SG 1 Desarrollar los requerimientos de cliente	SP 1.1 Obtener las necesidades	Relevar con entrevistas, cuestionarios, prototipos, observación o expertos; descubrir proactivamente lo que el cliente no dijo
	SP 1.2 Desarrollar los requerimientos de cliente	Consolidar lo relevado, completar lo que falta, resolver conflictos entre interesados, documentar
SG 2 Desarrollar los requerimientos de producto	SP 2.1 Establecer los requerimientos de producto y de componentes del producto	Pasar a términos técnicos ; derivar requerimientos de las decisiones de diseño
	SP 2.2 Asignar los requerimientos de componentes del producto	Repartir los requerimientos entre funciones y componentes
	SP 2.3 Identificar los requerimientos de interfaz	Interfaces externas e internas
SG 3 Analizar y validar los requerimientos	SP 3.1 Establecer los conceptos operativos y los escenarios	Escenarios y casos de uso de cómo se va a usar el producto
	SP 3.2 Establecer una definición de la funcionalidad requerida	Qué tiene que hacer el producto: el análisis funcional
	SP 3.3 Analizar los requerimientos	Que sean necesarios, suficientes, completos, factibles y verificables : el requisito vago o no medible
	SP 3.4 Analizar los requerimientos para alcanzar el equilibrio	Balancear lo pedido contra costo, plazo, rendimiento y riesgo

META	PRÁCTICA	CÓMO SE RECONOCE
	SP 3.5 Validar los requerimientos	Confirmar temprano y con los usuarios que son los correctos: prototipos, simulaciones, demostraciones

La tercera meta da soporte a las otras dos: el análisis y la validación acompañan todo el desarrollo de los requerimientos.

ATENCIÓN En RD, toda opción en la que **el equipo decide solo** es incorrecta: elegir los dispositivos, completar con métricas estándar de la industria, "lo resolvemos en las pruebas". RD exige **confirmar con el cliente**. En el AD de 2024, ante un cliente que pedía acceso desde dispositivos móviles sin decir cuáles, la correcta fue definir una lista de dispositivos y sistemas operativos y **confirmarla con el cliente**, para que el requisito fuera específico y verificable. Ante requisitos de usabilidad vagos, la correcta fue reunirse con el cliente para definir **métricas** alineadas con sus expectativas y actualizar la documentación; la trampa era completar con métricas estándar de la industria.

RD tiene tres fronteras que se preguntan. **Con REQM:** RD documenta las relaciones entre requerimientos al refinarlos, pero **mantener la trazabilidad** es REQM SP 1.4; y si se modifica un requerimiento ya aprobado, el contenido nuevo lo produce RD, pero la **administración del cambio** es de REQM. **Con VAL:** RD SP 3.5 valida **los requerimientos**, temprano y sobre representaciones como prototipos o simulaciones, mientras que VAL valida **el producto** o sus componentes; probar el sistema con el cliente —una prueba de aceptación, un piloto— es VAL. **Con el tiempo:** RD SP 1.2 incluye definir las restricciones del cliente para la verificación y la validación, así que V&V se prepara **desde el relevamiento**, y "la validación recién empieza al final del proyecto" es falso.

14.5 Las buenas prácticas de la guía

La guía cierra con una lista de mejores prácticas, separadas en desarrollo y gestión; la columna de la derecha se desarrolla en el capítulo siguiente.

DESARROLLO DE REQUISITOS	GESTIÓN DE REQUISITOS
Documentar el alcance y la visión del proyecto	Priorizar los requisitos
Mantener un glosario , con los acrónimos	Establecer líneas base de requisitos
Usar técnicas de obtención conocidas y probadas	Comunicación abierta , con la gente correcta
Involucrar a clientes y usuarios reales en la revisión	Capturar todos los cambios , con el histórico de razones y la volatilidad
Desarrollar los requisitos incrementalmente	Usar herramientas de gestión de requisitos, que ayudan pero no gestionan solas
Entender por quién y para quién es cada requisito	Mantener la trazabilidad bidireccional
Capturarlos con casos de uso	Un plan de mejora del proceso de requisitos
Validarlos (¿son los correctos?) y verificarlos (¿están completos y correctos?)	Formar a los analistas, para que no inventen requisitos

15. La gestión de requerimientos y de los cambios

Una vez definidos los requisitos —e incluso con el sistema ya implantado— aparecen necesidades nuevas, correcciones y mejoras. Cambian las leyes, la estrategia del negocio, el mercado o la percepción de los usuarios, o

se descubre que el problema estaba mal analizado. El cambio no es una falla del relevamiento: es inherente al software. La falla es no gestionarlo. El cliente le pide un ajuste por mail al programador, el programador lo hace esa tarde, y tres meses después nadie sabe por qué el sistema calcula distinto de lo que dice la especificación, ni qué otras partes tocó ese cambio.

La **gestión de requisitos** es el conjunto de actividades para **identificar, controlar y seguir** los requisitos y sus cambios en cualquier momento, y asegurar la **consistencia** entre los requisitos y el sistema construido. La guía le asigna tres objetivos: gestionar la **recogida**, definiendo los **canales** por los que entran los requisitos; obtener la **aprobación** de clientes y desarrolladores para establecer la **línea base**; y gestionar los **cambios**, que es la actividad más importante. Tiene que estar incluida en el plan del proyecto y dura toda la vida del producto, aunque la guía dice también que puede considerarse terminada cuando **el cliente acepta el producto**.

Línea base: conjunto de productos de trabajo **revisado y acordado formalmente**, que sirve de base para el desarrollo posterior y que **sólo puede cambiarse mediante un procedimiento formal de control de cambios**.

Un requisito entra a la línea base por **revisión formal**, y una vez adentro todo cambio pasa por el control de cambios. Cambiar la línea base de requisitos es, por definición, un **cambio de alcance**.

15.1 El orden ante un cambio

El proceso de la guía para cada petición de cambio tiene un orden que se pregunta una y otra vez. **Primero se registra y se evalúa el impacto:** con la matriz de trazabilidad se ve qué otros requisitos, módulos y casos de prueba se afectan —directamente o porque dependen de los afectados—, cuántos son y cuánto cuesta el cambio en tiempo y recursos. **Después se valora:** si el impacto es asumible, se acepta; si no, se **negocia con el cliente**; si se rechaza, queda registrado. **Recién entonces** se analiza la modificación, se **modifican todos los productos afectados** —la especificación, la matriz de trazabilidad y, si el cambio lo justifica, el plan del proyecto—, se establece una **nueva línea base** y se obtiene la **aprobación del cliente**.

ATENCIÓN Toda opción que **implemente antes de evaluar el impacto** es incorrecta. En el AD de 2024, ante un cliente que en plena implementación pedía restricciones horarias que afectaban a varios subsistemas, la correcta fue analizar cómo afectan a los otros subsistemas y actualizar la matriz de trazabilidad **antes de proceder**. La trampa más fina era "registrar el cambio, actualizar la matriz y **comenzar la implementación**"; parece REQM, pero se saltea la evaluación. Tampoco es gestión de cambios pedirle al cliente que **resigne otros requisitos** para no tocar nada: la negociación viene después de evaluar el impacto, no en su lugar.

En el mismo parcial, en la fase de pruebas de un comercio electrónico, el cliente quería registrar la navegación de los usuarios, lo que implicaba cumplir normas de protección de datos. El primer paso correcto fue **consultar con legales y privacidad** y reflejar el impacto en la matriz: evaluar el impacto desde el punto de vista de las partes interesadas, que en ese caso eran las de legales. Las leyes son fuente de requisitos y de cambios como cualquier otra.

15.2 La trazabilidad

Trazabilidad bidireccional: asociación discernible **en ambos sentidos** entre entidades. En requisitos, **hacia atrás**, del requisito a su origen, y **hacia adelante**, del requisito al diseño, el código y las pruebas.

Un requisito es **trazable** si se pueden identificar todas las partes del producto relacionadas con él. De cada uno hay que conocer su **origen** (quién lo propuso), su **necesidad** (por qué existe) y sus **dependencias** con otros re-

quisitos y elementos.

	HACIA ATRÁS	HACIA ADELANTE
Otro nombre	Pre-trazabilidad	Post-trazabilidad
Vincula	El requisito de software con los de usuario: lo anterior a la especificación	El requisito con el diseño, el código y los casos de prueba: lo posterior
Responde	¿De dónde viene? ¿Tiene una fuente válida?	¿Qué se rompe si lo cambio? ¿Está cubierto?

Sirve, sobre todo, para **evaluar el impacto de un cambio**. La relación entre requisitos y funciones es de **muchos a muchos**, así que antes de tocar código por un cambio hay que comprobar que no queden comprometidos otros requisitos. Y tiene un costo que conviene mantener bajo: si actualizar la trazabilidad es muy trabajoso, los desarrolladores cambian el código **sin actualizar el documento de requisitos**, que se vuelve inútil para probar y validar.

La guía propone dos matrices. La **matriz hacia atrás y hacia adelante** tiene una columna por nivel —requisito de negocio, de usuario, de sistema, caso de uso, diseño, código, casos de prueba unitarios, de integración y de sistema— y una última de **petición de cambio**, que es el puente entre la gestión de requisitos y la de configuración. La **matriz de dependencias** cruza requisitos contra requisitos y marca cuáles dependen de cuáles, para mantener la consistencia y analizar el impacto.

ATENCIÓN ¿Qué área **establece y mantiene** la trazabilidad entre los requerimientos y los otros productos de trabajo? **REQM**, SP 1.4. RD documenta las relaciones al refinar, pero remite a REQM para mantenerlas.

15.3 REQM – Gestión de requerimientos

Propósito de REQM: gestionar los requerimientos de los productos y componentes del proyecto, e **identificar inconsistencias** entre esos requerimientos y los **planes y productos de trabajo** del proyecto.

Es un área de **nivel 2** con una sola meta, **SG 1 Gestionar los requerimientos**: el proyecto mantiene un conjunto **actual y aprobado** de requerimientos durante toda su vida, gestionando los cambios, manteniendo sus relaciones con planes y productos, detectando inconsistencias y tomando acciones correctivas. Gestiona **todos** los requerimientos que el proyecto recibe o genera, vengan de una petición del cliente o de RD: sin importar la fuente, se gestionan.

PRÁCTICA	QUÉ HACE	CÓMO SE RECONOCE
SP 1.1 Obtener una comprensión de los requerimientos	Entendimiento compartido con el proveedor del requerimiento; canales oficiales y criterios de aceptación	"Revisar con el cliente qué significa", "toda solicitud que no cumpla será devuelta"
SP 1.2 Obtener el compromiso sobre los requerimientos	Acuerdo de quienes los implementan ; evaluar y negociar el impacto sobre los compromisos existentes	Reunión con el Sponsor para correr una fecha e incorporar cambios
SP 1.3 Gestionar los cambios de los requerimientos	Documentar cada cambio con su razón —de ahí sale la volatilidad— y evaluar su impacto	"Se está chequeando cómo afecta el cambio"
SP 1.4 Mantener la trazabilidad bidireccional de los requerimientos	Desde la fuente hasta el nivel más bajo y de vuelta	"Actualizar la matriz de trazabilidad "
SP 1.5 Identificar las inconsistencias entre el trabajo del proyecto y los requerimientos	Revisar planes y productos contra los requerimientos vigentes e iniciar acciones correctivas	"El caso de uso o el plan no coincide con el requerimiento"

Los criterios de aceptación de SP 1.1 repiten casi uno a uno los atributos de la obtención: requerimientos claros y correctos, completos, consistentes entre sí, **identificados de forma única**, apropiados para implementar, **verificables** y **trazables**.

La diferencia entre SP 1.1 y SP 1.2 es **con quién**. La comprensión se logra con el **proveedor** del requerimiento, el cliente o el usuario que lo pide; el compromiso, con los **participantes del proyecto** que lo van a implementar y con quien aprueba su impacto en el plan. Un ejemplo del cuestionario de la cátedra: acordar fehacientemente con el Sponsor posponer tres semanas la implementación para incorporar los cambios del gerente de ventas es **REQM SP 1.2**, porque se negocia el impacto de un cambio sobre un compromiso existente. No es PMC SP 1.2: no se monitorea un compromiso, se negocia uno nuevo.

En el CMMI de la cátedra, REQM es un área de la categoría **Ingeniería**, como RD; en versiones posteriores del modelo pasó a gestión de proyectos (conocimiento general).

15.4 Las solicitudes de cambio

La gestión de requisitos dice **qué** decidir ante un cambio; la **gestión de solicitudes de cambio** de RUP dice **cómo** se tramita. Su razón de ser es que todo cambio pase por **procedimientos estandarizados**, deje un **registro de decisiones** y se haga **de modo controlado y con efecto predecible**, con el impacto considerado antes de tocar nada.

Solicitud de cambio (CR): producto de trabajo **enviado formalmente** para rastrear todas las solicitudes de los interesados —funciones nuevas, mejoras, **defectos**, requisitos cambiados— junto con su estado e historial durante todo el ciclo de vida.

Hay dos tipos: las **solicitudes de mejora**, que piden características futuras, y los **defectos**, anomalías de un producto entregado. Cualquier interesado puede enviar una, y el responsable del artefacto es el gestor de control de cambios.

CCB (panel de control de cambios): grupo que supervisa el proceso de cambio, integrado por representantes de **todas las partes interesadas**. En un proyecto chico puede ser **una sola persona**, como el jefe de proyecto o el arquitecto.

La **reunión de revisión del CCB** —típicamente semanal, diaria cuando sube el volumen o se acerca el fin del release— decide primero si cada CR es **válida** y, si lo es, si está **dentro o fuera del ámbito del release actual**, según prioridad, planificación, recursos, esfuerzo, riesgo y gravedad.

Los estados de una CR cuentan ese recorrido. El camino feliz es **Enviado** → **Abierta** → **Asignada** → **Resuelta** → **Verificada** → **Cerrada**: la CR entra a la cola del CCB sin propietario; la reunión del CCB la abre; el gestor de proyectos la asigna y **actualiza la planificación**; el miembro asignado hace el cambio; un verificador lo prueba en la compilación de prueba; y se cierra cuando se verificó en el release. Los desvíos tienen nombre propio:

ESTADO	CUÁNDO
Pospuesta	Válida pero fuera del ámbito del release actual; vuelve a Enviado en un release futuro
Duplicada	Repite otra ya enviada; por eso el emisor debería buscar duplicados antes de enviar
Rechazada	No es válida o falta información; lo confirma una autoridad del CCB
Más información	No hay datos para confirmar el rechazo o el duplicado; vuelve al emisor
La prueba ha fallado	La resolución falló en la compilación de prueba o del release; vuelve a quien la resolvió

ATENCIÓN Sólo la reunión del CCB puede pasar una CR a Abierta, y sólo el administrador de revisión del CCB puede cerrarla. Se cierra en tres casos: la resolución se verificó en el release, se confirmó el rechazo o se confirmó el duplicado.

El formulario de una CR registra, entre otras cosas, el tipo (**problema o mejora**), la prioridad, el **costo estimado** del cambio y la decisión del equipo de revisión —**aprobado, no aprobado o diferido**— con los **elementos de configuración afectados**. Lo eficiente es guardarlas en una **herramienta o base de datos**; en un proyecto chico alcanza una planilla, que deja de ser manejable cuando crecen el equipo y los defectos.

En CMMI, este proceso implementa **CM SG 2 Seguir y controlar los cambios** (capítulo 20): **SP 2.1 Seguir las peticiones de cambio** y **SP 2.2 Controlar los elementos de configuración**. Las solicitudes de cambio son de **CM**, no de **REQM**, y no tratan sólo requisitos: también **defectos y fallos**.

15.5 REQM, RD o CM: cómo decidir

Las tres áreas giran alrededor de los requisitos, y separarlas es de lo más preguntado de la unidad. Dos pasos resuelven casi todo.

Primer paso: ¿el requisito ya está acordado? Si todavía se está relevando, aclarando, completando o validando, es **RD**, y la práctica sale de la tabla de 14.4. Si ya está en la línea base, seguí al segundo paso.

Segundo paso: ¿qué se hace con el requisito acordado?

SITUACIÓN DEL ENUNCIADO	ÁREA / PRÁCTICA
Entender con quien lo pidió qué significa; fijar canales oficiales o criterios de aceptación	REQM SP 1.1
Acordar con quienes implementan, o con el Sponsor, el compromiso o el corrimiento de una fecha	REQM SP 1.2
Llega un cambio y todavía se evalúa cómo afecta	REQM SP 1.3
Actualizar o consultar la matriz de trazabilidad	REQM SP 1.4
El plan, el diseño o los casos de uso no coinciden con los requisitos vigentes	REQM SP 1.5

El cambio ya se decidió aceptar y se tramita: seguir la petición de cambio, ver qué modificar en los casos de uso, cerrar la CR	CM SP 2.1
Controlar la autorización, el check-in y check-out o la versión del elemento modificado	CM SP 2.2
Revisar y acordar formalmente un conjunto de artefactos como base del desarrollo	CM SP 1.3 (línea base)
Alguien externo al proyecto detecta cambios hechos sin seguir el procedimiento de REQM y escala	PPQA SP 2.1

El ejemplo canónico viene del cuestionario de la cátedra. "El cliente pidió una variante de descuento no acordada; en este momento se está chequeando con el contador cómo afecta al esquema impositivo" es **REQM SP 1.3**. "Ya se decidió aceptar la variante; ahora se chequean las modificaciones a realizar en los casos de uso" es **CM SP 2.1**.

ATENCIÓN El análisis de impacto aparece **en las dos áreas**, REQM SP 1.3 y CM SP 2.1. Lo que decide es **el momento**: si todavía se evalúa si conviene, REQM; si ya se aceptó y está en trámite, CM. El criterio sale de las notas del cuestionario de la cátedra, no del texto del CMMI. Si el enunciado no da pista temporal, mirá el objeto: el requisito y su consistencia con los planes es REQM; la petición de cambio o el elemento de configuración, CM.

ATENCIÓN **REQM o CM según el contenido del cambio, no según quién lo pide.** Un cambio interno que **no altera ningún requisito** —el equipo refactoriza, optimiza una función o corrige un defecto de código— no es un cambio de requerimiento: se controla en **CM**. Si el cambio **sí altera** un requisito, es REQM SP 1.3 aunque lo proponga el equipo. La guía de resolución de exámenes que circula entre alumnos lo simplifica como "REQM sólo trata los cambios que pide el cliente": sirve como regla rápida, porque casi siempre coincide, pero el criterio de fondo es el contenido.

Dos trampas más. Un enunciado que **menciona** la línea base de requerimientos no es por eso REQM: si lo que se hace es avisar que aumentó la probabilidad de un problema, es un riesgo (PMC SP 1.3, capítulo 12). Y un informe de aseguramiento de calidad que **escala** cambios hechos sin el procedimiento sigue siendo **PPQA**: el área la define la actividad, no el tema.

15.6 Las señales en una pregunta BP

En las preguntas BP sobre requisitos, el problema de calidad casi siempre es uno de estos eslabones faltantes (el método general está en el capítulo 25):

SEÑAL EN LA PRÁCTICA DESCRIPTA	QUÉ FALTA
El cliente pide cambios por mail al programador , que los acepta solo	Evaluar el impacto y actualizar los documentos (REQM SP 1.3 y 1.4; CM SP 2.1)
Se cambian requisitos sin documentar la razón , sin evaluar el impacto o sin actualizar la trazabilidad	REQM SP 1.3 y 1.4
No hay canales oficiales de entrada, o los requisitos se aceptan sin criterio	REQM SP 1.1
No hay línea base , o se modifica sin control de cambios	CM SP 1.3 y REQM
No hay CCB, cualquiera aprueba, o las CR quedan abiertas sin seguimiento	CM SP 2.1

Los requisitos entran a la especificación sin pasar por una **puerta de calidad**

Control de las fugas de requisitos

Requisitos verbales, con "debería" o **no verificables**

RD SP 3.3 y REQM SP 1.1

El equipo define por su cuenta lo que debía confirmar el cliente, o el relevamiento lo hace **sólo el área comercial**

RD SP 1.2, 3.3 y 3.5

El **primer feedback del cliente** llega con el sistema instalado

RD SP 3.5 y VAL

Si falta una práctica de REQM, que es de nivel 2, la organización queda en **nivel 1**; si falta una de RD, de nivel 3, queda **como máximo en nivel 2**.

16. Verificación y validación

Un sistema puede fallar de dos maneras muy distintas. Puede **no hacer lo que dice su especificación**: el recargo está mal calculado, una pantalla acepta un dato que debería rechazar. O puede hacer **exactamente lo que dice su especificación y aun así no servirle al cliente**, porque la especificación quedó mal relevada. Son dos problemas diferentes, se detectan con preguntas diferentes, y CMMI les dedica dos áreas de proceso. Las preguntas son de **Boehm**:

Verificación (VER): asegurar que los productos de trabajo seleccionados **cumplen sus requerimientos especificados**. ¿Estoy construyendo **correctamente** el producto?

Validación (VAL): demostrar que un producto o componente **se ajusta a su uso previsto cuando se sitúa en su entorno previsto**. ¿Estoy construyendo el producto **correcto**?

La guía de V&V de INTECO lo dice con otras palabras: verificar es comprobar que el software está de acuerdo con su especificación, en requisitos funcionales y no funcionales; validar **va más allá**, porque las especificaciones no siempre reflejan las necesidades reales de usuarios y propietarios.

16.1 La distinción

Es la pareja de conceptos más importante de la unidad y la más fácil de confundir, porque las dos son procesos de **evaluación de productos**, se ejecutan frecuentemente **de forma concurrente** y pueden compartir parte del entorno. Las dos son áreas de **nivel 3**, de la categoría **Ingeniería**, necesarias pero no suficientes para alcanzar ese nivel.

	VERIFICACIÓN (VER)	VALIDACIÓN (VAL)
Contra qué contrasta	Los requerimientos especificados : especificación, diseño, minutas, glosario del proyecto	El uso previsto : las necesidades reales del usuario
Quién interviene	El equipo del proyecto: pares, testers, analistas. Es una cuestión interna	El cliente o usuario
Dónde	Un entorno de verificación	El entorno previsto , o uno que lo represente
Técnicas típicas	Revisiones entre pares , inspecciones, pruebas contra la especificación	Revisión de requisitos con el cliente , demostración de prototipos, pruebas de aceptación , piloto, beta
Niveles de prueba (INTECO)	Unitarias, integración y sistema	Unitarias y aceptación

ATENCIÓN El criterio que resuelve cualquier caso es **contra qué se contrasta y quién participa**, no qué tan terminado está el producto ni en qué fase ocurre. Unas pruebas de sistema ejecutadas por el equipo de QA sobre el producto terminado son **verificación**, porque se contrastan contra la especificación. Unas pruebas de aceptación con clientes reales son **validación**. Y revisar los requisitos **con el cliente** también es validación, aunque no se ejecute nada.

16.2 El criterio rápido para un enunciado

En el parcial casi nunca piden la definición: describen una actividad y preguntan el área. La regla tiene tres salidas. Si aparece el **cliente o usuario**, o se prueba **el uso real**, es **VAL**. Si se controla **un artefacto contra otro artefacto del proyecto** —casos de uso contra minutas, etiquetas contra el glosario, casos de prueba contra las particiones— sin el cliente, es **VER**. Y si se controla contra un **estándar o procedimiento de la organización**, es **PPQA** (capítulo 21): PPQA asegura que los procesos planificados se implementan; VER, que se satisfacen los requerimientos especificados. Un ejemplo de clase, sobre un mismo caso de uso: "en el alternativo 2.a no se usó la etiqueta de tiempo" es PPQA, porque no se respetó el estándar; "se usó *durante* cuando correspondía *reemplaza*" es VER, porque el contenido no satisface lo especificado.

El AD 2025 lo preguntó con **diagramas de conjuntos**. Un sistema de control de acceso lo prueba primero el equipo técnico y después el cliente, en un edificio piloto; con tres conjuntos —**S**, la especificación; **P**, el producto implementado; **Test**, las pruebas ejecutadas— se dibujaban dos planteos, y había que asignarlos:

PLANTEO	QUÉ REPRESENTA	ÁREA
Test casi entero dentro de S , cortando un poco a P	Los casos se derivan de la especificación y se mira si el producto la cumple; lo que de Test queda fuera de P es lo especificado que el producto no hace	VER
Test casi entero dentro de P , cortando un poco a S	Se prueba el producto real en su entorno de uso , incluso comportamiento que la especificación no contempla	VAL

La respuesta está confirmada por la corrección; la justificación es una lectura de las definiciones, porque la bibliografía no trae ese diagrama: la verificación mira desde la especificación, la validación mira desde el uso.

Queda una frontera con **RD SP 3.5 Validar los requerimientos** (capítulo 14), que valida **los requerimientos**, temprano y sobre prototipos o simulaciones. **VAL** valida **el producto o sus componentes**: elegir los casos de uso que entran en la prueba de aceptación, o probar con el cliente en el edificio piloto, es VAL. Si las opciones sólo ofrecen VER y VAL, revisar la especificación con el cliente también es VAL.

Para clasificar artefactos sirve la misma pregunta:

ARTEFACTO	ÁREA	POR QUÉ
Casos de prueba armados desde las particiones · diagramas · modelo de datos · plan de proyecto	VER	Se revisan contra los requisitos o las particiones, entre pares, sin el usuario
El software desarrollado · un prototipo	VAL	Se confrontan con el uso previsto; demostrar prototipos es un método de validación
La lista de requerimientos	Las dos	VER controla consistencia, completitud y verificabilidad; VAL la revisa con el cliente
El manual de usuario	VAL (y VER)	CMMI lo lista como validable; sus errores se corrigen por revisión entre pares
Una minuta de relevamiento	Depende	VAL si se controla en reunión con el cliente; VER si el equipo la revisa entre sí
La base de datos de prueba	VER	Es parte del entorno de verificación

ATENCIÓN Cuatro trampas que se repiten. **"Dinámica = validación" es falso como regla:** INTECO dice que las técnicas dinámicas están "más orientadas" a la validación, pero pone las pruebas de integración y de sistema en VER (16.9). **Una revisión entre pares es siempre VER;** una revisión **con el cliente** es VAL. **La inspección es una técnica, no un área:** la usa VER y también PPQA, y "las inspecciones sólo las puede hacer Verificación" fue un distractor del AD 2024. Y **prueba de aceptación → VAL, prueba de sistema → VER,** aunque las dos se hagan sobre el sistema completo; unas pruebas de estrés hechas por la propia software factory también son VER.

16.3 Metas y prácticas

Las dos áreas tienen la misma estructura —preparar, realizar, analizar los resultados—, con una diferencia: VER tiene una meta en el medio que VAL no tiene.

VERIFICACIÓN (VER)	VALIDACIÓN (VAL)
SG 1 Preparar la verificación — SP 1.1 Seleccionar los productos de trabajo a verificar · SP 1.2 Establecer el entorno de verificación · SP 1.3 Establecer los procedimientos y los criterios de verificación	SG 1 Preparar la validación — SP 1.1 Seleccionar los productos a validar · SP 1.2 Establecer el entorno de validación · SP 1.3 Establecer los procedimientos y los criterios de validación
SG 2 Realizar revisiones entre pares — SP 2.1 Preparar las revisiones entre pares · SP 2.2 Llevar a cabo las revisiones entre pares · SP 2.3 Analizar los datos de la revisión entre pares	(VAL no tiene esta meta)
SG 3 Verificar los productos de trabajo seleccionados — SP 3.1 Realizar la verificación · SP 3.2 Analizar los resultados de la verificación	SG 2 Validar el producto o los componentes de producto — SP 2.1 Realizar la validación · SP 2.2 Analizar los resultados de la validación

ATENCIÓN Las **revisiones entre pares existen sólo en VER.** Si el enunciado describe una revisión entre pares y la opción ofrecida dice **VAL/SP 2.1 Preparar las revisiones entre pares**, la respuesta es **NINGUNA:** hay que leer la sigla, no sólo el nombre de la práctica.

VER usa la **trazabilidad** que mantiene REQM (capítulo 15) para saber qué requerimientos debe satisfacer cada producto, y sus productos y métodos de verificación se integran al plan del proyecto. VAL, cuando encuentra que un problema está en los requerimientos o en el diseño, lo deriva a **RD** o a **TS**.

16.4 De la actividad a la práctica

Los ejercicios de "relacione" piden la práctica, y para elegirla sirven dos preguntas. La primera es **en qué momento** está la actividad: en futuro —"será chequeada", "se distribuirá", "se determinó que se controlará"— se está **preparando** (SG 1, o SP 2.1 si es una revisión entre pares); en presente —"está chequeando", "controla"— se está **realizando**; y "ya se documentaron los hallazgos, ahora se está almacenando la información" es **analizar los datos**. La segunda es **qué se prepara**: los productos y los métodos, el entorno, o los procedimientos y criterios.

LO QUE DESCRIBE EL ENUNCIADO	PRÁCTICA
Decidir qué productos se verifican y con qué método: "qué clases entran en la prueba de integración", "se controlarán mediante inspección"	VER SP 1.1
Amar el entorno: cargar la base de datos de pruebas, conseguir las herramientas	VER SP 1.2
Escribir los procedimientos, los criterios y los resultados esperados	VER SP 1.3
Definir el tipo de revisión ("será mediante un walkthrough"), el calendario, los roles, las listas de comprobación; distribuir el producto que será chequeado	VER SP 2.1
Los pares chequean el producto, plantean observaciones y documentan los defectos	VER SP 2.2
Almacenar los datos de la revisión y protegerlos , para que no se usen, por ejemplo, para evaluar a las personas: guardar las identidades de los revisores donde sólo accede Calidad	VER SP 2.3
Controlar un producto contra sus requerimientos (casos de uso contra minutas, etiquetas contra el glosario) y registrar los resultados	VER SP 3.1
Comparar resultados reales con esperados, informar, iniciar acciones correctivas	VER SP 3.2
Elegir qué casos de uso entran en la prueba de aceptación o en un workshop con el cliente	VAL SP 1.1
Conseguir el entorno de validación: el servidor del cliente, la agenda de horarios para la prueba de aceptación	VAL SP 1.2
Acordar con el cliente los criterios de aceptación; documentar el escenario operacional, las entradas y las salidas	VAL SP 1.3
Controlar con el usuario si las etiquetas de pantalla se entienden	VAL SP 2.1
Analizar qué no funciona en el entorno operativo previsto y derivar los problemas	VAL SP 2.2

ATENCIÓN "Determinar que se controlarán mediante inspección los diagramas de secuencia" es VER sin dudas. Si piden la práctica, encaja en SP 1.1 (elegir productos y métodos) y también en la primera subpráctica de SP 2.1 (determinar el tipo de revisión entre pares): fíjate cuál de las dos aparece entre las opciones.

16.5 Qué se valida y cómo

La validación se aplica a los productos de trabajo —requerimientos, diseños, prototipos— y al producto y sus componentes, y se hace **temprana e incrementalmente**, no al final. Los productos se eligen por su relación con las necesidades del usuario, y esa selección **se revisa con las partes interesadas**. El entorno debe **representar el entorno previsto**, y puede usarse completo o sólo en parte. Los métodos posibles son la discusión con usuarios, las demostraciones de prototipos, las demostraciones funcionales, los **pilotos** de materiales de formación, las pruebas realizadas por los usuarios finales y el análisis mediante simulaciones o modelado. Entre las fuentes de los criterios de validación están los **criterios de aceptación del cliente**.

Es validable más de lo que se suele pensar: requerimientos y diseños, el producto y sus componentes, las **interfaces de usuario**, los **manuales de usuario**, los **materiales de formación** y la documentación del proceso. Cuando el producto **se adquiere**, valida el adquiridor, el proveedor o ambos, según lo que fije el acuerdo con el proveedor.

La **verificación es incremental**: empieza por la verificación de los requerimientos, sigue con los productos de trabajo a medida que evolucionan, y culmina con la verificación del producto terminado.

16.6 Los requisitos: el primer producto que pasa por V&V

La especificación de requisitos es el primer producto que se verifica y se valida, y el que más caro sale si pasa con errores: genera costos importantes de retrabajo cuando el defecto aparece más adelante. **Validarla** es comprobar que define el sistema que el cliente desea, y se hace con **revisiones de requisitos, prototipos y generación de casos de prueba**. Sobre el documento se hacen cinco comprobaciones:

COMPROBACIÓN QUÉ MIRA

Validez	Que las funciones pedidas sean realmente las necesarias; el análisis puede descubrir funciones adicionales o distintas
Consistencia	Que los requisitos no se contradigan entre sí
Compleitud	Que estén todas las funciones y restricciones propuestas
Realismo	Que se puedan implementar con la tecnología existente , dentro del presupuesto y la planificación
Verificabilidad	Que se pueda construir un conjunto de pruebas que demuestre que cada requisito se cumple; reduce las discusiones entre cliente y contratista

Un final pide explicar **por qué la validación es particularmente difícil**. Porque las especificaciones no siempre reflejan las necesidades reales, así que no alcanza con comparar contra un documento. Porque los usuarios tienen que **imaginarse el sistema funcionando** en su trabajo, y por eso **rara vez se encuentran todos los problemas** de requisitos al validarlos. Porque hace falta el cliente y un entorno que represente el previsto. Y porque, aun aprobados, aparecen cambios por omisiones y malas interpretaciones.

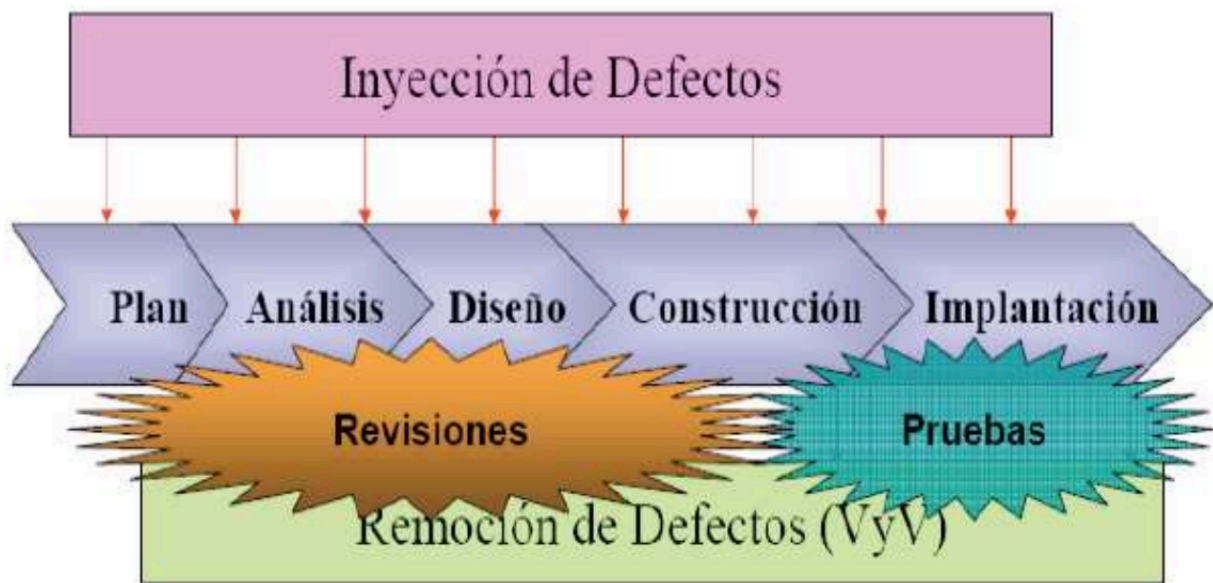
16.7 El grado de confianza

Verificar y validar no busca la ausencia total de defectos —que es inalcanzable— sino que el software sea **suficientemente bueno para su uso previsto**. Cuánta confianza hace falta depende de tres factores. La **criticidad del sistema**: el software que controla un sistema de seguridad crítico exige confianza alta; un prototipo para demostrar ideas, mucho menos. Las **expectativas del usuario**, que vienen subiendo porque la tolerancia a los fallos decrece. Y el **entorno de mercado**: con pocos competidores una empresa puede lanzar antes de estar completamente probada para llegar primero, y con un precio bajo los clientes toleran más defectos.

Conviene recordar que V&V son **procesos costosos**: en ciertos sistemas —los de tiempo real con restricciones no funcionales complejas, por ejemplo— superan **la mitad del presupuesto total** de desarrollo. Por eso se planifican desde etapas tempranas.

ATENCIÓN "El objetivo final de V&V es establecer **confianza** en que el sistema es adecuado" es **verdadero**, igual que "una compañía puede decidir lanzar antes de estar plenamente probada". Lo que no se sostiene es hacerlo a escondidas: testear hasta agotar el presupuesto de la fase y entregar sin avisar baja el grado de confianza sin que el cliente lo sepa. Lo correcto es informar y renegociar, o que entregar igual sea una decisión explícita del cliente, con los casos priorizados por riesgo. (El encuadre ético es conocimiento general.)

16.8 Inyección y remoción de defectos



Los defectos **se inyectan en todas las fases** —plan, análisis, diseño, construcción e implantación—, no sólo al programar. Verificación y validación son el conjunto de actividades de **remoción**, y se reparten en dos franjas que se solapan: las **revisiones**, que son estáticas, cubren desde el plan hasta la construcción; las **pruebas**, que son dinámicas, cubren desde la construcción hasta la implantación. INTECO lo plantea igual: **al final de cada fase** se evalúa el producto y se decide si pasa a la siguiente.

De ahí la regla económica que ordena toda la unidad: **cuanto más tarde se detecta un defecto, más caro sale corregirlo**. Una presentación complementaria de la cátedra, que no entra en el parcial, le pone número: en mantenimiento cuesta alrededor de **cien veces más** que en requisitos.

Eso responde una pregunta de final: **el riesgo de relegar las pruebas al final** del ciclo de vida es detectar los defectos cerca de la implementación, cuando corregirlos es muy caro, con aumento de presupuesto y atraso; y los defectos inyectados en requisitos y diseño viajan hasta el final. Las alternativas consisten en adelantar V&V: el **modelo en V** (capítulo 13), que planifica y diseña las pruebas de cada nivel en paralelo a cada fase, con una validación temprana —la revisión de los requisitos de usuario— y una tardía —la aceptación—; los ciclos **incrementales e iterativos**, que prueban en cada incremento; los **prototipos**; y las **técnicas estáticas** desde los requisitos.

16.9 Cuando las fuentes no dicen lo mismo

Algunas diferencias entre las fuentes aparecen como opciones de examen, y conviene saber qué criterio usar en cada una.

La prueba de aceptación. CMMI, en VER SP 1.1, incluye las "pruebas de aceptación" entre los ejemplos de métodos de verificación; INTECO dice que la aceptación sólo entra en validación, y el propio CMMI, en VAL SP 1.3, dice que los procedimientos de aceptación pueden cubrir los de validación. Una forma de conciliarlo es que la lista de VER SP 1.1 es de **ejemplos de métodos**, y un mismo método puede servir a las dos áreas según contra qué se lo use, como la inspección sirve a VER y a PPQA. En el examen se tomó **aceptación** → **VAL**: la ejecuta el cliente para confirmar que el producto le sirve en su entorno.

Las dinámicas y la validación. INTECO dice que las técnicas dinámicas están "más orientadas al área de validación" y las estáticas "ayudan más a la verificación", pero en su tabla de niveles pone la integración y el sistema en VER, y sólo la aceptación en VAL. Manda el criterio de siempre: **contra qué y con quién**. Una función probada por el equipo contra su especificación es VER, aunque la prueba sea dinámica.

La presentación introductoria. Una slide de la introducción a la calidad asocia la verificación con QA —"¿se construye el producto de manera correcta según el proceso definido?"— y la validación con QC, con pruebas de caja negra al final. No coincide con CMMI, donde VER incluye pruebas unitarias, de integración y de sistema, y controlar contra el proceso definido es terreno de PPQA; la misma presentación, en su tabla de QA y QC, pone las revisiones como ejemplos de QC. Regla práctica: pregunta de concepto sobre esa slide, la slide; ejercicio de áreas, CMMI.

La guía de requisitos. La guía de la unidad de requerimientos le atribuye a la validación la pregunta "¿el sistema se está desarrollando correctamente?", que en Boehm es la de la verificación. Para el examen, Boehm. Y la diferencia entre Myers e INTECO sobre la "confianza" se ve en 17.8.

17. La organización de las pruebas

Probar parece sencillo: usar el sistema y ver si anda. Pero si nadie decide de antemano qué se prueba, con qué datos y qué resultado se espera, cada prueba depende de la inspiración de quien la ejecuta, no queda registro de qué pasó, y cuando el código cambia no hay nada que repetir. Las organizaciones prueban para mejorar la calidad, bajar los costos de mantenimiento, suavizar los ciclos de liberación, cumplir con la legalidad y reducir riesgos, y nada de eso se logra probando a ojo. Este capítulo trata de cómo se organiza la actividad y con qué actitud se la encara.

17.1 Qué es un caso de prueba

Caso de prueba: conjunto de **entradas, condiciones de ejecución y resultados esperados**, desarrollado para un objetivo o condición particular, como verificar un requisito.

Myers lo reduce a dos componentes: la descripción de los **datos de entrada** y la descripción **precisa de la salida correcta**. En su forma mínima es un par ordenado: (**valor de entrada** → **resultado esperado**). Sin resultado esperado no hay caso de prueba, porque no habría forma de saber si pasó o falló. Para armarlo hacen falta además las **precondiciones y postcondiciones**.

Como las pruebas exhaustivas son imposibles (17.8), toda prueba trabaja sobre un **subconjunto** de los casos posibles, idealmente elegido con **políticas** de la organización —"toda sentencia se ejecuta al menos una vez"—

y con las técnicas del capítulo 18. Por eso es **falso** que alcance con un caso para probar si un requerimiento se cumple: como mínimo hacen falta uno positivo y uno negativo (18.7).

Nunca se prueba en producción: el entorno de pruebas debe estar **físicamente separado** y recrear las condiciones de producción.

17.2 El proceso de pruebas

INTECO trata las pruebas como un **subproyecto** con su propio plan, cuya eficacia se mide desde la perspectiva del proyecto. Ser eficiente es **evitar redundancias** —sin abandonar la estrategia ante el primer retraso— y **reducir costos**: herramientas y entornos sólo si se justifican. En el ciclo de vida, la **planificación** define la estrategia de pruebas y su estimación, el **diseño** diseña los casos, y en **codificación y pruebas** se ejecutan.

El proceso abarca la definición, elaboración, ejecución y evaluación de las pruebas, la resolución de los **casos fallidos** y algo que suele sorprender: la **documentación de usuario**. Los manuales de usuario y de administración se elaboran dentro del proceso de pruebas —INTECO los pone como salida de las pruebas de sistema— y sus errores se corrigen por revisión entre pares. Por eso, si preguntan en qué fase de una cascada se confecciona el manual de usuario, la respuesta es **prueba**, y los artefactos que sirven para escribirlo son la **interfaz gráfica** y los **casos de uso**, no el código ni el informe de pruebas unitarias. Las demás salidas son el producto probado, los planes e informes, y los **elementos de prueba** —scripts, programas, datos—, que van al repositorio de gestión de configuración (capítulo 20).

Plan de pruebas: describe el **alcance**, el **enfoque**, los **recursos** y la **planificación** de las pruebas: qué se prueba, quién hace cada tarea, el entorno, los riesgos y la contingencia, las técnicas de diseño y los **criterios de entrada y salida**.

Informe de pruebas: lo elaboran los **técnicos de pruebas**. Recoge los resultados, la evaluación contra los criterios de salida, el resultado final y **todas las ejecuciones de cada caso hasta superarlo**.

El **sistema de pruebas** tiene cuatro componentes —el **equipo**, los **recursos** (casos, datos, herramientas), los **procesos** y el **entorno**— y ninguno compensa la falta de otro: "el mejor equipo con malas herramientas consigue resultados mediocres". Su calidad se mide con ISO 9126.

17.3 Las estrategias de prueba

Como no es rentable detectar y corregir todos los fallos, la **estrategia** busca el equilibrio entre una **tasa de defectos aceptable** y la **inversión**, según el riesgo del negocio, y combina una dominante con otras. INTECO distingue seis. La **analítica** analiza requisitos y diseño: es minuciosa y buena para mitigar riesgos, pero cara en tiempo (su variante conocida es la basada en riesgos). La **basada en el modelo** incluye la basada en escenarios, como los casos de uso. La **metódica** usa estándares como objetivo: es rápida en sistemas estables, pero los cambios significativos la frenan. La **conformista** sigue un estándar externo, como IEEE 829. La **dinámica**, típica de los ágiles, planifica poco y se concentra en las últimas etapas, pero no da información de cobertura ni detecta defectos temprano. Y la **filosófica** parte de una creencia: la exhaustiva, la *shotgun* —que reparte el esfuerzo al azar según recursos y agenda— o la guiada externamente, que confía en que usuarios o soporte encuentren los errores.

Lo que analiza la estrategia analítica se llama **base de las pruebas**, y de ahí sale una afirmación de final que es **falsa**: que los artefactos de análisis y diseño se definen en función de las pruebas. Es al revés: "los productos de trabajo de desarrolladores y analistas son las bases de las pruebas", dice INTECO.

17.4 Los cuatro niveles de prueba

Los niveles se organizan por la secuencia en que las porciones del sistema quedan listas, y a veces se solapan a propósito. INTECO, además, asigna cada uno a un área:

NIVEL	QUÉ PRUEBA	QUIÉN Y CÓMO	ÁREA
Unitarias	Cada módulo o componente aislado, antes de integrar	El propio desarrollador , junto con el diseño y la construcción; usa stubs y drivers para aislar el módulo	Las dos
Integración	La interacción entre módulos ya integrados: defectos de interconexión	Ingeniero de pruebas y jefe de desarrollo, apoyados sobre todo en el diseño	VER
Sistema	El comportamiento global contra la especificación, funcional y no funcional	Un equipo independiente , con caja negra y un entorno lo más parecido a producción	VER
Aceptación	Que el producto satisface las necesidades del usuario	El usuario o cliente ; el jefe de proyecto hace el cierre formal	Sólo VAL

Las unitarias son de las dos áreas por razones distintas: VAL, porque detectan defectos y ayudan a construir el producto correcto; VER, porque comprueban estándares de codificación, modularidad y encapsulamiento. Los niveles se encadenan: la integración entrega la **línea base de sistema**; el sistema entrega el sistema probado y los **manuals de usuario y de administración**; la aceptación entrega el **producto aceptado** y la **línea base de producción**. En las de sistema, las técnicas de caja blanca se usan para **valorar la minuciosidad** de las pruebas.

Ningún nivel reemplaza a otro. Que haya pruebas de integración no quita que se hagan las unitarias, y viceversa.

Para armar la integración hay tres estrategias. **Big-bang** ensambla todo de una vez: no requiere simular nada, pero consume mucho tiempo rastreando causas y descubre los problemas al final; suele ser una mala elección. **Bottom-up** avanza desde los módulos inferiores: necesita **drivers** en cada nivel y no encuentra problemas de diseño hasta muy avanzado, pero es apropiada para orientación a objetos. **Top-down** avanza desde los componentes superiores: necesita **stubs** que simulen los inferiores, pero **descubre rápidamente los errores de arquitectura**.

17.5 Las pruebas de aceptación: alfa, beta y piloto

Las pruebas de aceptación son básicamente **funcionales sobre el sistema completo**, y **no buscan errores**: demuestran que el producto **está listo para producción**. Su ejecución es **facultativa del cliente**: si no se hacen explícitamente, se dan por incluidas en las de sistema. Cuando el producto es de **mercado masivo** y no se puede probar con cada cliente, se hacen en dos etapas, alfa y beta, a las que la materia suma el piloto:

MODALIDAD	QUIÉN	DÓNDE
Alfa	Un conjunto acotado de clientes preseleccionados. Para INTECO, se invita al cliente , con un experto siempre a mano, y el desarrollador registra errores y problemas de uso	Un entorno controlado; para INTECO, el entorno de desarrollo
Beta	Un conjunto más amplio, después de la alfa . Para INTECO, el cliente se queda a solas con el producto e informa los fallos	El entorno del cliente
Piloto	Un conjunto reducido de departamentos del cliente	Sus instalaciones, en ambiente de producción

La versión de clase y la de INTECO son compatibles; si una pregunta cita la guía, usá la textual.

Para clasificar una prueba hay tres ejes que conviene no mezclar: **por quién prueba** —internas, del equipo de desarrollo, o externas, del cliente: alfa, beta, piloto—, **por qué se prueba** —los cuatro niveles— y **por cómo se diseñan**: caja negra o caja blanca (capítulo 18).

ATENCIÓN Si un final pregunta cómo clasificar unas pruebas que ejecuta **un equipo de la empresa que desarrolla, usando un prototipo**, no son alfa ni beta: las dos requieren al cliente. Son **internas** y, por área, **VER**. La cátedra no dio respuesta oficial; es la que se deduce de las definiciones.

17.6 Los tipos de prueba

El **nivel** indica cuándo y sobre qué parte se prueba; el **tipo**, con qué objetivo. Los tipos que se nombran son las pruebas funcionales, las de prestaciones, las de usabilidad, las de seguridad o acceso, las de configuración, las de instalación y carga inicial de datos, y las de migración de datos; cuáles se hacen lo indica el plan de pruebas. INTECO llama **funcionales** a las que prueban lo que el sistema hace, y **no funcionales** a las que prueban atributos de calidad: prestaciones, usabilidad y también la regresión.

Las **funcionales** son de caja negra: comprueban, a partir de la especificación de requisitos o de los casos de uso, que el sistema cumple sus funciones, mirando entradas y salidas sin el funcionamiento interno. Las de **usabilidad** miden cuán fácil e intuitivamente interactúan los usuarios —navegación, organización de la información, etiquetas y mensajes—, habitualmente por **uso asistido**: un grupo de usuarios trabaja con el sistema y se anotan sus dificultades. Dentro de las de **prestaciones** hay cinco que se confunden:

PRUEBA	QUÉ BUSCA
Carga	Validar los requisitos de prestaciones definidos —tiempo de respuesta para N usuarios— con escenarios realistas
Capacidad	El punto umbral a partir del cual las prestaciones se degradan, incrementando la carga hasta la saturación
Estrés	El comportamiento en sobrecarga , excediendo los límites, con foco en la integridad
Escalabilidad	La capacidad de absorber requisitos mayores de prestaciones
Estabilidad	El comportamiento en el tiempo , bajo carga normal y durante un período largo, para detectar mala liberación de recursos

Con eso se clasifican las actividades que aparecen en los finales. Según la resolución de un final, coherente con INTECO: comprobar que la aplicación corre en los navegadores establecidos es una prueba **no funcional de configuración**, a nivel de sistema; la navegabilidad entre páginas es de **usabilidad**; recorrer los caminos de los casos de uso elegidos es **funcional**, también de sistema.

17.7 Regresión y confirmación

Son dos actividades distintas que se ejecutan juntas después de cada corrección. La **confirmación** verifica que **el defecto corregido realmente se solucionó**: misma prueba, mismas condiciones, mismos datos. La **regresión** verifica que **el arreglo no rompió otra cosa**.

La regresión puede detectar tres situaciones: que el cambio **creó un error nuevo** (regresión local), que **reveló errores que ya existían** (de exposición), o que el cambio en un área **rompió otra área** del sistema (remota, la más difícil de detectar, porque todos confían en lo que ya andaba).

Un ejercicio de final lo deja claro. Un módulo tiene 125 casos de prueba y fallan 27. Cuando desarrollo devuelve las correcciones, hay que ejecutar **los 125**: los 27 que fallaron como **confirmación** y los otros 98 como **re-**

gresión; además, por el principio 9 de Myers, donde hubo errores es más probable que haya más. Re-ejecutar sólo los 27 es confirmación, no regresión.

La estrategia más simple es la **fuerza bruta**, repetir todas las pruebas, y por eso la regresión es la mejor candidata a **automatizarse**: es la única manera de repetirlo todo en sistemas grandes, y conviene cuando su costo, alto, se recupera porque las pruebas se ejecutan con frecuencia. La otra alternativa es **seleccionar** qué re-ejecutar por **trazabilidad** (qué casos están vinculados a lo que se tocó), **análisis de cambios** y **análisis de riesgos de calidad**. De las herramientas de pruebas en general, el riesgo principal son las **expectativas poco realistas**.

17.8 Qué es probar: psicología y economía de la prueba

Myers sostiene que las pruebas pobres nacen de una **definición falsa** de qué es probar. Hay tres que suenan razonables y son trampas de multiple choice: que la prueba es el procedimiento para **demostrar que los errores no están presentes**; que su propósito es demostrar que un programa **realiza correctamente** sus funciones; y que es el proceso de **establecer la confianza** en que un programa hace lo que debe. La correcta es otra:

Prueba (Myers): el proceso de **ejecutar un programa con la intención de encontrar errores**.

De ahí salen dos definiciones que van contra la intuición: un **buen caso de prueba** es el que tiene alta probabilidad de encontrar un error todavía no descubierto, y un **caso de prueba exitoso** es el que **lo descubre**.

La **economía de la prueba** parte de que las pruebas exhaustivas son imposibles: de caja negra, porque probar todas las entradas de un programa tan simple como el que clasifica triángulos requeriría infinitos casos; de caja blanca, porque un programa pequeño puede tener del orden de **cien trillones** de secuencias lógicas. No se puede garantizar un programa libre de errores, y la economía pasa a ser la consideración central. Dijkstra lo resumió: "*las pruebas sólo pueden demostrar la presencia de errores, no su ausencia*".

ATENCIÓN Si preguntan si un sistema está **libre de errores** porque todas las pruebas de caja negra se planificaron y ejecutaron a conciencia, la respuesta es **no**: por Dijkstra, y porque nunca se probó "todo". Al terminar sólo se puede afirmar que el sistema **pasó los casos ejecutados** y alcanzó un **grado de confianza** para su uso previsto. Decir que "cumple con las especificaciones" se pasa de rosca.

Una tensión entre fuentes: Myers pone como falsa la definición de prueba como "establecer confianza", e INTECO dice que las pruebas "intentan proporcionar confianza". Se resuelve mirando qué se pregunta: si es **qué es probar según Myers**, ejecutar con la intención de encontrar errores; si es **el objetivo de V&V**, establecer confianza es verdadero (16.7).

17.9 Los diez principios de Myers

Son el marco de actitud con el que se encara el testing, y condensan buena parte de lo anterior.

1. Una parte **necesaria** de un caso de prueba es la definición de la **salida prevista**.
2. Un desarrollador debe evitar **probar su propio programa**.
3. El personal de prueba **no debería depender** del área de desarrollo.
4. **Inspeccionar concienzudamente los resultados** de cada prueba.
5. Escribir casos **tanto para las condiciones de entrada esperadas como para las no esperadas**.
6. Examinar un programa para comprobar **que no hace lo que no se supone** que haga: un programa de sueldos que emite bien los cheques pero también los emite para empleados inexistentes está mal.
7. Evitar casos de prueba **desechables y sin documentar**, salvo que el programa sea desechable.

8. **No planificar** el esfuerzo de pruebas suponiendo que no se encontrarán errores.
9. La probabilidad de encontrar errores adicionales en una sección es **proporcional al número de errores ya encontrados** en esa misma sección: los errores aparecen en grupos.
10. Las pruebas son una tarea **altamente creativa** y un desafío intelectual.

Los principios 2 y 3 merecen desarrollo, porque suelen preguntarse. Hay tres razones por las que quien desarrolla no debe probar su propio código. La primera es de actitud: **desarrollar es un proceso creativo y probar es un proceso destructivo**, y cuesta cambiar de una mentalidad a la otra sobre el trabajo propio. La segunda es la **visión de túnel**: quien desarrolló tiende a probar los caminos que ya sabe que funcionan. La tercera es la más grave: si el error está en **cómo se entendió el requerimiento**, quien lo entendió mal va a probar según su propio malentendido; por eso se prueba también **con el cliente**. Y el personal de prueba no debe depender de desarrollo porque compartiría sus **plazos y presiones** —con el incentivo de declarar que el sistema anda— y su visión del requerimiento.

El principio 7 tiene su justificación de final: los casos inventados sobre la marcha se pierden, y cuando haya que volver a probar después de una mejora se reinventan, menos rigurosos, y un error introducido por la modificación pasa desapercibido; guardarlos para re-ejecutarlos **es** la prueba de regresión. El 8 se ve con un ejemplo: si se planifica un solo día de pruebas sin margen y aparece un error, no queda tiempo para corregirlo ni para la regresión. Y el 9 tiene una consecuencia directa: conviene concentrar el esfuerzo **en los módulos donde más defectos aparecieron**.

17.10 Buenas prácticas

Las buenas prácticas que recoge INTECO resumen el capítulo. La prueba es un proceso **continuo e iterativo** a lo largo de todo el ciclo de vida, ordenado y sistemático, que no depende de la inspiración del tester: previsto, planificado y documentado. Cada caso se asocia **al menos a un requisito**, lo que da **trazabilidad**; las pruebas, en especial las de regresión, son **repetibles y automatizables**; el entorno es **estable**, lo más parecido a producción, y controla la promoción de nuevas versiones mientras se prueba. Los desarrolladores no prueban su propio código, y los elementos de prueba van al repositorio de gestión de configuración.

ATENCIÓN En las preguntas BP de pruebas (capítulo 25) las señales de problema se repiten: el programador prueba su propio programa, los datos de prueba se deciden **al ejecutar**, los casos no tienen resultado esperado, no se registran los resultados, en mantenimiento se prueba **sólo lo modificado**, o la única prueba es la del usuario. Que haya **testers dedicados** es bueno: si el enunciado los tiene pero improvisa los datos, el problema está en el diseño, no en quién prueba.

18. Las técnicas dinámicas

Las técnicas dinámicas **ejecutan** el software y comparan la salida con el resultado esperado: buscan **fallos**. Existen porque **no se puede probar exhaustivamente** (17.8): cada técnica es una forma sistemática de elegir un subconjunto de casos con alta probabilidad de encontrar defectos. Las **basadas en la especificación**, o de **caja negra**, se concentran en **qué** hace el software y sacan los casos de la especificación; las **basadas en la estructura**, o de **caja blanca**, parten del código; y las **basadas en la experiencia** se apoyan en lo que saben usuarios y técnicos.

Antes de ver cada una, conviene tener el criterio de selección, porque los enunciados suelen describir la situación y pedir la técnica:

LO QUE DICE EL ENUNCIADO	TÉCNICA
Un campo con rango, longitud o formato; "valores válidos e inválidos"	Partición de equivalencia o valores límite
Varias condiciones que se combinan, reglas de negocio que se cruzan	Tablas de decisión
Una secuencia de estados, "pasa de X a Y"	Transición de estados
Flujo básico y flujos alternativos, escenarios de punta a punta	Casos de uso
Se dispone del código; cobertura de sentencias o decisiones	Caja blanca
No hay especificación adecuada o no hay tiempo	Basadas en la experiencia
Volver a probar lo que ya funcionaba después de un cambio	Regresión

18.1 Particionamiento de equivalencia

La idea es agrupar las condiciones de entrada que **el sistema trata igual**. Si el programa funciona para un valor de la partición, se asume que funciona para todos; si falla para uno, se asume que falla para todos. Por eso alcanza con probar **un representante por partición**.

Las particiones se dividen en **válidas** —las que el sistema debe aceptar— e **inválidas** —las que debe rechazar—. También existen las **particiones de salida**, que agrupan por resultado: si el sistema aplica tres tasas de interés distintas, cada tasa define una partición.

La tabla del formato de la cátedra tiene cuatro columnas —**atributo, tipo, particiones válidas, particiones inválidas**—, cada partición lleva un código único (AV1, AI1 por atributo: si te olvidaste una, la agregás sin renumerar) y el tipo se escribe con el nombre del enunciado. Qué particiones salen de cada campo:

CAMPO	VÁLIDAS	INVÁLIDAS
Rango ("10 a 100")	Una; o una por tramo si la regla de negocio cambia el resultado	Por debajo y por encima
Lista de opciones ("ROJO, BLANCO, NEGRO")	Una si todas se comportan igual; una por valor si el comportamiento cambia	Valor fuera de la lista
Condición de obligación ("letras mayúsculas")	Una	Una
Cadena con longitud	La longitud permitida	Más larga, más corta que el mínimo, caracteres no permitidos
Fecha	Un solo campo, con una partición para los años bisiestos	Fecha inexistente (29/02 de un año no bisiesto, 31/04), algo que no es fecha
Cualquier campo	—	El vacío , siempre aparte; y una por cada tipo distinto del esperado (real, booleano, fecha)

El vacío no es lo mismo que una cadena corta, y si el campo es **opcional** pasa a ser una partición **válida**. La partición por tipo no hace falta en los campos de texto, donde un número entra como texto.

Un ejemplo: un campo "día del mes", entero positivo entre 1 y 31, con un recargo que cambia cada diez días.

PARTICIONES

Válidas PV1) $1 \leq X \leq 10$ · PV2) $11 \leq X \leq 20$ · PV3) $21 \leq X \leq 31$

Inválidas PI1) letras · PI2) $X \leq 0$ · PI3) $X > 31$ · PI4) vacío · PI5) imagen · PI6) carácter especial · PI7) cadena de caracteres

La segunda tabla lista los casos, uno por partición, con cuatro columnas: caso, partición, entrada y salida esperada. Serían diez: tres válidos (1, 12, 30) y siete inválidos.

ATENCIÓN Dos errores arruinan este ejercicio. El primero es **olvidar las particiones intermedias**: si un sistema se activa "por encima de 25" y se apaga "por debajo de 20", entre 20 y 25 hay una zona donde no pasa nada, y esa zona **es una partición válida**. El segundo es **dejar que las particiones se pisen**: si la válida llega hasta 31, la inválida es $X > 31$, no $X \geq 31$. La versión de este ejemplo que circuló en clase tenía justamente ese error.

18.2 Análisis de valor de frontera

Es una **mejora** del particionamiento, no una alternativa. En vez de un solo representante por partición, prueba **más de un caso en cada una**, concentrados en los **extremos**, porque es donde se agrupan los errores: si un campo debe aceptar de 1 a 10, lo probable es que acepte valores justo afuera o rechace los que están justo en el límite.

Para un rango de 10 a 100 se prueban **9, 10, 100 y 101**. Si el sistema trabaja con dinero a dos decimales, el "siguiente valor" es 0,01: la frontera de 15.000 se prueba con 14.999,99 y 15.000,00. Para conjuntos ordenados, el primer y el último elemento. Y hay valores especiales: en minutos, siempre 0 y 59; en fechas, los límites de mes y los **años bisiestos y no bisiestos**.

Los valores límite se aplican a atributos **con rango**: números, longitudes, fechas. Si no hay rango, queda la partición pura. Para campos no numéricos, el criterio dado en clase es: si el campo admite **una letra cualquiera**, alcanza un caso; si admite un **conjunto cerrado** de valores, uno por valor; si es una **cadena de longitud fija**, esa longitud exacta; y si acepta **hasta N caracteres**, el límite se toma sobre la longitud: $N-1$, N y $N+1$, y según una consulta con la cátedra también un carácter (máximo $20 \rightarrow 1, 19, 20$ y 21). La cátedra pidió además incluir en la tabla de límites un caso por cada inválida que no es rango, como un booleano en un campo entero; no todos lo dan por seguro, pero incluirlas no resta.

Cuando una consigna pide "cubrir todas las particiones válidas", pide **representantes**, no bordes. Cuando pide valores límite, pide los **extremos**. Es la diferencia entre las dos técnicas y es lo que se evalúa.

Frente a la **prueba aleatoria**, los valores límite son sistemáticos y reproducibles y apuntan adonde se concentran los errores; la aleatoria puede no tocar nunca las fronteras.

18.3 De las particiones a los casos de prueba

Los casos se escriben en uno de dos formatos. **Por campo**, atributo por atributo, con las columnas ID, atributo, partición, entrada y salida esperada. **De función**, en conjunto, con una columna por atributo y la salida esperada; ahí se arranca con casos de **todos los valores válidos** —si un atributo tiene varias particiones válidas, se prueban todas— y después va **un inválido por vez**, dejando el resto válido. En los finales se pidieron los dos formatos, así que hay que leer la consigna. Si el enunciado no da el formato de la salida, alcanza con "OK" o "Error".

Elegir entre partición pura y valores límite cambia los valores que salen de la misma partición:

	PARTICIÓN PURA	VALORES LÍMITE
Qué valores	Un representante cualquiera de cada partición	Los extremos: el último válido y el primer inválido de cada lado
Edad con tramos 000–024 / 025–065 / 066–999	010, 040, 080 (más –3, 1500...)	000, 024, 025, 065, 066, 999, –1, 1000
Qué detecta	Que cada tramo se trate como corresponde	Además, los errores de frontera: un $>$ en lugar de $\geq$

La de límites es mejor porque prueba justo donde cambia la regla: si el código dice > 50000 en lugar de ≥ 50000 , un representante como 30.000 no lo detecta; 50.000,00 y 50.000,01 sí.

El AD 2025 preguntó **qué hay que tener en cuenta al definir los casos de prueba a partir de las particiones**, con varias opciones correctas. Lo que sí: las **entradas**, las **salidas esperadas** y el **comportamiento esperado** —es la definición misma de caso de prueba—, y la **técnica a emplear**, porque partición pura y valores límite sacan valores distintos de la misma partición. Lo que no: **si la prueba es alfa o beta** y **si la ejecuta personal de desarrollo**, que clasifican pruebas pero no cambian el diseño del caso, ni las **limitaciones del lenguaje**, porque es caja negra. Las **restricciones del negocio** quedan dudosas: ya están en las particiones.

ATENCIÓN La corrección sólo confirmó que "la técnica a emplear" era correcta y que marcar "alfa o beta" costaba el punto; el resto se deduce de Myers y de INTECO. El error de fondo es mezclar un criterio de **clasificación** de las pruebas con el **diseño** del caso.

Si el ejercicio lo pide, falta un paso: dejar la **base de prueba** con datos de ejemplo que hagan funcionar los casos.

18.4 Los errores que se repiten en los finales

Las resoluciones de alumnos de finales viejos que circulan como práctica repiten los mismos errores, y casi todos son de traducción del enunciado a particiones:

EL ENUNCIADO DICE	EL ERROR TÍPICO	LO CORRECTO
"40 % de descuento si la edad es superior a 65"	Partición ≥ 65 "	66 en adelante : si no, un pasajero de 65 recibe el descuento
"Los mayores de 17 ingresan pagando" (edad entera)	Una partición válida de un solo valor (17)	Partición ≥ 18 ; los límites son 17 y 18
"Monto mayor a cero"	Inválida "menor a 0"	Inválida ≤ 0 : el 0 también es inválido y quedaba sin cubrir
Válida de 1 a 31	Inválida $X \geq 31$ "	$X > 31$, para que no se pise con la válida
"5 % de descuento si las noches son más de 6"	Descuento desde 6	Desde 7 ; los límites son 6 y 7
Tipo de moneda con recargos distintos	Una sola partición válida	Una por valor , porque el comportamiento cambia
Un campo donde 0 significa "sin límite"	Inválida "menor a 1"	El 0 es válido , con partición propia; la inválida es ≤ -1 (≤ -2 si el -1 también tiene significado)

EL ENUNCIADO DICE	EL ERROR TÍPICO	LO CORRECTO
Una salida booleana (<code>guardado_ok</code>)	Particiones de entrada para la salida	La salida no se particiona como una entrada
Una reunión con el cliente para evaluar la especificación	"Prueba funcional, de caja negra"	Estática: no se ejecuta nada; por el cliente, VAL

Tres criterios más salen de los mismos finales. Cuando el resultado depende de una **combinación** —una estadia de hotel que cruza de temporada baja a alta, con precio por noche—, la combinación es una partición propia. Una aclaración como "los descuentos no son acumulables, se aplica el mayor" **no genera particiones** si los tramos no se superponen. Y cuando el enunciado deja algo abierto, hay que **declarar el supuesto**: la cátedra corrige por la justificación. Un último aviso: un ejemplo de clase usaba (1, 2, 6) como triángulo escaleno, pero $1 + 2 < 6$, así que ni siquiera es un triángulo; para el escaleno sirve (3, 4, 5).

18.5 Tablas de decisión

Se usan cuando **múltiples combinaciones de entradas** producen resultados distintos, y ahí la partición y los valores límite se vuelven difíciles de usar. A diferencia de esas técnicas, que miran un campo por vez, esta se centra en la **lógica y las reglas de negocio** que cruzan varios campos.

La tabla tiene filas de **condición** y filas de **acción**, y **cada columna es una regla de negocio**, es decir, un posible caso de prueba.

Armarla tiene cuatro pasos. Se identifican las **condiciones**, que son los atributos que deciden el resultado. Se identifican las **acciones**, que son los resultados distintos que el sistema puede ejecutar. Se generan las **columnas**, combinando los valores posibles de cada condición. Y se **reduce** la tabla, eliminando las combinaciones imposibles y colapsando las indiferentes.

Un ejemplo que aclara el conteo. Un gimnasio tiene socios de categoría Coulson, IronMan (10 % de descuento) o Hulk (15 %), y quien tiene la cuota al día recibe un 5 % adicional acumulable. Las condiciones son dos: la categoría y el estado de la cuota. Pero la categoría **no es binaria**: tiene tres valores mutuamente excluyentes. Entonces las columnas son $3 \times 2 = 6$, y las acciones son **cuatro**: aplicar 0 %, 10 %, 15 % y el 5 % adicional.

ATENCIÓN El error típico es contar toda condición como binaria y calcular 2^n columnas. Una condición puede tener tres o más valores, y en ese caso el producto cambia. Y a la inversa: cuando hay **condiciones incompatibles** —si el usuario no existe, la contraseña es irrelevante—, las combinaciones se colapsan y quedan menos casos de los que sugiere la potencia de dos.

Un final muestra cuándo elegirla: una calificación que depende de la edad —de 7 a 14 años— y del peso, con un criterio distinto para cada edad. La edad no da una partición sino **ocho**, y para el conjunto la técnica más apropiada es la **tabla de decisión**: son múltiples condiciones que, combinadas, llevan a distintas acciones.

18.6 Transición de estados

Se aplica a sistemas modelables como **máquina de estados finitos**, donde la salida ante la misma entrada **depende del estado anterior**. El ejemplo canónico es un trámite que pasa de Inscripto a Cursando y de ahí a Aprobado o Desaprobado.

Partiendo del diagrama de estados, la técnica permite revisar qué hace falta para llegar a cada estado y detectar incompatibilidades: **transiciones que faltan**, o estados de los que no se puede salir. Es floja para identificar

pruebas negativas, y por eso una prueba completa no se limita al camino feliz: debe incluir las **transiciones no válidas** —intentos fallidos, timeouts— y los **eventos no especificados**, como cancelar a mitad de camino.

18.7 Derivación de casos de prueba desde casos de uso

Las técnicas anteriores miran un campo o una regla por vez. Los casos de uso permiten probar el sistema **de punta a punta**, siguiendo el recorrido real de un usuario. Por eso sirven sobre todo en los niveles de **sistema y aceptación**, descubren **defectos de integración** y los del **uso real**, y dan casos de **mejor calidad** que los armados campo por campo: validar una tarjeta en un cajero contempla muchas más cosas que probar tres números sueltos en un formulario. Para derivarlos, el caso de uso tiene que especificar sus **precondiciones** y su **estado final**.

La guía es la directriz *Caso de prueba* de RUP, cuya traducción despista: "**guión de prueba**" es el caso de prueba y "**caso de ejemplo**" es el escenario. La heurística base: cada requisito necesita al menos un caso **positivo**, que demuestra que se alcanza, y uno **negativo**, que demuestra que sólo se alcanza en las condiciones deseadas. El método tiene cuatro pasos, que se ven con el cajero automático de la presentación de la cátedra.

Primero, la tabla de escenarios. Un escenario es el **flujo básico** solo, o combinado con uno o más **alternativos**. En el cajero, E1 es el retiro satisfactorio (sólo el flujo básico); E2, la tarjeta no válida (alternativo 1.a); E3 y E4, el PIN incorrecto con y sin intentos restantes; E5 a E7, el cajero sin fondos, la cuenta sin fondos y el máximo diario alcanzado; y E8, la cancelación, que puede ocurrir en cualquier paso.

Segundo, la matriz V/I: una fila por caso de prueba y una columna por cada condición o dato, tanto las entradas como las **condiciones que valida el sistema**, aunque vengan de la base. **V** marca la condición válida, que sigue el flujo básico; **I**, la que **dispara el alternativo**; **n/a**, la que no llega a evaluarse. Un fragmento:

CP	ESCENARIO	TARJETA	FONDOS		RESULTADO ESPERADO
			PIN	EN CUENTA	
1	E1 – Retiro satisfactorio	V	V	V	Retiro satisfactorio
2	E2 – Tarjeta no válida	I	n/a	n/a	Expulsa la tarjeta con un mensaje; fin del caso de uso
3	E4 – PIN incorrecto, sin intentos	V	I	n/a	Mensaje; retiene la tarjeta; fin del caso de uso
4	E6 – Fondos insuficientes en la cuenta	V	V	I	Mensaje; vuelve al paso 3

La matriz muestra de un vistazo **qué condiciones quedaron sin probar**: una columna sin ninguna I. **Tercero**, se repite con valores reales. **Cuarto**, si se pide, se describe el **contenido de la base antes de probar**: sólo las tablas y columnas que intervienen, sin claves primarias.

Las reglas de resolución son pocas. El primer caso es **positivo** del flujo básico; los demás son **negativos del flujo básico y positivos de su alternativo**. Un escenario puede necesitar **más de un caso**: el PIN incorrecto admite "quedan intentos", "no quedan" y "acierta en el último". Las **reglas de negocio** se prueban **dentro, fuera y en el límite**: un importe mayor que el saldo va al alternativo; uno menor y uno igual siguen el flujo básico. **No se especifican bucles**, se cubren sólo los caminos que plantea el caso de uso, el mensaje va textual si el caso de uso lo da, y al final se eliminan los casos duplicados.

ATENCIÓN El error típico está en la matriz. En un caso de uso de alquiler de vehículos, la resolución de un final marcaba como I la **fecha** y la **duración** para provocar "no hay vehículos disponibles", cuando esos datos son válidos: lo que dispara el alternativo es la **disponibilidad**, una condición que valida el sistema con datos de la base y que necesita su propia columna.

Los casos de uso no alcanzan solos: las **especificaciones complementarias** también generan casos. Las de **rendimiento** piden al menos un caso por sentencia de rendimiento y por caso de uso crítico, con valores **por debajo, en y por encima del umbral** (carga: mil cajeros responden en menos de 30 segundos; estrés: dos cajeros piden la misma cuenta a la vez). Las de **acceso y seguridad** comprueban que **sólo los actores especificados** ejecuten cada caso de uso. Las de **configuración** piden un caso por configuración crítica y por la más problemática —el hardware de menor rendimiento, la conexión más lenta—. Las de **instalación** cubren instalación nueva, completa, personalizada y actualización. Y para la **aceptación** se usan los mismos casos o un subconjunto, con los criterios **acordados antes** con el cliente.

18.8 Caja negra y caja blanca

La distinción es qué información tiene quien prueba.

En **caja negra** sólo se ven entradas y salidas. Es rápida y no requiere acceso al código, pero cuando algo falla **no se sabe dónde está el error**. Todas las técnicas anteriores son de caja negra.

En **caja blanca** se dispone del **código fuente**, lo que permite saber cuántas sentencias y condiciones se ejecutaron y sacar un porcentaje de cobertura. Trabaja sobre tres estructuras: secuencia, selección e iteración.

TÉCNICA	OBJETIVO
Pruebas de sentencia	Ejecutar cada sentencia ejecutable al menos una vez. Es una cobertura demasiado débil como medida de efectividad
Pruebas de decisión	Evaluar cada decisión (IF-THEN-ELSE, DO-WHILE) en verdadero y falso
Pruebas de caminos	Recorrer cada camino de ejecución independiente. No prueba todas las combinaciones: con bucles serían infinitas

Conviene notar que un defecto puede manifestarse **aunque todas las sentencias se hayan ejecutado al menos una vez**, porque el problema aparece recién al **combinarse** ciertos caminos. La cobertura del 100 % de sentencias no garantiza ausencia de defectos.

Ver el código también **mejora los casos de caja negra**, porque permite identificar particiones adicionales. Un final lo muestra con una función que devuelve la proporción A/B entre dos enteros que genera otro módulo: con el código a la vista —`return A/B`, sin ninguna validación— aparece un caso que hay que agregar, **B = 0**, que provoca una división por cero. En ese momento la prueba **pasó de caja negra a caja blanca**.

La relación funciona también al revés: **la caja negra puede dar la impresión de que todo está bien**, porque si el código tiene una rama que ninguna partición distingue, los casos pueden no recorrerla nunca; la cobertura de decisión obliga a ejecutarla. Pero la caja blanca sola tampoco alcanza: la prueba exhaustiva de caminos es imposible, y no detecta **lo que falta**, porque si un requisito no se implementó no hay código que recorrer (conocimiento general, coherente con INTECO, que hace las pruebas de sistema con caja negra).

La cantidad de pruebas que necesita un componente se estima con la **complejidad ciclomática**, que produce el análisis estático (19.8); su valor coincide con la cantidad de caminos independientes del código (conocimiento general).

18.9 Técnicas basadas en la experiencia

Se usan cuando **no hay una especificación adecuada** o **no hay tiempo**, y aprovechan la experiencia de usuarios y técnicos para elegir las áreas más importantes. La **adivinación de errores** complementa a las técnicas formales y depende de la habilidad e intuición del técnico. Las **pruebas exploratorias** consisten en recorrer el software para entender qué hace, qué no hace y dónde está débil, diseñando las pruebas mientras se ejecutan.

19. Las técnicas estáticas: revisiones

19.1 Por qué existen

Las técnicas dinámicas necesitan el sistema andando, y eso limita cuándo se pueden aplicar. Las **estáticas** —"técnicas de no ejecución"— analizan las representaciones del sistema: requisitos, diseños, código, historias de usuario, modelos, **sin ejecutar nada**, y por eso se pueden aplicar en **cualquier momento** del ciclo de vida. Buscan **defectos**, no fallos.

Son la **primera forma de prueba aplicable** en un proyecto y están vinculadas principalmente a la **verificación** (con la salvedad de 16.2: una revisión con el cliente es validación). Además de los defectos encuentran sus **causas**, lo que permite mejorar el proceso y reducir el retrabajo. Detectan desviaciones de estándares, requisitos ambiguos, diseños que no encajan con los requisitos, código difícil de mantener y especificaciones inconsistentes. Hay dos variantes: las manuales, que son las **revisiones**, y las automatizadas, que son el **análisis estático** (19.8).

	ESTÁTICAS (REVISIONES)	DINÁMICAS (PRUEBAS)
Qué encuentran	Defectos	Fallos
Qué necesitan	Documentos	El sistema ejecutable
Cuándo se aplican	En cualquier momento	Sólo con el producto andando
Ventaja	Se sabe dónde está el problema; se encuentran varios a la vez; se corrige temprano	Más rápidas de ejecutar
Desventaja	Más lentas	No se sabe dónde está el error

Son **complementarias**: ninguna reemplaza a la otra, y las dinámicas siguen siendo las predominantes.

19.2 Beneficios y costo

Las revisiones mejoran la calidad y la comprensión de los entregables, validan que soportan la solución final, gestionan las expectativas del negocio, identifican tareas de alto riesgo y forman al equipo. Al reducir los errores que llegan a la etapa de pruebas, **acortan los períodos de prueba y bajan sus costos**.

Aun así, muchas organizaciones no las implementan, y la explicación que da la materia es que tienden a **sobreestimar su costo y subestimar sus beneficios**.

La inspección tiene además tres ventajas concretas frente a las pruebas: en una prueba **un error puede enmascarar a otro**, y en una inspección no; se pueden inspeccionar **versiones incompletas**, sin construir software de soporte; y evalúa atributos más amplios que la corrección, como el cumplimiento de **estándares**, la **portabilidad** y la **mantenibilidad**.

19.3 Formalidad

Una revisión puede ser informal o formal, y la diferencia está en si hay proceso.

Las **informales** no tienen proceso definido, no tienen roles y habitualmente no se planean. Cualquier intercambio entre pares cuenta: preguntarle a un compañero si le parece bien un pedazo de código es una revisión informal.

Las **formales** tienen objetivos definidos, proceso documentado, roles asignados a personas entrenadas, checklists y reglas, **reporte de resultados** y recolección de datos para el control del proceso.

La formalidad importa porque deja **trazabilidad documentada** de las acciones y decisiones, lo que permite demostrar después que los procedimientos se cumplieron.

19.4 El proceso y los tipos

El proceso básico es común a todas: se identifican los entregables a revisar, se arma la lista de participantes, los revisores **estudian** el documento por su cuenta, identifican problemas y se los **comunican al autor**, y el autor **responde y actualiza**.

TIPO	QUIÉN LO DIRIGE	FOCO	FORMALIDAD
Revisión informal	—	Encontrar defectos a bajo costo; documentar es opcional. Pasarle un borrador a un colega, programar de a pares	Mínima
Walkthrough	El propio autor, que guía paso a paso	Entendimiento común, evaluar contenidos, discutir alternativas. Útil cuando los asistentes no son del palo del software	Media
Revisión técnica	Un moderador capacitado o un experto técnico	Consenso técnico entre revisores expertos; no es búsqueda de defectos	Variable
Revisión entre pares	Colegas del mismo proyecto	Identificar y eliminar defectos temprano , de forma incremental	Media
Inspección	Un moderador formado, nunca el autor	Registrar defectos comparando el producto con sus fuentes y con checklists; las discusiones se posponen. Seguimiento formal con criterios de salida	Máxima

ATENCIÓN El walkthrough y la inspección se distinguen por quién dirige: el autor en el primero, un moderador que nunca es el autor en la segunda. Y la revisión técnica busca **consenso**, mientras la inspección **registra defectos**.

19.5 Las revisiones entre pares

Para INTECO, una revisión entre pares es un **examen metódico** de productos de trabajo, hecho **incrementalmente** por compañeros con interés y conocimiento del elemento, mediante inspecciones, walkthroughs u otros métodos; en CMMI es la meta SG 2 de VER (16.3). **Los gerentes no deberían participar**, porque limitan el diálogo abierto. Se aplican a mucho más que el código: también a planes, descripciones de procesos, documentación y material de formación.

Las guías para que funcionen son concretas: crear un **entorno seguro y no amenazante**, capacitar al personal en sus roles, **documentar los defectos** con ubicación, descripción y origen, **enfocarse en el producto y no en la persona**, e **incluir las revisiones en la planificación del proyecto**, para que tengan tiempo asignado y no se saltee la preparación. Sus ventajas, si un final las pide: detectan defectos **temprano**, recomiendan mejoras, **comunican** al equipo, **forman** a los menos experimentados y generan datos para prevenir defectos.

19.6 Los cinco roles

El **moderador** dirige el proceso, determina junto con el autor el tipo de revisión y la composición del equipo, y hace el seguimiento. El **autor** creó el documento y busca mejorar su calidad. El **documentador** anota cada defecto y sugerencia; en la práctica suele ser el propio autor. El **revisor** valida el material buscando defectos **antes** de la reunión. El **supervisor** decide destinar tiempo del proyecto a las revisiones y determina si se cumplieron los objetivos.

CMMI, en VER SP 2.1, nombra otros roles de ejemplo: **líder, lector, notario y autor**. No se contradice con INTECO, pero fíjate qué nombre usa la opción.

19.7 Los factores de éxito

Hacer una revisión no garantiza que sirva. INTECO enumera lo que la hace funcionar: un **objetivo claro y acordado**; elegir los **documentos más críticos**, como los requisitos o la arquitectura; usar el **tipo de revisión adecuado** al objetivo, **sin inspeccionar todo**; reservar las horas **en el plan del proyecto** y seguir el tiempo invertido; **formar** a los participantes, incluidos los aspectos psicológicos, para que la experiencia sea positiva para el autor; ajustar la formalidad a la **cultura y la madurez** de la organización; e **informar los resultados y beneficios** cuanto antes.

19.8 Análisis estático

Es la variante automatizada: busca defectos **sin ejecutar** el programa, pero **una vez escrito el código**, sobre el código fuente y los modelos, con herramientas llamadas **analizadores estáticos**. Son una gran ayuda para las inspecciones. INTECO distingue cinco tipos de análisis:

ANÁLISIS	QUÉ BUSCA
Flujo de control	Bucles con varias entradas o salidas, código inalcanzable
Uso de los datos	Variables no inicializadas , escritas dos veces sin uso intermedio, declaradas y nunca usadas
Interfaz	La consistencia entre la declaración de una rutina y su uso
Flujo de información	Las dependencias de las variables de salida; no detecta anomalías, resalta información para la revisión
Caminos	Los caminos del programa y las sentencias de cada uno

Detecta además inconsistencias entre módulos, vulnerabilidades de seguridad y violaciones de los estándares de programación. Sus ventajas son la detección temprana, la mejora de la **mantenibilidad**, la **prevención** (ataca la causa raíz) y algo que las pruebas dinámicas no pueden hacer: encontrar **inconsistencias en los modelos**. Sin herramientas, aplicar un estándar de codificación en una organización probablemente falle.

Produce también **métricas de código** —frecuencia de comentarios, profundidad de anidamiento— y, la más usada, la **complejidad ciclomática**, que sirve para **estimar cuántas pruebas** necesita un componente:

Complejidad ciclomática = sentencias de decisión binarias + 1

Un componente con tres decisiones binarias tiene complejidad 4. En la medición (capítulo 22) la misma métrica se calcula sobre el grafo de flujo —aristas menos nodos, más dos por cada componente conexas: un grafo de 9 aristas y 7 nodos da $9 - 7 + 2 = 4$ —, con la recomendación de que ningún módulo supere 10. En un programa estructurado las dos cuentas dan lo mismo (conocimiento general).

20. La gestión de configuración (CM)

Un producto de software es un conjunto de piezas —requerimientos, diseño, código, casos de prueba, manuales y hasta las herramientas con que se construyó— que cambian todas, a ritmos distintos y en manos distintas. Cuando la software factory vende el mismo producto a varios clientes, cada uno con su versión, y el equipo corrige errores en paralelo, aparecen las frases con que la guía de la cátedra retrata a un proyecto sin gestión de configuración: "¿cuál es la versión que tiene el cliente?", "no puedo reproducir el problema en mi versión", "¿está corregido el error también en esa versión?".

El riesgo principal es **entregar al cliente la versión incorrecta**: con errores, con cambios no probados o imposible de reproducir. Detrás vienen otros: no tener el inventario de componentes cuando hace falta, repetir pruebas porque se probó lo que no correspondía y **no poder recuperar una línea base anterior** para mantener a un cliente que sigue usándola. Lo que la gestión de configuración protege es la **integridad**.

Gestión de configuración (CM): establecer y mantener la **integridad** de los productos de trabajo utilizando la **identificación** de configuración, el **control** de configuración, el **registro del estado** de configuración y las **auditorías** de configuración.

Integridad quiere decir saber **exactamente qué se le entregó a cada cliente** y conocer el **estado y el contenido** de cada línea base y de cada elemento de configuración. Cada función del propósito responde una pregunta: la identificación, ¿cuáles son los elementos de configuración?; el control, ¿cómo se controlan sus cambios?; el registro del estado, ¿cuál es su estado actual?; la auditoría, ¿cumplen los requisitos?

CM es un área **de soporte de nivel 2**: no produce nada propio, sino que pone bajo control lo que generan las demás. Junto con PPQA y MA forma el grupo de **áreas de soporte básicas**, que dan servicio a todas las otras.

20.1 Metas y prácticas

Las tres metas cuentan una historia en orden: se **crean** las líneas base, se las **mantiene** con cambios controlados, y se **documenta y audita** su integridad.

- **SG 1 Establecer líneas base.** SP 1.1 Identificar elementos de configuración · SP 1.2 Establecer un sistema de gestión de configuración · SP 1.3 Crear o liberar líneas base.
- **SG 2 Seguir y controlar los cambios.** SP 2.1 Seguir las peticiones de cambio · SP 2.2 Controlar los elementos de configuración.
- **SG 3 Establecer la integridad.** SP 3.1 Establecer registros de gestión de configuración · SP 3.2 Realizar auditorías de configuración.

20.2 Qué es un elemento de configuración

Elemento de configuración (EC): cualquier producto de trabajo —final o intermedio, entregable al cliente o interno— **cuyo cambio pueda resultar crítico** para el proyecto. Para CMMI es una **agregación de productos de trabajo** que se trata como **una entidad única**.

Por lo general cada producto de trabajo es un EC; la diferencia aparece cuando varios se gestionan juntos. Un manual de usuario escrito por capítulos tiene un producto de trabajo por capítulo, pero es **un solo EC**, y el código de un proyecto suele ser un único EC, con su versión y sus autores (así lo explicó Ripani en clase, según un resumen de alumno).

En los parciales el término aparece junto a otros tres que conviene no mezclar (capítulo 8):

	QUÉ ES	EJEMPLOS
Producto de trabajo	Resultado útil de una tarea del proyecto ; no necesariamente se entrega	Una minuta, un informe de taller, el código fuente, un <code>.class</code>
Producto	El producto de trabajo que se entrega al cliente o al usuario final	La aplicación y sus manuales
Activo	Artefacto hecho para usarse en las tareas de los proyectos ; incluye las herramientas que la organización adquiere	Plantillas, Word, Java, OpenOffice

Ser EC no es una cuarta categoría: un producto, un producto de trabajo interno o una herramienta pasan a ser EC cuando **se decide ponerlos bajo control**.

20.3 Qué se pone bajo configuración

Identificar los EC (SP 1.1) es la base de todo lo demás: lo que no se identificó no se puede controlar, registrar ni auditar. Los criterios son los mismos en la guía y en CMMI: van bajo configuración los productos **usados por dos o más grupos**, los que **pueden cambiar** por cambios de requisitos o por errores, los que **dependen entre sí** y los **críticos** para el proyecto. En la práctica, los planes, la especificación de requisitos y la **matriz de trazabilidad**, el diseño, el **código fuente**, los **casos y datos de prueba**, los informes, los manuales y todos los entregables. CMMI agrega los **productos adquiridos** y lo que más se olvida: las **herramientas**.

ATENCIÓN Las herramientas también son elementos de configuración si hacen falta para reconstruir una versión: sin el lenguaje y el compilador **en la versión que se usó**, el código fuente no alcanza para regenerar el ejecutable.

La pregunta del año 2002. El AD 2024 describió una software factory que desde 2002 desarrolla aplicaciones de escritorio en un lenguaje A; hace dos años sumó apps móviles en un lenguaje B; aplica un estándar de interfaces desde 2010; sus manuales eran PDF hechos con MS-Word 2002, y en los últimos diez años los reemplazó un módulo de manual on-line. La pregunta: "si estuviera en el año 2002, ¿qué debería quedar bajo la gestión de configuración para garantizar la evolución que se tuvo?".

El criterio es **pararse en el año del enunciado** y poner todo lo que **existía entonces y hace falta para reconstruir y hacer evolucionar** el software: la documentación del sistema, la app de escritorio, su **código fuente**, el **lenguaje A**, **MS-Word 2002** y los manuales en PDF. Lo que apareció después **no puede estar**: ni el lenguaje B —marcarlo dejó la pregunta en cero—, ni el estándar de interfaces, ni el módulo on-line.

ATENCIÓN Queda sin confirmar si valían los **manuales en .doc**: no se marcaron, y el formulario no muestra las correctas omitidas. A favor: guardar Word 2002 sólo sirve si también se guarda el archivo editable del manual. Conviene marcarlos, sabiendo que es un riesgo. El comentario del corrector —"se pide productos de software, no productos de trabajo"— es ambiguo; lo seguro es que valieron el código fuente y las herramientas.

Desde la perspectiva de las **pruebas**, CM sirve para **controlar la versión de los casos de prueba, identificar qué versión del software se está probando y seguir los cambios** de los casos de prueba; no sirve para desarrollar casos nuevos ni para detectar que hacen falta.

20.4 Las líneas base

Línea base (LB): conjunto de productos de trabajo **revisado y acordado formalmente**, que sirve de base para el desarrollo posterior y que **sólo puede cambiarse mediante el procedimiento de control de cambios**.

Para entrar en una línea base, un producto tiene que ser un EC y estar **acabado y formalmente aprobado**; lo que dispara la incorporación es una tarea de aceptación formal, típicamente una **revisión formal**. La **línea base más los cambios aprobados** forman la **configuración vigente**. Crearla o liberarla es la SP 1.3: hace falta la **autorización del comité de cambios**, se arma sólo con EC que ya están en el sistema de configuración y se **documenta qué EC contiene**.

Lo más importante que hace una línea base no está en la definición: **relaciona entre sí las versiones de los distintos artefactos**. Así lo muestra el ejemplo de la guía para un ciclo en cascada, donde las letras son versiones:

HITO (REVISIÓN FORMAL)	CONTENIDO DE LA LÍNEA BASE
Revisión de requisitos	ERS A
Revisión de diseño	ERS B · Diseño A
Disponibilidad de las pruebas	ERS C · Diseño B · Código A · Plan de pruebas A
Aceptación	ERS D · Diseño C · Código B · Plan de pruebas B · Manual de usuario A · Base de pruebas de regresión A

Sin esa relación no hay forma de saber que el producto aceptado se armó con la versión D de la especificación y la B del código: es lo que falla en el BP de versionado del AD 2024 (sección 20.12).

ATENCIÓN Los tipos de línea base dependen de la fuente. La guía habla de línea base **funcional** (requisitos revisados), **de desarrollo y de producto** (lo entregado); CMMI, de **funcional, asignada y del producto**. Si la pregunta cita CMMI, la intermedia es la **asignada**.

20.5 Versiones, variantes y releases

Con varios clientes en versiones distintas hay que poder identificar la de cada uno, saber **en qué versión entró cada cambio y reconstruirla**. La cátedra fija los dos términos básicos en el enunciado de un final:

Versión: variación **temporal** de un producto o de un EC: el mismo elemento, evolucionado en el tiempo.

Variante: variación **espacial**: el mismo producto adaptado a otro ambiente, plataforma o lenguaje, que **coexiste** con los demás.

Una app para clínicas que pasa de Android 1.1 a Android 4.3 sin cambiar su funcionalidad **no es una versión nueva, es una variante**. Dentro del número de versión, la **mayor** sube con un cambio de funcionalidad o cuando la versión pasa a ser definitiva, y la **menor**, con correcciones o cambios chicos. Algunos esquemas agregan un tercer nivel, la **revisión** (V1.02.01), aunque ninguna fuente oficial lo define: sale de la corrección de un final hecha por un alumno.

Release: versión de lanzamiento, en la que el software **se hace público**.

Es la definición que dio el enunciado del AD 2024, y en el parcial manda ésa; la guía usa **liberación** en un sentido más amplio, que incluye las distribuciones internas. De ahí sale otra pregunta del mismo parcial: el **mínimo de releases** es la cantidad de veces que el software se hace público obligatoriamente, y lo decide el ciclo de vida (capítulo 13). En **cascada** es **una**: hay una sola entrega, al final. En un **incremental con n incrementos** son **n**, porque cada incremento se entrega al usuario.

ATENCIÓN En el enunciado real el producto se llamaba "ALFA 3" y tenía 4 incrementos: la respuesta es **4**. El "3" es parte del nombre y no cuenta.

La **política de versionado** es la regla escrita que dice el **formato** de la versión, **con qué valor arranca** cada componente, **cuándo sube** cada uno, si los demás **se reinician** y qué pasa con los **cambios simultáneos**. Si el producto está hecho de módulos, versiona los dos niveles —en un final se propuso **vXX.YY** para los módulos y **vXX.YY.ZZ** para los productos— y **registra qué versión de cada módulo compone cada versión del producto**, que es la línea base.

20.6 Cuánto sobrevive un esquema de versión

Un esquema de numeración tiene capacidad finita: con dos dígitos hay 99 valores, y cuando se agotan, el esquema "muere". El AD 2024 pidió calcular cuántos años dura uno, y este método reproduce la respuesta que se puntuó como correcta:

1. Contar los **valores** de cada componente: dos dígitos desde 01 son 99; desde 00, 100.
2. Fijar la **frecuencia** de cada uno: trimestral, 4 por año; mensual, 12; quincenal, 24 a 26.
3. Si el menor **se reinicia** cuando cambia el mayor, verificar que **no se desborde dentro de un período del mayor**. Si no se desborda, el que limita es el mayor.
4. Supervivencia $\approx$ **valores del mayor / frecuencia anual del mayor**.

El enunciado: versión **<x>.<y>**; **x** cambia con la funcionalidad, tiene dos dígitos y la primera es 01; **y** sube con cada corrección, tiene dos dígitos y **vuelve a 0 cuando cambia la funcionalidad**; la primera versión es 01.00; la funcionalidad cambia **cada trimestre** y los errores se corrigen **cada quince días**.

¿Se desborda y?	Un trimestre tiene unas 6 correcciones quincenales: y llega a 06 o 07, lejos de 99. Manda x.
Capacidad de x:	de 01 a 99 → 99 versiones funcionales.
Tiempo:	4 cambios por año → $99 / 4 \approx 24,75$ años.
Respuesta:	24, la opción más cercana sin pasarse.

ATENCIÓN El distractor es dividir los 99 valores de **y** por las quincenas del año, como si no se reiniciara: da menos de cuatro años. Contar desde 00 daría 25, que no estaba entre las opciones, y 28 ya excede la capacidad del esquema.

20.7 La convención de nombres

Todo EC necesita un **identificador único**, y un buen nombre permite además **deducir los atributos del archivo sin abrirlo** y **filtrar** grupos de archivos por prefijo. La cátedra nombra así todos sus apuntes:

PARTE	SIGNIFICADO	VALORES
MM	Asignatura	IS , Ingeniería de Software
TTT	Tipo de activo	ART plantilla de artefacto · ARTD descripción de artefacto · CAT información de la cátedra · CHK checklist · EJEMP ejemplo · ENUN enunciado · PRACT práctica · PRES presentación · PRO proceso · TEOR apunte teórico
AA	Área, de 2 a 4 caracteres	CM · EXAM · GRAL · INT · MA · PP · PPQA · PROC · REQM · RSKM · TP · VyV
nn	Número correlativo (se deduce de los ejemplos)	01 , 02 ...
nombre	Descripción	Texto libre, sin espacios ni acentos
x	Versión mayor	Un dígito: 0 mientras no es definitiva, 1 la primera definitiva
yy	Versión menor	Dos dígitos , más una letra minúscula desde la b si el cambio es sólo de formato de impresión

Así, IS-TEOR-CM02_..._v1_01 es el apunte teórico número 02 del área de configuración, en versión definitiva 1, menor 01, e IS-ENUN-EXAM214 es el enunciado 214 del área de exámenes: el parcial AD de 2024. Para **proponer** un formato, cada pregunta que el nombre debe responder (¿qué módulo?, ¿qué tipo?, ¿qué versión?) se vuelve una parte; las partes van **de lo general a lo particular**, y cada una lleva su significado, su longitud y sus valores posibles.

ATENCIÓN El formato del nombre no es la **política de versionado**. Si se pide la política, se espera el formato de la versión, su valor inicial y cuándo sube cada número; según una guía de resolución de alumnos, contestar con la estructura del nombre es un error repetido en los finales.

20.8 Los repositorios y el control de versiones

No tiene sentido controlar igual el borrador que un programador tiene en su máquina y la versión entregada al cliente. Por eso el **sistema de gestión de configuración** (SP 1.2) —almacenamiento, procedimientos y herramientas— distingue tres repositorios:

REPOSITORIO	QUÉ CONTIENE	CONTROL
Dinámico (de desarrollo o de autor)	Lo que se está creando o revisando , en el entorno del desarrollador	Control de versiones , a cargo del propio desarrollador
Máster (controlado)	La línea base actual y sus cambios	Control de configuración completo
Estático	Las líneas base ya liberadas y archivadas	Control de configuración completo

CMMI gradúa lo mismo en **niveles de control**: de **creación** (controla el autor), de **ingeniería** (se notifica a los interesados), de **desarrollo** (controla el nivel inferior del comité de cambios) y **formal** (su nivel superior, **con el cliente**). En el día a día, un archivo no se modifica sin **desprotegerlo** (check-out) y vuelve como **versión nueva** al protegerlo (check-in); la línea base está en un área restringida que controla una sola persona, y tras cada cambio se corren **pruebas de regresión**. El sistema incluye también los permisos de acceso, que el gestor de

configuración fija al inicio del proyecto, y las **copias de seguridad**, guardadas en un sitio distinto al de los originales.

ATENCIÓN **Control de versiones no es gestión de configuración formal:** a algunos productos les alcanza con que su dueño sepa qué versión está en uso; otros necesitan líneas base acordadas formalmente. Esa decisión existe en **todas** las áreas como práctica genérica **GP 2.6 Gestionar configuraciones:** poner bajo control los registros de **otra** área —los informes de PPQA, por ejemplo— es la GP 2.6 de esa área, no una práctica de CM.

20.9 El control de cambios y el comité

Los cambios son inevitables. Lo que CM garantiza es que **sólo entren los aprobados**, que se sepa **en qué versión y en qué línea base** entró cada uno, y que todos trabajen con las versiones correctas.

Comité de control de configuración (CCB): o comité de control de cambios. **Evalúa y aprueba o rechaza** los cambios propuestos a los EC, asegura que se implementen los aprobados y autoriza las líneas base. En un proyecto chico puede ser el líder o una persona asignada.

Necesidad de cambio → solicitud sobre un EC → ¿está completa? (si no, se completa)
→ análisis de impacto → revisión del CCB
→ rechazada: se informa al solicitante
→ aprobada: se asigna, se planifica, se implementa, se prueba y se cierra

La solicitud la inicia **cualquiera, en cualquier momento**, y no trata sólo requerimientos: en CMMI las peticiones de cambio incluyen también **fallos y defectos**, y se registra el tipo —defecto o mejora— para sacar métricas. El **análisis de impacto** mira el producto, los **productos relacionados**, el presupuesto y el calendario. El comité puede **aceptar, modificar, rechazar o aplazar**, y la decisión **siempre queda documentada**; la autorización puede tener varios niveles según la criticidad o el impacto, y CMMI admite que venga del CCB, del jefe de proyecto o del cliente. Al cerrar, una auditoría de configuración y una verificación de calidad confirman que **sólo se hicieron los cambios aprobados**.

Las dos prácticas de SG 2 se reparten el circuito. La **SP 2.1** registra la petición, analiza el impacto y la **sigue hasta su cierre**, cuanto antes: las abiertas inflan las listas, cuestan y confunden. La **SP 2.2** controla el elemento que cambia: autorización, check-in y check-out, revisión de efectos no deseados y registro del cambio y de su razón; para CMMI, **los cambios no son oficiales hasta que se liberan**. Un ejemplo de un resumen de alumno: se pide cambiar el código de una línea base; el análisis de impacto muestra que también cambia el manual, y se abre una **segunda petición**; las dos se aprueban, el proceso técnico que corresponde —**no CM**— hace las modificaciones, y CM registra ambos cambios y libera la **nueva línea base**.

El **gestor de configuración** escribe el plan de gestión de configuración (SCMP, según IEEE 828), y tanto el plan como las actividades del comité están sujetos a revisión y auditoría de **PPQA**. Los estados por los que pasa una solicitud de cambio están en el capítulo 15.

20.10 El registro del estado y las auditorías

El **registro del estado** (SP 3.1) guarda el historial de cada EC con detalle suficiente para **recuperar versiones anteriores**, y dice **qué versión de cada EC constituye cada línea base y qué diferencias hay entre líneas base sucesivas**. Por eso "cada vez que se genera una compilación se genera un documento con las modificaciones respecto de la anterior" es SP 3.1.

Auditoría de configuración: verificación de que un EC, o el conjunto de EC que forman una línea base, se ajusta a un estándar o requerimiento especificado.

Se hace en **puntos clave** del ciclo de vida, y **una auditoría exitosa es requisito para establecer la línea base del producto**. Comprueba que los EC estén en el **directorío que corresponde**, que su estado sea **consistente**, que la información de las líneas base esté al día y que se hayan cumplido los procedimientos; por ejemplo, que se haya codificado a partir de **la versión correcta del diseño**. Integra el equipo un representante de pruebas, y el resultado es un informe con las **no conformidades**, que se **siguen hasta el cierre**. CMMI distingue tres tipos:

AUDITORÍA	VERIFICA QUE...
Funcional (FCA)	Las características funcionales del EC, ya probadas , logran los requerimientos de su línea base funcional
Física (PCA)	El EC tal como fue construido es conforme con la documentación técnica que lo define
De gestión de configuración	Los registros de configuración y los EC son completos, consistentes y exactos

ATENCIÓN La auditoría de configuración **no evalúa el producto en sí**: comprueba que esté **bien gestionado**, con **todos los cambios registrados** y **trazabilidad** entre los cambios y los productos afectados. No es VER (¿cumple su especificación?) ni PPQA (¿se siguió el proceso de la organización?). Si el enunciado habla de la integridad de las líneas base o de los EC, es **CM SP 3.2**, aunque la haga gente de QA.

La **liberación** cierra el ciclo: el producto se **construye a partir de la línea base**, se libera según la **severidad de los problemas** y la **densidad de defectos** de la versión, y se entrega empaquetado en sus versiones correctas, con **notas de versión**: funciones nuevas, problemas conocidos y requisitos de plataforma.

20.11 Cómo distinguir CM de las áreas vecinas

CM comparte vocabulario con REQM (los cambios), con PPQA (las auditorías y los estándares) y con OPD (los repositorios), y los ejercicios de relacionar actividad con área juegan con eso:

LO QUE DICE EL ENUNCIADO	ÁREA Y PRÁCTICA
Se decide qué archivos y herramientas se guardan y con qué identificador	CM SP 1.1
Se definen las carpetas, los permisos y las copias de seguridad del repositorio	CM SP 1.2
Un conjunto de artefactos, revisados y acordados formalmente , pasa a ser la base del resto del desarrollo	CM SP 1.3
Llega un cambio de requisito y todavía se evalúa cómo afecta	REQM SP 1.3 , no CM
El cambio ya se decidió aceptar y se siguen las modificaciones a los artefactos	CM SP 2.1
Se controla que los nombres de archivo cumplan las reglas del proyecto	CM : en la práctica se resolvió como SP 2.2, aunque podría defenderse SP 3.2
Personal externo al proyecto controla que se cumpla un estándar de la organización	PPQA SP 1.2 , no CM

Cada compilación genera un documento con las diferencias respecto de la anterior	CM SP 3.1
Se verifica que lo construido coincide con su documentación técnica, o que los registros de configuración son exactos	CM SP 3.2
Se arma la biblioteca de activos o el repositorio organizacional de plantillas y procesos	OPD, no CM
Se ponen bajo control de versiones los registros de otra área	GP 2.6 de esa área

ATENCIÓN El análisis de impacto aparece **en REQM y en CM**, así que no sirve para separarlas. Decide **el momento**: si todavía se evalúa si conviene aceptar el cambio, es REQM; si ya se aceptó y se tramita, es CM. Ese criterio sale de las notas del cuestionario de la cátedra. Si el enunciado no da una pista temporal, fijate en el objeto: el requerimiento es de REQM; la petición de cambio y el EC, de CM.

20.12 CM en las preguntas BP

En el formato BP (capítulo 25), los problemas de configuración tienen una firma reconocible: cada artefacto se versiona **sin relación con los demás**, se guarda **sólo la última versión**, la documentación vive en **carpetas personales** con nombres a criterio de cada uno, los cambios entran **sin petición ni autorización**, las solicitudes se **cierran sin actualizar** la documentación o el pase a producción se hace **de palabra y sin registro**. La consecuencia es siempre la misma: no se sabe qué versión tiene el cliente ni se la puede reconstruir.

El BP del AD 2024 es el modelo. La software factory vende productos a varios clientes y tiene una política que identifica la versión del producto final y la de cada artefacto, pero **cada artefacto tiene su número independiente y no se guarda la relación con las versiones de los otros**. Todo queda en el repositorio de cada proyecto, que se elimina recién cuando el producto se discontinúa. La respuesta es **"sí podría generar problemas de calidad, porque en caso de ser necesario volver a una versión anterior no hay información sobre la relación de versiones entre artefactos"**: falta la línea base, el identificador que agrupa una colección de EC (SP 1.3) y el registro de qué versión de cada uno la compone (SP 3.1).

ATENCIÓN La opción que se marcó en el parcial —"el problema principal es que no se almacenan en un **repositorio organizacional**"— fue **incorrecta**: que el repositorio sea por proyecto no es lo que exige CM, y el repositorio organizacional es tema de OPD. También son falsas "es una implementación correcta de CM, dado que se conservan todas las versiones" —las versiones sueltas no alcanzan— y "es una implementación correcta de OPD", que ni siquiera es el área en juego. Sólo está confirmado que la marcada era incorrecta; que la correcta sea únicamente la de la relación entre versiones se deduce de SP 1.3 y SP 3.1.

21. El aseguramiento de la calidad (PPQA)

Un proyecto atrasado o pasado de presupuesto tiende a saltar el proceso: la revisión prevista no se hace, la trazabilidad no se actualiza, un requerimiento se cambia por teléfono. Nadie lo decide de mala fe; es lo que pasa bajo presión. PPQA existe para que alguien **que no está sometido a esa presión** mire si el proceso se sigue y se lo diga a quien puede corregirlo. Lo dice CMMI: cuando evalúan personas que no son responsables directas del proceso, "el aseguramiento creíble de la adherencia puede proporcionarse incluso en los momentos en los que el proceso se encuentra bajo estrés".

Aseguramiento de la calidad de proceso y de producto (PPQA): proporcionar al personal y a la gerencia una **visión objetiva** de los procesos y de los productos de trabajo asociados.

Es un área **de soporte de nivel 2**. Evalúa **los procesos ejecutados, los productos de trabajo y los servicios** contra las **descripciones de proceso, los estándares y los procedimientos** aplicables; es decir, mira la **adherencia**: si se trabajó como estaba definido. Como da servicio a todas las áreas, la práctica genérica **GP 2.9 Evaluar objetivamente la adherencia**, que tiene cada una, se implementa a través de PPQA.

ATENCIÓN PPQA **no evalúa contra requerimientos**: eso es VER. CMMI lo dice en una frase: PPQA asegura que **los procesos planificados se implementan**; VER, **que se satisfacen los requerimientos especificados**. Pueden mirar el mismo producto, pero desde perspectivas distintas.

21.1 Objetividad e independencia

La palabra clave del propósito es **objetiva**, y CMMI la descompone en dos ingredientes: **independencia** y **uso de criterios**.

Evaluar objetivamente: revisar actividades y productos de trabajo frente a **criterios que minimizan la subjetividad y el sesgo** del revisor.

La forma tradicional de lograr independencia es un **grupo de QA independiente del proyecto**. CMMI admite otra: en una organización con **cultura abierta y orientada a la calidad** —lo más factible en organizaciones **pequeñas**—, la evaluación puede hacerse **parcial o totalmente entre pares**, con tres condiciones: evaluadores **formados en QA, separados** de quienes desarrollan el producto evaluado, y un **canal independiente** para escalar a la gerencia.

Lo que excluye a alguien de evaluar un producto es **haber participado en armarlo**; tener poca experiencia en QA (se lo forma) o ser nuevo no lo excluyen. Las formas de evaluar van de las **auditorías formales** de grupos de QA separados a las revisiones entre pares, las "auditorías de escritorio" y las revisiones distribuidas: las menos formales dan cobertura diaria, y las formales se hacen periódicamente.

ATENCIÓN El criterio rápido para un BP: si el evaluador depende de **la misma cadena jerárquica** que el evaluado —QA dentro de Programación, o SQA y líderes de proyecto bajo el mismo gerente que decide sobre los proyectos—, **no hay independencia** y hay problema de calidad.

21.2 Metas y prácticas

- **SG 1 Evaluar objetivamente los procesos y los productos de trabajo.** SP 1.1 Evaluar objetivamente los procesos · SP 1.2 Evaluar objetivamente los productos de trabajo y los servicios.
- **SG 2 Proporcionar una visión objetiva.** SP 2.1 Comunicar y asegurar la resolución de las no conformidades · SP 2.2 Establecer registros.

La primera meta **evalúa** los procesos y productos **designados**, que pueden elegirse por muestreo; la segunda hace que la evaluación **sirva para algo**: comunica, asegura la resolución y registra. QA **empieza temprano**, participando cuando se definen planes, estándares y procedimientos para que después sirvan de criterio, y evalúa los productos **antes de entregarlos al cliente**, en hitos y en forma incremental. La primera subpráctica de SP 1.1 pide además **un entorno que incentive al personal a comunicar los problemas de calidad**; un resu-

men de alumno agrega "sin represalias", que no está en CMMI pero es la lectura correcta: castigar a las personas por las no conformidades desalienta que se informen.

Para relacionar una actividad con su práctica:

LO QUE DICE EL ENUNCIADO	PRÁCTICA
Personal externo controla si se siguió el procedimiento o el workflow definido	SP 1.1
Personal externo controla si un artefacto cumple la plantilla o la directriz de la organización	SP 1.2
Se informa, eleva o escala una no conformidad; se la sigue hasta que se cierra	SP 2.1
Se registra, archiva o actualiza el estado de las actividades de QA	SP 2.2
El estándar es del proyecto : nombres de archivo, líneas base	No es PPQA: CM
Se compara contra requerimientos o artefactos del proyecto : minutas, glosario	No es PPQA: VER
Participa el usuario o el cliente	No es PPQA: VAL
Se usan los informes de no conformidades para mejorar el proceso de la organización	No es PPQA: OPF

21.3 Las no conformidades: del hallazgo al cierre

No conformidad (NC): problema identificado en una evaluación que refleja **falta de adherencia** a los estándares, las descripciones de proceso o los procedimientos aplicables.

La SP 2.1 es la práctica más preguntada, porque sus subprácticas describen el **ciclo de vida de una no conformidad**: se intenta **resolverla en el proyecto**; si no se puede, se **documenta** y se **escala** al nivel de gerencia designado; se analizan las **tendencias** de calidad, se informa a los interesados y se revisan periódicamente las abiertas; y se **sigue cada una hasta su resolución**. Resolverla admite **tres caminos**, en una lista cerrada: **corregirla, cambiar el estándar o la descripción de proceso** que se incumplió, u **obtener una excepción**.

De ahí salen afirmaciones que los parciales toman casi textualmente:

AFIRMACIÓN	RESPUESTA	POR QUÉ
"Una auditoría termina cuando se resuelven las no conformidades" (AD 2024)	Verdadera	Presentar el informe y registrar la auditoría son pasos intermedios
"El trabajo de auditoría termina cuando se presentan las no conformidades" (AD 2025)	Falsa	Por lo mismo
"Las no conformidades se intentan resolver a nivel proyecto" (AD 2025)	Verdadera	Es el primer paso de SP 2.1
"El escalamiento es un aviso que dice que no hay no conformidades" (AD 2025)	Falsa	Es lo contrario: se escala una no conformidad que no se resolvió
"Las no conformidades deben escalarse aunque se hayan resuelto" (banco de preguntas)	Falsa	Sólo se escala lo que no se pudo resolver
"Una no conformidad podría cambiar la descripción de un proceso" (banco de preguntas)	Verdadera	Es una de las tres formas de resolverla

ATENCIÓN El escalamiento no puede faltar. En la corrección de un final, Ripani marcó como lo más importante de la respuesta de PPQA la frase "en caso de persistir esta situación, se podrá elevar el problema a la gerencia acompañado de un registro respaldatorio": **sin ella, la respuesta era incorrecta.**

21.4 El proceso de aseguramiento de la cátedra

La cátedra adopta un proceso de aseguramiento de la calidad de INTECO: el **equipo de calidad audita periódicamente** la ejecución de los procesos de cada proyecto, desde el **arranque** hasta el **cierre**. Los roles son el **SQA** —el asesor de calidad del proyecto, asignado **antes de que el proyecto empiece** y que **reporta al responsable de QA**, no al jefe de proyecto—, el **responsable de QA**, el **jefe de proyecto** y el **gerente del proyecto**.

Al planificar, el SQA colabora con el jefe de proyecto para que las actividades de QA queden en la **EDT y en el calendario**, aporta lecciones aprendidas y ayuda a elegir el ciclo de vida y a adaptar los procesos; **el jefe de proyecto prepara el plan de calidad y el SQA lo revisa**, igual que revisa los planes de proyecto, de riesgos y de configuración. Durante la ejecución, el SQA hace las **auditorías periódicas** —de procesos y también de configuración—, ayuda a prevenir defectos, **verifica que las métricas que van a la base de la organización sean completas y correctas** e informa los hallazgos al jefe de proyecto y al responsable de QA. Al cierre, asegura que el informe retrospectivo llegue al histórico de la organización. Cuándo auditar también está pautado:

SITUACIÓN	CUÁNDO SE AUDITA
Regla general	Al final de cada fase del ciclo de vida
El paso entre dos fases se alarga mucho	Revisión bimensual
Mantenimiento evolutivo pequeño	Al menos a mitad del proyecto y antes de terminar
Pequeñas peticiones de mantenimiento correctivo	Revisión aleatoria mensual

El reparto que más sirve para los BP: **corregir la no conformidad le corresponde al jefe de proyecto**, pero **auditar, seguir y escalar le corresponde al SQA**, que lleva lo no resuelto al **responsable de QA y al gerente**. El checklist con que la cátedra audita al propio proceso de QA precisa que se escala por **dos causas** —que **no se acuerde** una acción correctiva, o que la acordada **no se cierre en fecha**— y que la decisión de la autoridad superior **queda registrada**. Entre las métricas del proceso están las no conformidades detectadas y solucionadas, por tipo y por área, y el **tiempo medio de solución**.

ATENCIÓN El proceso y el checklist no nombran igual el destino del escalamiento —el gerente del proyecto en uno, el director técnico en el otro—, y CMMI sólo pide el nivel de gerencia designado con un canal independiente. Para el examen, lo seguro es **"fuera del proyecto, a una gerencia independiente de la que maneja el proyecto"**.

21.5 El sector de SQA no es el área PPQA

Según una guía de resolución de alumnos, el **sector** de SQA de una empresa suele hacer dos cosas: **evaluar** procesos y productos, que es PPQA, y **proponer mejoras** a procesos y plantillas, que es OPF; dejarlas en la biblioteca de activos es de OPD. El ejemplo de la guía encadena las tres áreas: OPD crea la plantilla de casos de prueba, PPQA controla si los proyectos la completan bien, y si varios informes muestran que no, OPF propone una guía de llenado. Por eso "el sector de SQA comunica por mail los cambios de plantillas" describe un **despliegue** (OPF SP 3.1), no PPQA.

Tampoco es lo mismo **aseguramiento** que **control** de la calidad. La guía complementaria de gestión de la calidad distingue el **QA**, que revisa de forma planificada y periódica la **ejecución** del proyecto para asegurar que si-

que los estándares —mira el proceso y es preventivo—, del **QC**, que inspecciona **resultados específicos** y decide su aceptación o su retrabajo.

21.6 PPQA en las preguntas BP

Las señales de problema se repiten: el evaluador está **dentro del equipo o bajo la misma gerencia** que decide sobre el proyecto; la no conformidad **no se sigue hasta el cierre**; **no hay escalamiento**, o se escala a quien es juez y parte; las no conformidades se **archivan** o se **sancionan** en lugar de resolverse.

Los dos BP de PPQA de los parciales describen la misma práctica con un final distinto: SQA controla estándares de proceso y de producto, informa las no conformidades, el líder de proyecto debe subsanarlas y SQA vuelve a revisar **a los 15 y a los 30 días**. Lo que cambia es qué pasa si la no conformidad sigue abierta:

	AD 2024	AD 2025
Si la NC sigue abierta...	SQA escala al gerente de desarrollo, "con quien se tiene buena comunicación ya que tanto los PMs como SQA dependen de esa gerencia"	El informe se archiva en "proyectos incumplidores" y queda en el legajo del líder
El eslabón que falta	Independencia : el gerente que recibe el escalamiento manda también sobre los PMs	Escalamiento y seguimiento hasta el cierre : después del día 30 nadie sigue la NC
La opción correcta	"Sí... porque el área de SQA debería depender de otra gerencia para evitar conflicto de intereses al escalar"	"Sí... porque no contempla un mecanismo para controlar, por parte de un grupo externo al desarrollo, que se resuelvan las no conformidades hasta su fin , escalándolas si fuera necesario"

En 2024 hay seguimiento y escalamiento, pero el escalamiento va a quien es **juez y parte**: cuando el proyecto se atrasa, ese gerente tiene incentivos para priorizar la entrega por sobre la no conformidad, y la "buena comunicación" es un distractor. En 2025 **no hay escalamiento**: sancionar al líder **no resuelve** la no conformidad, y usar mediciones para evaluar personas es un **uso inapropiado** de los datos (capítulo 22).

ATENCIÓN La opción "no contempla un mecanismo para controlar... que se resuelvan las no conformidades" aparece **en los dos parciales**: en 2024 es **falsa**, porque el mecanismo existe; en 2025 es **la correcta**. Hay que leer el final del enunciado, no responder de memoria.

Los distractores también se repiten: "SQA no está verificando que se cumplan los requisitos" (eso es VER), "faltan almacenar métricas de las no conformidades escaladas al área de testing" (inventado), "SQA debería participar de los relevamientos" (no es su rol), "las inspecciones sólo las puede hacer Verificación" (falso) y "SQA hace sólo controles a nivel artefacto" (el enunciado dice proceso y producto). Y **no** son problemas que la no conformidad se intente resolver primero en el proyecto, que el líder sea quien la corrige, o que QA se haga entre pares en una organización chica que cumple las condiciones de la sección 21.1.

21.7 Los indicadores que da un checklist

El AD 2025 preguntó qué indicadores se obtienen de un checklist. Se auditaron 30 casos de uso de una organización donde, si no hay caminos alternativos, se escribe `<vacío>`, y el historial de versiones lleva una línea por versión, con número, fecha, responsable y descripción. El checklist preguntaba si se expresó la **meta**, si la sección de camino alternativo **tiene contenido**, y si cada línea del historial tiene **descripción** y **responsable**.

Sale **"10 casos de uso sin meta"**: se cuentan los "No" de la primera pregunta. **No** sale "20 casos de uso sin caminos alternativos", porque un caso de uso sin caminos alternativos **igual tiene contenido** —el `<vacío>`— y la pregunta da "Sí"; un "No" significa plantilla incumplida. Y **no** sale "5 casos de uso sin historial": las preguntas controlan las líneas del historial, presuponen que existe, y ninguna pregunta si existe.

ATENCIÓN Un checklist sólo produce los indicadores que sus preguntas permiten contar. Al diseñar uno, conviene separar "la sección está completa" de "hay caminos alternativos reales", "existe el historial" de "está bien formado", y prever N/A para lo que la plantilla no exige.

De los conteos se derivan porcentajes —10 de 30 es un 33 % de casos de uso sin meta—, que es lo que la guía de medición propone hacer con cualquier checklist: medir el **porcentaje de criterios alcanzados**.

22. La medición y el análisis (MA)

Las decisiones de un proyecto —¿llegamos a la fecha?, ¿el software está listo para entregar?, ¿cuánto esfuerzo reservo para retrabajo?— se toman igual, con datos o sin ellos. Medir sirve para que se tomen sobre **evidencia objetiva** y no sobre la intuición del líder: para **entender** lo que pasa en el desarrollo y el mantenimiento, **controlar** los proyectos, **estimar** y **mejorar** procesos y productos. Pero medir por medir no sirve: "la medición no tiene sentido si no hay un responsable de toma de decisiones con una necesidad de información que lo motive", dice la guía de la cátedra, y CMMI pide que siempre haya respuesta para "**¿por qué estamos midiendo esto?**".

Medición y análisis (MA): desarrollar y sustentar una **capacidad de medición** que se utiliza para dar soporte a las **necesidades de información de la gerencia**.

Es un área **de soporte de nivel 2**, la tercera del grupo básico junto con CM y PPQA; en SPICE, en cambio, la medición está en la categoría de gestión de proyectos. Su foco inicial es el **proyecto** —estimar con datos, seguir el rendimiento real contra el plan, detectar problemas a tiempo—, aunque sus mediciones deberían servir también a la organización.

22.1 Métrica, medida e indicador

En el habla cotidiana "métrica" y "medida" son sinónimos. En la materia no:

TÉRMINO	QUÉ ES	EJEMPLO
Métrica	La forma de medir un atributo —un método o una fórmula— con su escala	Líneas de código
Medida	El valor que se obtiene al medir	40.000 líneas de código
Medición	La acción de obtener ese valor	Contar las líneas
Indicador	Métrica o combinación de métricas con criterios de decisión : da base para decidir	Productividad de los programadores

Las métricas **directas** —**medidas base** en CMMI— se obtienen midiendo un atributo: líneas de código, horas, defectos. Las **indirectas** —**medidas derivadas**— se calculan a partir de dos o más medidas base: densidad de defectos, productividad, valor ganado.

Para decidir qué medir, la guía propone **GQM (Goal-Question-Metric)**: se parte de una **meta**, se formulan **preguntas** que permitan saber si se la alcanza, y recién entonces se eligen las **métricas** que las responden. Para la meta "mejorar la calidad de los entregables", la pregunta "¿cuál es la efectividad de la eliminación de defectos?" lleva a medir los defectos introducidos y detectados en cada fase. El método existe porque las métricas definidas sin un objetivo explícito dan resultados contradictorios y no sobreviven. La **meta** es cualitativa ("reducir el tiempo de entrega"); el **objetivo**, cuantitativo ("reducirlo un 20 % a fin de año").

22.2 Primero alinear, después medir

- **SG 1 Alinear las actividades de medición y análisis.** SP 1.1 Establecer los objetivos de medición · SP 1.2 Especificar las medidas · SP 1.3 Especificar los procedimientos de recogida y de almacenamiento de datos · SP 1.4 Especificar los procedimientos de análisis.
- **SG 2 Proporcionar los resultados de la medición.** SP 2.1 Recoger los datos de la medición · SP 2.2 Analizar los datos de la medición · SP 2.3 Almacenar los datos y los resultados · SP 2.4 Comunicar los resultados.

La primera meta **define todo antes de medir**: para qué (objetivos derivados de las necesidades de información y del negocio), qué (medidas con definiciones tan precisas que dos personas obtengan el mismo resultado), cómo, cuándo, quién y dónde se recogen y guardan los datos, y cómo se analizan. Recién la segunda **produce resultados**. Dentro de SG 1 el orden es flexible —hasta conviene especificar el análisis antes que la recolección—, pero **SG 1 va antes que SG 2**.

El AD 2025 lo preguntó con cuatro afirmaciones de un equipo que va a medir plazos, defectos y productividad. Son coherentes con MA la 1, "las métricas deben estar **alineadas con los objetivos del negocio**" (SP 1.1); la 3, "documentar **cómo y cuándo se tomarán los datos y quién los analizará**" (SP 1.3 y 1.4), y la 4, "el propósito de medir es obtener **evidencia objetiva** para decidir y mejorar". No lo es la 2, "**empezar a recopilar ya, sin definir los procedimientos**": saltea la primera meta, y esos datos no son repetibles ni comparables. Respuesta: **sólo 1, 3 y 4**.

Para ubicar una práctica en un enunciado, y para reconocer cuándo una actividad que usa mediciones **no** es MA:

LO QUE DICE EL ENUNCIADO	ÁREA Y PRÁCTICA
La gerencia decide qué necesita saber y para qué	MA SP 1.1
Se define qué se mide, con qué fórmula y en qué unidad	MA SP 1.2
Se define quién carga los datos, cada cuánto y dónde se guardan	MA SP 1.3
Se define cómo se analizarán: gráfico, umbral, quién analiza	MA SP 1.4
Se cargan los partes de horas o se cuentan defectos, verificando que no falten datos	MA SP 2.1
Se interpretan los datos / se guardan en el repositorio / se envía el informe	MA SP 2.2 / 2.3 / 2.4
Se usan las métricas para comparar plan contra real y corregir el proyecto	PMC: MA provee la medición, PMC la usa
Se estima con datos históricos	PP
Se establece el repositorio de medición de la organización	OPD SP 1.4
QA verifica que las métricas estén completas y correctas antes de ir al repositorio	PPQA
Se analiza estadísticamente la variación del proceso	QPM, de nivel 4

Según un resumen de alumno, Ripani mostró en clase que las áreas se cumplen en el trabajo diario: las tres horas que un programador pasa revisando el caso de uso de un compañero son **VER**, porque es una revisión entre pares, y cargarlas en el parte de horas es **MA SP 2.1**.

ATENCIÓN El uso inapropiado de los datos. Al almacenar (SP 2.3) hay que prevenir que se revele información confidencial, que se interpreten datos incompletos o fuera de contexto, que se **usen las medidas para evaluar a las personas o para clasificar proyectos**, o que se cuestione la integridad de alguien. Una "productividad por desarrollador" para premiar o castigar es señal de problema en un BQ igual que medir sin objetivo (o no medir), recolectar sin procedimiento o estimar durante años con un factor de productividad de mercado que nunca se recalibró.

22.3 Métricas de proceso, de proyecto y de producto

Es lo que más se pregunta del área. La guía clasifica las métricas en tres niveles, y los parciales los preguntan de a dos: el AD 2024 pidió separar proceso de proyecto, y el banco de preguntas, proyecto de producto.

	PROYECTO	PROCESO	PRODUCTO
Carácter	Tácticas	Estratégicas	Describen el software o el servicio
Qué describen	El proyecto y su ejecución	Cómo rinde una actividad del ciclo de vida , para mejorar el desarrollo y el mantenimiento	Características del producto : tamaño, complejidad, calidad, rendimiento
Quién las usa	El jefe de proyecto, para seguir, ajustar y estimar	Los responsables de la mejora; se consolidan entre proyectos	El proyecto, el cliente, el mantenimiento
Ejemplos	Cumplimiento de plazos e hitos, esfuerzo planificado contra real, costo, % de retrabajo	Eficiencia de las revisiones, defectos por fase, efectividad en la eliminación de defectos, tiempo de respuesta del proceso de corrección	Líneas de código, puntos función, complejidad ciclomática, densidad de defectos, cobertura de pruebas, tiempo medio de fallo

La regla para decidir sale de esas definiciones:

1. ¿Compara **lo planificado contra lo real de este proyecto** —plazo, esfuerzo, costo, hitos, tareas a tiempo—? Es de **proyecto**.
2. ¿Mide **cómo funciona una actividad del proceso** —las revisiones, las pruebas, la detección de defectos **por fase**, lo que tarda un paso como aprobar código o corregir un defecto—? Es de **proceso**.
3. ¿Mide un **atributo del software** —tamaño, complejidad, defectos en producción, cobertura—? Es de **producto**.

Con esa regla se resuelve el AD 2024. Las métricas eran: (1) porcentaje de cumplimiento de los plazos de cada fase, (2) número de defectos identificados por cada fase del ciclo de vida, (3) horas planificadas contra reales por iteración, (4) porcentaje de tareas completadas a tiempo en cada iteración y (5) tiempo promedio de aprobación de código en revisiones. La 1, la 3 y la 4 comparan plan contra real: son de **proyecto**. La 2 mide dónde detecta defectos el proceso, y la 5, el rendimiento de una actividad, la revisión: son de **proceso**. Respuesta: **proceso 2 y 5, proyecto 1, 3 y 4**.

ATENCIÓN La palabra "fase" no decide sola: la métrica 1 habla de fases y es de proyecto, porque mide el cumplimiento del plazo. Y los **defectos** pueden caer en cualquier nivel: "por fase" los hace de proceso, pero "defectos en producción" o "densidad de defectos del producto entregado" serían de producto.

En el banco de preguntas aparece el par proyecto–producto: el cumplimiento de hitos de entrega y el costo de horas por iteración son de **proyecto**; la tasa de defectos en producción y la cobertura de pruebas unitarias automatizadas, de **producto**. La propia guía admite zonas grises: la **productividad** es de proyecto cuando se compa-

ra con el objetivo de ese proyecto, y el **tamaño en puntos función** es de proyecto cuando se usa para estimar y de producto cuando describe el software.

22.4 Indicadores, ejemplos y análisis

Un indicador se documenta en una **ficha**, con su objetivo, su **fórmula**, su unidad y, sobre todo, sus **criterios de análisis**: cómo interpretar el valor y qué hacer según el resultado. Las fórmulas que conviene reconocer:

MÉTRICA	FÓRMULA
Desviación del esfuerzo	$(\text{esfuerzo real} - \text{estimado}) \times 100 / \text{estimado}$
Efectividad en la eliminación de defectos	$\text{defectos hallados en pruebas y revisiones} \times 100 / (\text{esos} + \text{los hallados después de la entrega})$
Índice de volatilidad de requisitos	$(\text{añadidos} + \text{eliminados} + \text{modificados}) \times 100 / \text{aprobados inicialmente}$
Valor ganado	$\text{CPI} = \text{EV} / \text{AC} \cdot \text{SPI} = \text{EV} / \text{PV}$
Densidad de defectos entregada	$\text{defectos en las revisiones del cliente y la aceptación} / \text{tamaño}$
Complejidad ciclomática	$\text{aristas} - \text{nodos} + 2 \times \text{partes desconectadas del grafo}$

La efectividad en la eliminación de defectos es, para la guía, una métrica de **proceso**: cuanto más alta, menos defectos llegan a producción. En la plantilla de métricas de la cátedra los defectos se ponderan —fatal $\times 1$, mayor $\times 0,3$, menor $\times 0,1$ —, y las siete **métricas obligatorias** son la satisfacción del cliente, el porcentaje de cumplimiento de la calidad, la eficiencia en la eliminación de defectos y las desviaciones de calendario y de esfuerzo: las totales, contra la última línea base del plan, y las de cada fase. La ficha de complejidad ciclomática pide que ningún módulo supere 10, aunque un ejemplo de la misma guía usa 8.

Las métricas se analizan **apenas se calculan, comparándolas con los objetivos**: "la recolección de datos es un proceso inútil si no se hace nada con ellos". La guía propone las siete herramientas de Ishikawa: el **checklist**, del que sale el porcentaje de criterios alcanzados; el **diagrama de Pareto**, que ordena las causas de defectos de mayor a menor frecuencia; el **histograma**, que muestra frecuencias en intervalos ordenados (con categorías sin orden, es un gráfico de barras); el **diagrama de dispersión**, entre dos variables; el **gráfico de ejecución**, un parámetro a lo largo del tiempo; el **gráfico de control**, que le agrega límites para ver si el proceso está controlado, y el **diagrama causa-efecto**, la espina de pescado de las causas raíz. Para comunicar, el **dashboard** muestra el **rendimiento** en tiempo real, sin objetivos, y el **scorecard**, el **progreso hacia los objetivos**, con umbrales: el primero dice qué se hace; el segundo, qué tan bien.

22.5 Dónde se guardan los datos

Los datos de medición pueden quedar en un **repositorio del proyecto** o, si se comparten, en el **repositorio de medición de la organización**. Ese segundo repositorio **no lo establece MA sino OPD** (SP 1.4), porque es un activo de la organización (capítulo 7). MA, de nivel 2, guarda los datos que produce (SP 2.3); OPD, de nivel 3, crea el repositorio común donde los proyectos vuelcan las medidas compartidas y del que sacan datos históricos para estimar. La guía recomienda definir primero un **conjunto común** de medidas, que cada proyecto completa con las suyas, **empezar con pocas métricas y automatizar la recolección**, porque establecer las medidas lleva de 6 a 9 meses.

ATENCIÓN Un resumen de alumno sostiene que MA "cambia sus prácticas" entre el nivel 2 y el 3. Las prácticas de MA son **las mismas** en cualquier nivel; lo que aparece en el nivel 3 es el repositorio organizacional, que es de OPD.

23. Las pericias informáticas

La informática aparece hoy en casi cualquier conflicto judicial: es **herramienta o vehículo** del delito —estafas, amenazas anónimas, robo de claves o de datos— y también su **objeto**, y por eso atraviesa todos los fueros. Plantea además un problema que otras pruebas no tienen en la misma medida: la evidencia digital **se altera con facilidad**. Encender un equipo apagado puede modificar fechas, y ejecutar cualquier programa puede cambiar las fechas de acceso de los archivos. La disciplina existe para que la prueba llegue al juez **indubitable**: desde el secuestro hasta el análisis pericial, el procedimiento no puede dejar dudas. En el fuero penal la prueba suele surgir de un **secuestro**; en los fueros civil, laboral y comercial también se usan herramientas forenses para **pre-constituir prueba** antes del pleito.

23.1 Forensia, pericia y perito

Informática forense: la ciencia de adquirir, preservar, obtener y presentar datos que han sido procesados electrónicamente y guardados en un medio computacional (definición del FBI).

La guía de INCIBE habla de **análisis forense digital**: los procedimientos de recopilación y análisis de evidencias para responder a un incidente de seguridad, que a veces deben servir como prueba ante un tribunal y que responden qué, dónde, cuándo, por qué, quién y cómo. La **pericia informática**, en cambio, es el **acto procesal**: se pide cuando, para descubrir o valorar una evidencia, hacen falta **conocimientos especiales** en informática forense. La forensia es la herramienta; la pericia, el acto en que se usa.

El **perito** trabaja **en el laboratorio** y como **asesor científico** del operador judicial, que dirige la investigación; el allanamiento y el secuestro **no son tareas suyas**, sino de la **policía con el fiscal**. Puede evacuar consultas previas para definir el alcance de los **puntos de pericia**, que conviene aclarar siempre: si el juez pide "buscar evidencia" sin palabras clave —un número de cuenta, una IP, un correo—, el resultado queda librado al criterio del perito. Si hay **peritos de parte**, la tarea se consensúa con ellos. Y como cambian el escenario, las pruebas y las herramientas, **nunca hay dos pericias iguales**, aunque el delito sea el mismo.

23.2 El principio de Locard y las fases

Principio de intercambio de Locard: siempre que dos objetos entran en contacto, transfieren parte del material que incorporan al otro.

Todo delito deja rastro, y **el propio análisis también lo deja**: de ahí la regla que ordena la unidad, **afectar el sistema lo menos posible**. El procedimiento tiene que ser **verificable** (se puede comprobar la veracidad de las conclusiones), **reproducible** (las pruebas se pueden repetir en cualquier momento), **documentado** e **independiente** (llega a las mismas conclusiones sin importar quién lo haga ni con qué metodología). Recorre cinco fases:

FASE	QUÉ SE HACE
Preservación	Que no se pierdan evidencias: no apagar , rotular, registrar cada operación, transportar lejos del calor y de campos electromagnéticos
Adquisición	Recolectar las evidencias, con sus hashes
Análisis	Examinarlas según el tipo de incidente, sin descartar lo "obvio"
Documentación	Fotografías, cadena de custodia , bitácora con fecha y hora, y dos informes: uno ejecutivo y uno técnico
Presentación	Conclusiones pedagógicas y objetivas, sin afirmaciones no demostrables ni juicios de valor

ATENCIÓN Las fases **no son estrictamente secuenciales**: están entrelazadas. La documentación empieza en la preservación, no cuando termina el análisis.

23.3 La evidencia y el orden de volatilidad

Evidencia: cualquier prueba que pueda usarse en un proceso legal.

Se distingue la **evidencia física** —el soporte: un disco, un pendrive— de la **digital** —la información: un archivo, un proceso en ejecución, un log—. Para servir tiene que ser **admisible** (cumple la legislación), **auténtica** (sin manipulación; para eso, los hashes), **completa** (sin visiones parciales), **confiable** (sin dudas sobre cómo se obtuvo) y **creíble** (comprensible para un tribunal). INCIBE define dos veces la autenticidad —"sin manipulación" y "corresponde al incidente"—, pero los cinco nombres coinciden.

La otra distinción es la **volatilidad**. La memoria, las conexiones de red, los procesos y las contraseñas en uso **se pierden al apagar**; el disco y los logs sobreviven. Por eso la adquisición puede ser **en vivo** (*live*), con el sistema funcionando, o **estática** (*dead*), con el sistema apagado, y en vivo lo primero que se toma es la **fecha y hora del sistema**, comparada con la hora UTC. INCIBE adopta el **RFC 3227**, que manda recolectar **de mayor a menor volatilidad**:

1. Registros y contenido de la caché
2. Tabla de enrutamiento, caché ARP, tabla de procesos, estadísticas del kernel, memoria
3. Información temporal del sistema
4. Disco
5. Logs del sistema
6. Configuración física y topología de la red
7. Documentos

ATENCIÓN La **memoria va antes que el disco**: tomar primero la imagen del disco, o apagar el equipo para trabajar tranquilo, contradice el orden.

El RFC pide además notas con **fecha y hora**, aclarando si es hora local o UTC, **minimizar los cambios** y, ante un dilema entre recolectar y analizar, **recolectar primero**. Y prohíbe tres cosas: **apagar** antes de tener lo volátil, **confiar en los programas del sistema investigado** —se usan herramientas propias, desde un medio de sólo lectura— y ejecutar algo que cambie las fechas de acceso de los archivos. Ante la duda sobre qué es relevante, mejor de más que de menos.

23.4 La adquisición: imagen forense, hash y bloqueo de escritura

La regla es **no trabajar nunca sobre el original**.

Imagen forense: copia **bit a bit** de la evidencia, que se hace **sólo para analizar sobre ella** y que conserva toda la información, incluida la oculta o remanente.

	IMAGEN FORENSE	BACKUP
Qué es	Copia bit a bit de la evidencia	Copia simple de archivos
Para qué	Analizar sin tocar el original; evita la contaminación de la prueba	Resguardar los datos del dueño, como medida de seguridad
Efecto sobre la evidencia	No la altera	Invasivo: la altera
Información oculta o remanente	La conserva	No la conserva
Quién la hace	El perito	El área de sistemas o el propietario; fuera del alcance pericial

ATENCIÓN El **perito no hace backups**: el protocolo los excluye de las tareas periciales. Si un oficio pide "un backup del disco para analizarlo", lo que corresponde es una imagen forense.

Lo habitual es copiar **el disco a un archivo de imagen**, que es lo más rápido y permite sacar tantas copias como haga falta; si no se puede, se copia **de disco a disco**, y si no hace falta todo, se hace una **copia selectiva**. Siempre se analiza la copia. Para probar que la imagen **no se modificó después**, se calcula su **hash** y se **anota en la cadena de custodia**: si al recalcularlo coincide, la imagen es la misma.

ATENCIÓN **MD5 tiene colisiones** —dos archivos distintos pueden dar el mismo valor— y eso permite cuestionar la prueba; SHA-1 tiene un problema parecido. Conviene **SHA-256 o SHA-512**.

Al copiar se usa un **bloqueador de escritura** (*write blocker*), **por hardware o por software**, que impide escribir sobre el original. Su límite se pregunta: en un **disco de estado sólido con TRIM**, el disco borra las celdas liberadas **con sólo tener corriente**, y ni el bloqueador ni cambiarlo de equipo lo evitan; el hash puede dar distinto aunque nadie lo haya tocado. De las herramientas forenses, la unidad deja dos ideas: **gratis no significa peor**, y ninguna es mejor en términos absolutos, así que conviene **combinarlas**.

23.5 La cadena de custodia

Entre el secuestro y el perito pasan **mucho tiempo y muchas manos**: allanamiento, comisaría, juzgado, laboratorio. Sin registro no se pueden atribuir responsabilidades por un faltante ni probar que la evidencia no se manipuló.

Cadena de custodia: registro documentado de **dónde, cuándo y quién** descubrió, recolectó, manejó y custodió la evidencia, y de **cada cambio de manos**.

Empieza en el **primer contacto** con la evidencia —normalmente el secuestro—, no cuando llega al perito, y uno de los textos de la unidad la llama la **historia clínica** de la evidencia, que no es una "simple formalidad". En el protocolo judicial, la integridad física se asegura con **precintos o etiquetas de seguridad** que la policía

coloca **desde el secuestro** en cada entrada eléctrica y en todo lo que pueda abrirse; sus números de serie van en el **acta de allanamiento** y en el **oficio**. Al recibir el material se cotejan, y al terminar la pericia se ponen etiquetas nuevas, detalladas en el dictamen. El protocolo no menciona el hash, que aparece en la guía de INCIBE.

ATENCIÓN Si un precinto llega **roto** y el expediente no documenta una intervención forense hecha por profesionales calificados, **no se puede asegurar la integridad y se pierde ese medio de prueba**. Se deja constancia; la alteración previa a la pericia es responsabilidad **policial**.

23.6 El protocolo de actuación y el allanamiento

El protocolo de actuación para pericias informáticas del Poder Judicial de Neuquén busca **evitar la contaminación de la prueba**, **formalizar** el procedimiento pericial y **definir el alcance** de los servicios forenses. Divide el procedimiento penal en dos etapas:

ETAPA	QUIÉN	DÓNDE
Incautación confiable y cadena de custodia	La policía , con el fiscal , que planifica el procedimiento	En el lugar del hecho
Análisis e informe pericial	El perito	En el laboratorio

Por regla se prefiere el secuestro a peritar en el lugar, que es **excepcional**; otro texto de la unidad admite la forensia **in situ** con herramientas portables cuando hay más de cincuenta equipos. El material llega **siempre con el oficio que fija los puntos de pericia**; quedan **excluidas** las tareas ajenas a la disciplina —transcribir, cruzar datos, imprimir, escuchar, filmar y hacer **backups**—; se priorizan las causas con **personas detenidas**, después los delitos graves con autores ignorados, y luego el orden de ingreso; y el material se **conserva hasta el fin del proceso**, para poder repetir o ampliar la pericia.

En el allanamiento, la guía operativa pide **separar a las personas** de los equipos, **fotografiar todo antes de mover o desconectar**, que **nadie busque en directorios ni haga copias** sin software forense, embalar en **bolsas antiestáticas** —nunca plásticas—, precintar y evitar el secuestro masivo de discos y pendrives. Lo que más se confunde es qué hacer con un **equipo encendido**, porque las dos fuentes de la unidad no dicen lo mismo:

FUENTE Y CONTEXTO	EQUIPO ENCENDIDO	EQUIPO APAGADO
INCIBE : un técnico responde a un incidente	No apagarlo hasta haber recolectado toda la información volátil	No encenderlo : puede modificar fechas o, si hay un rootkit, ocultar archivos
Protocolo de Neuquén : allanamiento policial	Queda prendido y se consulta a un especialista cómo apagarlo; sin asesoramiento, se desenchufa del lado del gabinete	Queda apagado y se desconecta de su toma en el equipo, no de la pared . En notebooks se quitan las baterías

No es una contradicción estricta: INCIBE le habla a un técnico que puede volcar la memoria, y el protocolo, a un policía que no debe tocar nada. Pero el resultado difiere —desenchufar pierde la memoria—, así que hay que identificar **qué fuente** cita la pregunta. En el protocolo el cable se desconecta siempre **del lado del equipo**: "desenchufarlo de la pared" es la trampa.

24. Cómo reconocer el área de proceso en un enunciado

El ejercicio más repetido de la materia es una variación de la misma consigna: "*relacione la actividad descripta con el área de proceso y la práctica específica que corresponda, o indique NINGUNA*". Parece de memoria y no lo es. Casi cualquier actividad se puede forzar en dos o tres áreas, y lo que decide es leer bien **quién hace qué, sobre**

qué objeto, contra qué lo compara y **en qué momento** ocurre. Este capítulo junta los criterios de los capítulos anteriores en un solo lugar, para usarlo durante el examen.

La justificación de fondo está casi siempre en una **subpráctica** de CMMI. Las subprácticas no se toman como tales, pero son las que convierten una corazonada en una respuesta defendible: las asignaciones de este capítulo salen de resoluciones de finales verificadas contra el texto del modelo.

24.1 El método en cinco preguntas

1. **¿Es de un proyecto o de toda la organización?** Si el enunciado dice "en el proyecto X", la respuesta está entre PP, PMC, REQM, RD, CM, VER, VAL, RSKM y PPQA, que evalúa *dentro* de los proyectos. Si dice "en la software factory" o "para todos los proyectos", lo más probable es OPD, OPE, OT, o MA cuando define cómo se mide en toda la organización.
2. **¿Cuál es el objeto?** Un requerimiento, un elemento de configuración, un plan, un producto de trabajo, un activo de proceso, la formación de la gente. El objeto descarta áreas enteras: un activo nunca lo controla VER, y una plantilla de la organización no es asunto de la gestión de configuración del proyecto.
3. **¿Contra qué se compara y quién participa?** Contra la especificación o los artefactos del propio proyecto, por el equipo o entre pares → **VER**. Con el cliente, contra lo que necesita → **VAL**. Contra un estándar de la organización, por alguien externo al proyecto → **PPQA**. Contra una convención del propio proyecto → **CM**. Contra el plan → **PMC**.
4. **¿En qué momento está el relato?** El tiempo verbal elige la práctica dentro del área (preparar, realizar, analizar) y separa áreas vecinas: PP de PMC, REQM de CM, la SG 2 de la SG 3 de OPF (24.2).
5. **¿La opción tiene la sigla correcta?** Con el área y la práctica decididas, lee la opción entera. Una práctica real con la sigla de otra área es falsa, y si ninguna opción encaja, la respuesta es **NINGUNA**.

La primera pregunta tiene dos excepciones que conviene recordar. El seguimiento de un proceso nuevo en un **proyecto piloto** es OPE, aunque ocurra dentro de un proyecto. Y al revés, decidir qué datos se registran **en el proyecto Y** es PP SP 2.3, no MA, aunque hable de mediciones.

Un ejemplo de punta a punta. *"Ayer, dos analistas controlaron los casos de uso de otro analista contra las minutas y las reglas de negocio aprobadas; los hallazgos ya se documentaron y se enviaron al autor; en este momento se almacenan las identidades de los participantes en una parte del repositorio a la que sólo accede Calidad."* Es un proyecto. El objeto es un producto de trabajo, los casos de uso. Se los compara contra artefactos del propio proyecto, entre pares y sin el cliente: **VER**, revisión entre pares. Y el momento es posterior a la revisión: no se prepara ni se lleva a cabo, se **analizan y protegen los datos** de la revisión, que es **VER SP 2.3**. La mención de Calidad es un distractor: no hay ningún estándar de la organización en juego.

24.2 El tiempo decide la práctica

Muchos ejercicios cuentan la misma actividad en momentos distintos y cambian la respuesta. Estas son las marcas temporales que más se repiten:

LO QUE DICE EL ENUNCIADO	LO QUE CORRESPONDE
"Se distribuirá", "será chequeada", "se determinará que se controlará" (futuro)	Preparar: SP 1.x de VER o de VAL, o VER SP 2.1 si es una revisión entre pares
"Está chequeando", "se realiza un control" (presente)	Realizar o llevar a cabo
"Ya se revisó y se documentó; ahora se almacena..."	Analizar los datos: VER SP 2.3
"Previo al inicio del desarrollo"; reunión de lanzamiento futura	PP
"En la última reunión de avance", "respecto a lo planificado", "durante el proyecto"	PMC

Hay además tres cadenas en las que el momento elige la práctica (la de OPF, propuesta frente a modificación ya aceptada, está en el capítulo 7).

La cadena de PMC SG 2. "Analizando la situación se concluyó que la causa es..." es **SP 2.1 Analizar los problemas**. "En este momento se determinó reemplazar al usuario clave" es **SP 2.2 Llevar a cabo las acciones correctivas**. "El reemplazo se hizo hace dos semanas y ahora se evalúa si las dudas se subsanaron" es **SP 2.3 Gestionar las acciones correctivas**, cuya subpráctica es analizar la eficacia de la acción.

La cadena de PPQA. Si ayer se controló que los casos de uso respetan las directrices de la organización (SP 1.2), los analistas ya fueron informados (SP 2.1) y **en este momento** se arma el informe con los casos de uso con problemas, los responsables y las fechas comprometidas, la respuesta es **SP 2.2 Establecer registros**.

REQM o CM: se evalúa o ya se aceptó. El análisis de impacto de un cambio aparece en las dos áreas. Si el cambio de requisito **todavía se evalúa** —se consulta al contador cómo afecta al esquema impositivo—, es **REQM SP 1.3**. Si **ya se decidió aceptarlo** y se tramitan las modificaciones a los casos de uso, es **CM SP 2.1** Seguir las peticiones de cambio.

24.3 Los pares que se confunden

PAR	CÓMO SE CORTA
PP / PMC	Por el momento: reunión de lanzamiento futura o "previo al inicio" → PP; reunión de avance ya ocurrida o "respecto a lo planificado" → PMC. Todo cambio durante el proyecto es seguimiento
PPQA / VER	PPQA controla que el producto respete el estándar de la organización; VER, que su contenido sea correcto contra lo especificado
PPQA / VAL	Un checklist del estándar aplicado antes de entregar al cliente, sin el cliente, es PPQA SP 1.2; con el cliente y contra su necesidad, es VAL
PPQA / CM	Estándar de la organización controlado por alguien externo al proyecto → PPQA; convención del propio proyecto (nombres de archivo, líneas base, versiones) → CM
PPQA / PMC	Comparar cómo se ejecutaron las tareas contra los criterios de OPD → PPQA SP 1.1; contra el plan → PMC
PPQA / OPF	PPQA evalúa e informa no conformidades y tendencias; OPF analiza esos informes y propone la mejora

PAR	CÓMO SE CORTA
OPF / OPD	OPF propone, pilotea y despliega ; OPD crea y deja asentado el activo. Cadena: PPQA emite la no conformidad → OPF propone → OPD actualiza el activo
OPF SG 2 / SG 3	Mejora propuesta , a probar en pilotos → SG 2; mejora ya aceptada , a desplegar → SG 3
OT / PP SP 2.5	Capacidades para la organización → OT; cursos para la gente de un proyecto → PP SP 2.5
PP SP 1.2 / SP 1.4	Tamaño (puntos función) → SP 1.2; esfuerzo, horas o costo derivados del tamaño → SP 1.4
PP SP 2.3 / MA	Qué datos se registran en un proyecto → PP SP 2.3; cómo y cuándo se mide en toda la organización → MA SP 1.3
MA / OPD SP 1.4	MA especifica las medidas y los procedimientos de recogida; OPD SP 1.4 define la estructura del repositorio de la organización
MA / PMC	MA produce la medición; usarla para comparar el plan contra lo real y corregir el proyecto es PMC
REQM / RD	RD obtiene, desarrolla y analiza requisitos que todavía no están acordados; REQM gestiona los ya acordados: compromiso, cambios, trazabilidad, inconsistencias
REQM / CM	¿El cambio altera un requisito ? → REQM, aunque lo proponga el equipo. ¿Es un cambio técnico interno que no toca requisitos? → CM. Y un cambio de requisito ya aceptado, que se tramita como petición de cambio → CM SP 2.1
RD SP 3.5 / VAL	Confirmar con el usuario los requisitos , antes de construir → RD SP 3.5; el producto o sus componentes con el cliente → VAL
VER / VAL	Contra qué se contrasta: la especificación → VER; el uso previsto , con el cliente → VAL. Prueba de sistema → VER; prueba de aceptación → VAL
VER / OPD	Pares que revisan un producto de un proyecto → VER; pares que revisan un activo de la organización aún no publicado → OPD
PP SP 2.2 / PMC SP 1.3 / RSKM	Identificar y acordar riesgos para el plan → PP SP 2.2; seguirlos y comunicar su estado → PMC SP 1.3; fuentes, categorías, parámetros, estrategia, planes de mitigación → RSKM
Sector SQA / PPQA	El sector de SQA de una empresa hace PPQA y también OPF; el área la define la actividad, no el departamento

ATENCIÓN Una guía de resolución de alumnos resume el par REQM / CM como "REQM trata sólo los cambios que pide el cliente; los del equipo van a CM". Sirve como regla rápida y coincide con la mayoría de los enunciados —controlar que se cumplan los pedidos de modificación **hechos por integrantes del grupo de desarrollo** es CM SP 2.1—, pero lo que decide de verdad es si el cambio toca un requisito y en qué momento está (capítulo 15).

24.4 De la palabra del enunciado al área

La tabla junta las expresiones que se repiten en los finales y en la práctica del AD, con el área y la práctica a las que llevan. No reemplaza al método: sirve para confirmar una respuesta o para destrabarse cuando dos áreas parecen posibles.

LO QUE APARECE EN EL ENUNCIADO	ÁREA / PRÁCTICA
Una plantilla o un procedimiento para toda la SF (por ejemplo, la plantilla para calcular puntos función)	OPD SP 1.1

LO QUE APARECE EN EL ENUNCIADO	ÁREA / PRÁCTICA
Reunión entre pares para revisar un procedimiento aún no publicado , redactado por otro	OPD SP 1.1
Las fases por las que pasan los sistemas " desde que nace la idea hasta que el software es retirado "; aprobar los modelos de ciclo de vida	OPD SP 1.2
" En base a qué características " se elige un ciclo de vida o una plantilla; plantilla breve o completa; procedimiento para pedir una excepción	OPD SP 1.3
La estructura de la base de datos de métricas de todos los proyectos	OPD SP 1.4
El sitio de Intranet con los documentos del proceso; eliminar los links de plantillas fuera de vigencia	OPD SP 1.5
Herramienta, ambiente o software estándar para todas las PC de la SF	OPD SP 1.6
Objetivos de performance de los procesos (por ejemplo, la tasa de eliminación de defectos)	OPF SP 1.1
Conseguir el apoyo del Presidente para evaluar; comparar los procesos con CMMI; puntos fuertes y débiles	OPF SP 1.2
Analizar tendencias en las lecciones aprendidas o en los informes de QA para detectar mejoras	OPF SP 1.3
Qué proyectos piloto hacen falta para verificar la efectividad de una propuesta	OPF SP 2.2
Atender las consultas por la entrada en vigencia de una plantilla nueva; decidir cómo disponibilizarla	OPF SP 3.1
Qué proyectos en curso deben incorporar la reciente modificación	OPF SP 3.2
Incorporar lecciones aprendidas y propuestas de mejora a los activos	OPF SP 3.4
Las capacidades que debe tener quien ejerce un rol en la SF	OT SP 1.1
Acordar con el PM cómo cubrir una necesidad de formación	OT SP 1.2
Conseguir un instructor ; formar a alguien para que después capacite a otros	OT SP 1.4
Definir qué se mide , con qué fórmula y en qué unidad	MA SP 1.2
Guía para todos los proyectos de cómo y cuándo registrar las horas; qué cuenta como retrabajo	MA SP 1.3
Calcular los puntos función / calcular las horas o el costo a partir del tamaño	PP SP 1.2 / PP SP 1.4
Establecer las fases del proyecto X	PP SP 1.3
Previo al inicio , revisar con el cliente el impacto o la probabilidad de los riesgos documentados	PP SP 2.2
Qué datos se registran en el proyecto (autor, aprobador, tipo de documento)	PP SP 2.3
Cantidad de personas y dedicación del equipo	PP SP 2.4
Cursos obligatorios para los integrantes del proyecto X	PP SP 2.5
Tabla de roles y responsabilidades con la dedicación de los stakeholders por fase	PP SP 2.6
Ajustar fechas en el documento que se presentará en la reunión de lanzamiento futura	PP (SP 2.4 o SP 3.2; ver 24.6)
Asistencia de los usuarios clave a las reuniones; si responden los mails	PMC SP 1.5

LO QUE APARECE EN EL ENUNCIADO	ÁREA / PRÁCTICA
Informar el aumento de probabilidad o de prioridad de un riesgo respecto de lo planificado	PMC SP 1.3
Inconvenientes, adelantos y demoras de las últimas x semanas / controles al final de cada fase	PMC SP 1.6 / PMC SP 1.7
" Toda solicitud que no cumpla será devuelta "; requisito indispensable para aprobar un caso de uso	REQM SP 1.1
Acordar con el Sponsor que la fecha se corre por un cambio	REQM SP 1.2
Cambio de requisito cuyo impacto todavía se evalúa ; documentar el cambio y su razón	REQM SP 1.3
Matriz de trazabilidad ; tabla CRUD entre casos de uso y tablas	REQM SP 1.4
Qué artefactos deben modificarse por un cambio en la línea base de requerimientos	REQM SP 1.5
Relevar y entrevistar; un requisito vago que hay que volver medible; confirmarlo con el cliente antes de construir	RD (SP 1.1 · SP 3.3 · SP 3.5)
Códigos de los casos de uso; responsable de cada artefacto	CM SP 1.1
Procedimiento de back up ; carpetas y permisos del repositorio	CM SP 1.2
Artefactos revisados y acordados formalmente como " basamento " del desarrollo	CM SP 1.3
Seguir hasta su cierre los pedidos de modificación del equipo ; un cambio ya aceptado en trámite	CM SP 2.1
Documento con las diferencias entre una compilación y la anterior	CM SP 3.1
Lo construido coincide con su documentación; los nombres de archivo respetan la convención del proyecto	CM SP 3.2 (para los nombres, ver 24.6)
Personal externo al proyecto controla un artefacto contra las directrices de la SF; checklist del estándar antes de entregar al cliente	PPQA SP 1.2
Evaluar cómo se ejecutaron las tareas contra los criterios de OPD	PPQA SP 1.1
Elevar un informe al Director tras observaciones reiteradas; comunicar tendencias de calidad	PPQA SP 2.1
Qué clases entran en la prueba de integración ; definir que algo se controlará por inspección	VER SP 1.1
Cargar la base de datos de pruebas antes de probar	VER SP 1.2
Distribuir entre analistas un caso de uso que será chequeado en una reunión; definir que será un walkthrough	VER SP 2.1
Revisión entre pares ya hecha; ahora se protegen los datos	VER SP 2.3
Qué casos de uso entran en la prueba de aceptación o en el workshop con el cliente	VAL SP 1.1
El servidor del cliente o la agenda para la prueba de aceptación	VAL SP 1.2
Los criterios con los que el cliente da por cumplido el contrato	VAL SP 1.3 (ver 24.6)
Fuentes, categorías, umbrales , estrategia, planes de mitigación o contingencia	RSKM

24.5 Trampas que ya se tomaron

Leer la sigla, no sólo el texto. Si la respuesta es VER y la opción dice "VAL / SP 2.1 Preparar las revisiones entre pares", la respuesta es **NINGUNA**: las revisiones entre pares existen sólo en VER, y una práctica correcta con la sigla equivocada es una opción falsa.

El área la define la actividad, no el tema. Elevar al Gerente General un informe porque se cambian requerimientos sin seguir el circuito de REQM es **PPQA SP 2.1**: lo que se hace es escalar una no conformidad. Del mismo modo, un enunciado que menciona la línea base de requerimientos no es automáticamente REQM: si lo que se hace es avisar que aumentó la probabilidad de un problema, es riesgo (PMC SP 1.3). Y que el sector de SQA comunique por mail los cambios de plantillas es un despliegue, OPF SP 3.1, no PPQA.

Nombrar a Calidad no lo vuelve PPQA. En el ejemplo de 24.1, guardar los datos de una revisión entre pares donde sólo accede Calidad sigue siendo VER. PPQA aparece cuando se controla la **adherencia a un estándar o a un proceso** de la organización.

Un checklist antes de entregar no es validación. Aplicar el checklist definido según el estándar al modelo de casos de uso antes de entregarlo al cliente es **PPQA SP 1.2**: el checklist mide adherencia al estándar y el cliente no participa. Alguna resolución de alumnos lo daba como VAL, y es incorrecto.

Las pruebas no se reparten por fase. La prueba de aceptación es VAL y la de sistema es VER, aunque se haga sobre el producto terminado; las unitarias, de integración y de stress que hace la SF son VER. La inspección es una técnica estática vinculada a VER, pero no es exclusiva de VER: "las inspecciones sólo las puede hacer Verificación" es falso, porque PPQA también inspecciona.

Un riesgo sigue siendo riesgo. "Aún es posible resolverlo" no convierte un riesgo en un problema ocurrido: informar que aumentó su probabilidad es PMC SP 1.3, no SP 2.1. Y RSKM casi nunca es la respuesta: en los finales recopilados no aparece ninguna vez, y en la práctica oficial del AD ni siquiera figuraba entre las opciones. Sólo hay que elegirla cuando el enunciado habla de fuentes, categorías, parámetros, umbrales, estrategia o planes de mitigación.

24.6 Cuando las resoluciones no coinciden

En algunos enunciados las resoluciones de alumnos dan prácticas distintas. En casi todos el **área** no se discute; lo que cambia es la SP. La estrategia es mirar cuál de las candidatas ofrecen las opciones y, si aparecen dos, elegir la más defendible.

ENUNCIADO	CANDIDATAS	LO MÁS DEFENDIBLE
Calcular si un cambio de caso de uso pedido por el cliente afecta la fecha de entrega	REQM SP 1.2 · REQM SP 1.3 · CM SP 2.1	REQM SP 1.2 (impacto sobre los compromisos); según una resolución, el profesor aceptaba también las otras dos
Estimar con el arquitecto cuántas clases habría que cambiar por una política nueva del cliente	REQM SP 1.3 · CM SP 2.1 · REQM SP 1.2	REQM SP 1.3, porque es un cambio del cliente cuyo impacto todavía se evalúa
Definir que un artefacto se controlará por walkthrough o por inspección	VER SP 2.1 · VER SP 1.1	Walkthrough → SP 2.1 (tipo de revisión entre pares); inspección → SP 1.1 (método de verificación)
Acordar con el cliente con qué criterios se dará por cumplido el contrato	VAL SP 1.3 · REQM SP 1.1 · PP SP 2.6	VAL SP 1.3: son criterios de aceptación del cliente
Revisar los informes de no conformidad de SQA para cambiar una plantilla o un checklist	OPF SP 1.3 · OPF SP 3.4	Las dos aparecen en las resoluciones; el área es OPF

ENUNCIADO	CANDIDATAS	LO MÁS DEFENDIBLE
"De ahora en más, todos los procedimientos se documentan también con un diagrama de actividad"	OPF SP 1.3 · OPD SP 1.1	Dudoso; las resoluciones se dividen
Acordar con el PM si el curso que propone un especialista es adecuado	OT SP 1.2 · OT SP 1.4	SP 1.2, porque lo que ocurre en ese momento es acordar; si sólo ofrecen SP 1.4, esa
Ajustar fechas en el documento para la reunión de lanzamiento futura	PP SP 2.4 · PP SP 3.2	El área es PP, por la reunión futura
Controlar los nombres de archivo contra la convención del proyecto	CM SP 2.2 · CM SP 3.2	El área es CM; no es PPQA porque la convención es del proyecto

25. Las preguntas BP

En los AD de 2024 y 2025, el formato que más pesó no pide una definición: describe cómo trabaja una software factory —una política, un procedimiento, una plantilla— y pide decidir si esa práctica **puede generar problemas de calidad y por qué**. La consigna arranca con "BP -->", y de ahí el nombre. Lo que se evalúa es si se entiende **para qué sirve** cada práctica de CMMI: se muestra una práctica a la que le falta algo, y hay que darse cuenta de qué falta y qué consecuencia tiene.

25.1 Tres formatos, un mismo razonamiento

FORMATO	DÓNDE APARECIÓ	QUÉ SE PIDE	QUÉ SE CORRIGE
Opción múltiple	Parciales AD 2024 y 2025	Elegir una o varias afirmaciones del tipo "No genera problemas... porque..." o "Sí podría generar problemas... porque...", o NINGUNA	La razón , no sólo el sí o el no
Práctica ausente	Finales 2012-2015	Qué problemas tendría una SF que no aplicara cierta práctica, y en qué nivel quedaría	Una cadena causa → efecto razonada, "no una mera copia del CMMI"
Hallazgos	Finales 2013-2015	Situaciones relevadas por un consultor: qué problemas generan y qué se recomienda, con su área	Recomendaciones concretas : "no alcanza con mencionar el nombre de una práctica"

Los tres se resuelven con el mismo razonamiento: **qué área de CMMI se está implementando, qué le falta y qué consecuencia tiene esa falta**. El AD usa el primero, pero los otros dos sirven para practicar, porque entrenan justamente la cadena que permite juzgar las razones de las opciones.

25.2 El método

1. **Leer la práctica como un proceso**: quién hace qué, contra qué criterio, cuándo y, sobre todo, **qué pasa al final**. ¿Alguien cierra el circuito? ¿Qué pasa si no se cumple? El problema casi siempre está en la última oración: el escalamiento, el archivo, el registro, quién aprueba.
2. **Identificar el área de proceso** con los criterios del capítulo 24: quién participa, contra qué se compara, si es de un proyecto o de la organización.
3. **Contrastar con las prácticas de esa área** y buscar el **eslabón que falta** (25.4). Si falta uno, **sí** hay problema. "No genera problemas" es correcto sólo si la práctica cumple lo que pide el área y lo que agrega el enunciado es un detalle sin consecuencias.

4. **Evaluar cada razón por separado.** Un "Sí... porque X" es correcto sólo si X es **cierto**, **ataca el eslabón que falta** y pertenece **al área en juego**.
5. **Aplicar la regla de no mezclar** (25.3) y recién ahí marcar.

25.3 La regla de no mezclar

La consigna lo dice textualmente: pueden elegirse varias opciones, pero **no pueden combinarse** las que consideran que sí hay problemas de calidad con las que consideran que no. Eso tiene dos consecuencias. Si se marcan varias, tienen que ser todas del mismo lado y todas con razones verdaderas. Y si hay un problema pero ninguna de las razones ofrecidas lo describe, la respuesta es **NINGUNA**, no la menos mala de las opciones.

ATENCIÓN Un "Sí... porque X" con un X falso es tan incorrecto como un "No". Buena parte de los distractores dicen "Sí" y fallan en la razón.

25.4 Señales de problema por área

La tabla resume qué buscar en el enunciado según el área que se está implementando, y qué práctica es la que falta en cada caso.

ÁREA	SEÑALES DE PROBLEMA EN LA PRÁCTICA DESCRIPTA	LO QUE FALTA
PPQA	Evaluator dentro del equipo o bajo la misma gerencia que decide sobre el proyecto · no conformidades que nadie sigue hasta el cierre · sin escalamiento, o escalamiento a quien es juez y parte · la no conformidad se archiva o se sanciona en vez de resolverse · controla sólo producto o sólo proceso · sin criterios ni registros	Objetividad e independencia (SP 1.1, 1.2) · comunicar, escalar y seguir hasta la resolución (SP 2.1) · registros (SP 2.2)
CM	Artefactos versionados sin relación entre sí · sólo se guarda la última versión · documentos en carpetas personales · nombres sin convención · cambios aceptados sin petición ni autorización · peticiones cerradas sin actualizar la documentación · pase a producción verbal · no se sabe qué versión tiene el cliente	Identificar los elementos (SP 1.1) · sistema de CM (SP 1.2) · líneas base (SP 1.3) · seguir y controlar los cambios (SP 2.1, 2.2) · registros y auditorías (SP 3.1, 3.2)
PP	Estimar sólo por experiencia o con factores de mercado · tareas agregadas, sin EDT · propuesta armada sin técnicos · no se distinguen los entregables · no se planifican la involucración y los roles del cliente · perfiles sin definir · riesgos sin identificar	EDT (SP 1.1) · estimaciones con datos históricos (SP 1.2, 1.4) · riesgos (SP 2.2) · datos (SP 2.3) · habilidades (SP 2.5) · involucración (SP 2.6) · compromiso (SP 3.3)
PMC	Reuniones de avance sin el cliente · compromisos o riesgos que no se siguen · desvíos que no se analizan · sin revisiones de hito	Monitorizar contra el plan (SP 1.1 a 1.7) · acciones correctivas hasta el cierre (SP 2.1 a 2.3)
REQM	Cambio pedido por mail directo al programador , que lo acepta solo · cambio sin análisis de impacto · sin trazabilidad (se prueba sólo lo modificado) · requisitos aceptados sin criterio · sin compromiso del equipo	Criterios de aceptación (SP 1.1) · compromiso (SP 1.2) · gestionar los cambios (SP 1.3) · trazabilidad (SP 1.4) · inconsistencias (SP 1.5)
RD	Requisitos verbales o informales · el cliente no revisa nada hasta la instalación · relevamiento sin técnicos	Obtener y desarrollar los requerimientos (SP 1.1, 1.2) · validarlos (SP 3.5)
MA	Se mide sin objetivo , o no se mide · factor de productividad de mercado que nunca se recalibra · años sin datos históricos propios · métricas usadas para evaluar personas · recolección sin procedimiento	Objetivos (SP 1.1) · procedimientos de recogida (SP 1.3) · almacenamiento y uso apropiado (SP 2.3)

ÁREA	SEÑALES DE PROBLEMA EN LA PRÁCTICA DESCRIPTA	LO QUE FALTA
VER	El programador prueba su propio programa · datos de prueba decididos al ejecutar · casos sin documentar o sin resultado esperado · resultados sin registro · en mantenimiento se prueba sólo lo modificado · sin revisiones entre pares	Seleccionar (SP 1.1) · procedimientos y criterios (SP 1.3) · revisiones entre pares (SG 2) · realizar y analizar, con registro (SP 3.1, 3.2)
VAL	La única prueba es la del usuario, y la diseña él · el primer feedback del cliente llega con el sistema terminado · sin criterios de aceptación acordados	Seleccionar (SP 1.1) · procedimientos y criterios (SP 1.3) · validar (SP 2.1)
OPF	Mejoras comunicadas por mail y nada más · nadie controla si los proyectos las adoptan · lecciones aprendidas archivadas sin que nadie las consulte · mejoras sin plan ni piloto	Planes de acción (SP 2.1, 2.2) · desplegar los activos (SP 3.1) · monitorizar la implementación (SP 3.3) · incorporar experiencias (SP 3.4)
OPD	Cada grupo usa su propia estructura de documento · nivel de detalle "según el autor" · sin guías de adaptación · sin biblioteca ni repositorio de medición · entornos distintos entre desarrollo y testing	Las seis prácticas, de SP 1.1 a SP 1.6
OT	Personal sin formación en las técnicas que usa la SF · no se registra quién sabe qué · se asigna gente sin mirar su legajo · no se mide si la capacitación sirvió	Necesidades y plan (SP 1.1 a 1.4) · impartir (SP 2.1) · registros (SP 2.2) · eficacia (SP 2.3)
RSKM	Los riesgos se tratan cuando ya ocurrieron	Preparar, identificar y mitigar antes (SG 1 a SG 3)

Una práctica puede fallar **por defecto** —no se mide nada, no hay guías de adaptación— o **por exceso o mala aplicación**: se mide todo sin objetivo, se aceptan cambios sin control, un proyecto chico completa la plantilla de uno grande porque no hay guías de adaptación. Las dos formas generan problemas de calidad.

25.5 Lo que no es un problema

Las trampas también están del lado del "Sí": hay rasgos que parecen sospechosos y son exactamente lo que pide el modelo. Que haya **testers dedicados exclusivamente** a probar es la independencia que piden los principios de Myers; si hay un problema, está en otro lado. Que la no conformidad se intente resolver **primero dentro del proyecto** es lo que pide CMMI. Que el **líder de proyecto sea responsable de corregir** es cierto; lo que no puede faltar es que QA siga la no conformidad y la escale. Un QA **embebido o hecho por pares** en una organización chica es válido si los evaluadores están formados, no participaron del producto evaluado y tienen un canal independiente hacia la gerencia. Que el repositorio sea **por proyecto** no es lo que exige CM. Y que un **supervisor revise las estimaciones** está bien; el problema, si lo hay, es con qué base se estimó.

25.6 Los distractores que se repiten

PATRÓN DEL DISTRACTOR

POR QUÉ ES FALSO

"No genera problemas: es una implementación correcta (o completa) de X"	Justamente le falta un eslabón de X. Y si X ni siquiera es el área en juego —"correcta de OPD" para un versionado de producto—, es falsa por sí sola
Una razón de otra área : "SQA no verifica que se cumplan los requisitos" (eso es VER), "no se hacen pruebas dinámicas de no conformidad" (mezcla VER y PPQA), "no se guarda en un repositorio organizacional" (eso es OPD)	Cada área tiene su responsabilidad; la razón tiene que atacar la falla del área que se está implementando
Una razón inventada , que agrega un requisito que nadie pide: "faltan almacenar métricas de las no conformidades escaladas al área de testing", "SQA debería participar de los relevamientos"	No es lo que pide el modelo ni lo que falla en el enunciado
La sanción como mecanismo de resolución: "si no las resuelve, es sancionado en su legajo"	Sancionar no resuelve la no conformidad; además, usar medidas para evaluar a las personas es un uso inapropiado de los datos (capítulo 22)
La buena comunicación o la confianza en lugar del mecanismo: "con quien se tiene buena comunicación"	No reemplaza la independencia ni el registro
" No es necesario definir X en cada proyecto" o "eso ya está en los manuales de la empresa"	CMMI lo pide por proyecto : los manuales describen puestos, no qué hace cada interesado en este proyecto

25.7 Cuatro casos

SQA en sus dos versiones (AD 2024 y 2025). SQA controla los estándares de proceso y de producto, informa las no conformidades, el líder de proyecto tiene que subsanarlas y SQA vuelve a revisar a los 15 y a los 30 días. Hasta ahí, todo bien; la diferencia está en la última oración. En **2024**, si la no conformidad sigue abierta, SQA la reporta al Gerente de Desarrollo, "con quien se tiene buena comunicación ya que tanto los PMs como SQA dependen de esa gerencia": hay escalamiento, pero **no es independiente**, porque ese gerente también manda sobre los jefes de proyecto y, con el proyecto atrasado, tiene incentivos para priorizar la entrega. La correcta fue *"el área de SQA debería depender de otra gerencia para evitar conflicto de intereses al escalar"*, y la buena comunicación era el distractor. En **2025**, el informe se archiva en la categoría de proyectos incumplidores y queda en el legajo del líder: **no hay escalamiento ni seguimiento hasta el cierre**. La correcta fue *"no contempla un mecanismo para controlar, por parte de un grupo externo al desarrollo, que se resuelvan las no conformidades hasta su fin escalándolas si fuera necesario"*, y la trampa era la opción que defendía la sanción. En 2024, una opción casi idéntica a esa era **falsa**, porque ahí el seguimiento existía: las mismas palabras valen o no según el enunciado.

El versionado sin línea base (AD 2024). La SF vende productos a varios clientes; la política identifica la versión del producto y la de cada artefacto, pero cada artefacto se versiona por su cuenta y **no se guarda relación** con las versiones de los demás. Parece prolijo —no se borra nada— y ahí está la trampa: falta la **línea base**, que dice con qué versión de cada artefacto se armó cada versión del producto, y sin eso no se puede reconstruir lo que tiene un cliente. La razón que se deduce de CM es *"si es necesario volver a una versión anterior, no hay información sobre la relación de versiones entre artefactos"*. La que se marcó y estaba mal fue *"el problema principal es que no se almacenan en un repositorio organizacional"*: el repositorio por proyecto no es un defecto de CM, y el organizacional es tema de OPD.

El plan de proyecto sin el cliente (AD 2025). La plantilla del plan indica que el equipo del proyecto incluya sólo roles de la software factory, más las responsabilidades de OT, OPF y OPD. El área es **PP** y la práctica es **SP**

2.6 Planificar la involucración de las partes interesadas, cuyo plan incluye los roles de los interesados relevantes —el cliente entre ellos— por fase del ciclo de vida. Si el usuario clave no sabe que tiene que ir a los relevamientos ni aprobar el prototipo, los requisitos se relevan o se validan tarde, y PMC no puede monitorizar su involucración, porque compara contra un plan que no la dice. La correcta fue *"no contempla que los miembros de proyecto por parte del cliente no conozcan bien lo que deben hacer en el proyecto"*. Lo de OT, OPF y OPD es un segundo error —son áreas de la organización, no roles de un proyecto—, pero ninguna opción del "Sí" lo atacaba.

Testers dedicados que improvisan los datos (final). Una SF tiene testers dedicados exclusivamente a probar, pero deciden los datos de prueba al ejecutar, incluso en mantenimiento. Hay problema, pero **por los datos, no por los testers**: el personal dedicado es la independencia que piden los principios de Myers, mientras que el diseño improvisado pasa por alto casos y, en mantenimiento, impide repetir las pruebas anteriores, con el riesgo de errores de regresión. Falta tener casos documentados antes de ejecutar, revisados y reutilizables (VER). Una opción que dijera "Sí, porque los testers no deberían dedicarse sólo a probar" sería falsa aunque el "Sí" fuera correcto.

25.8 Los formatos de desarrollo de los finales

En los finales la misma idea se pide por escrito, y una guía de alumnos de 2015, que coincide con cómo se corrigieron, muestra qué se espera. Se parte del **problema directo** —lo que deja de existir si no se aplica la práctica— y se lo **escala**: el **producto** (errores, no cumple las expectativas), el **proyecto** (retrasos, más costo por retrabajo) y la **organización** (pérdida de imagen y de competitividad). Si una SF no calcula los puntos función (PP SP 1.2), no tiene base para estimar esfuerzo, costo ni calendario; se compromete con plazos imposibles; el proyecto se atrasa o cuesta más, y la organización pierde competitividad. El punto de llegada casi siempre es el mismo: lo que se evalúa es **el camino**.

Se escribe **en potencial** ("podría", "se corre el riesgo"). No se **transcribe CMMI**: "no transcriban del CMMI, tienen que razonar", pidió Rozas en una consulta. Si se pide el nivel de madurez, vale la regla del capítulo 6: un área de nivel 2 deja a la organización en nivel 1, y una de nivel 3, como máximo en nivel 2. Y las **recomendaciones** son actividades concretas, "bajadas a tierra" para alguien que no conoce CMMI, con su área de proceso; la práctica específica no es obligatoria.

ATENCIÓN En una recomendación de PPQA, el **escalamiento** no es opcional. En un final en que SQA proponía plantillas nuevas y los analistas seguían usando las viejas, el profesor Ripani marcó como lo más importante la frase "en caso de persistir esta situación, se podrá elevar el problema a la gerencia acompañado de un registro respaldatorio": sin esa oración, la respuesta era incorrecta. Es la misma idea que decidió los BP del AD 2024 y 2025.

26. Confusiones frecuentes

Para cerrar, las confusiones que más se repiten, agrupadas por tema. No son trucos de examen: cada una es un concepto mal entendido, y casi todas aparecieron como opción incorrecta en algún parcial o final.

Sobre la calidad y el software. Atribuir a CMMI la definición de "necesidades explícitas o implícitas", que es la de ISO; la de CMMI es la que incluye al **proceso**. Creer que asegurar la calidad es probar el producto, cuando las revisiones y las pruebas son **control** (QC, reactivo, sobre el producto) y el aseguramiento es **preventivo** y mira el **proceso**. Creer que el software son los programas, cuando son **programas, datos y documentos**. Confundir el mantenimiento **perfectivo**, que mejora la calidad interna, con el **evolutivo**, que cambia lo que el sistema hace; y llamar evolutivo a un cambio de entorno, que es **adaptativo**. Y tomar como principios de la disciplina frases como "las personas y el tiempo son intercambiables" o "hazlo rápido y después correcto", que son sus principios **dados vuelta**.

Sobre CMMI y sus niveles. Creer que las prácticas son obligatorias, cuando lo requerido son **sólo las metas**; que las subprácticas son esperadas, cuando son **informativas**; y que las prácticas específicas tratan la institucionalización, que es cosa de las **genéricas**. Creer que un nivel superior exige de las metas de los anteriores, cuando los niveles son **acumulativos**: una organización que cumple todo el nivel 3 pero falla un área de nivel 2 queda en **nivel 1**. Creer que el nivel 4 elimina las causas comunes, cuando trata las **especiales** y las comunes son del **nivel 5**. Creer que la capacidad mide a la organización, cuando se mide **por área**, y que va de 0 a 3, cuando en la versión 1.2 va de **0 a 5**. Y creerle a las erratas de la traducción: OT es de **nivel 3**, OID de **5**, RSKM de **3**, y OPF, OPD, OT, OPP y OID son de **gestión de procesos**. El aseguramiento de la calidad nace en el **nivel 2**, con PPQA.

Sobre la gestión de procesos. Confundir OPF con OPD: **OPF diagnostica y despliega, OPD define y guarda**. Confundir pilotear con desplegar: una propuesta todavía no aprobada se prueba en pilotos (**OPF SG 2**); una ya aceptada se despliega a toda la organización (**SG 3**). Atribuir a MA el repositorio de medidas de la organización, que es un activo de **OPD SP 1.4**, y confundirlo con la biblioteca de activos (**SP 1.5**), que guarda documentos. Llamar OT a toda capacitación, cuando la de la gente de un proyecto se planifica en **PP SP 2.5**. Y confundir la **plantilla** de caso de uso, que es un **activo**, con el caso de uso, que es un **producto de trabajo**.

Sobre SPEM y RUP. Creer que SPEM es un lenguaje de modelado de procesos en general o la descripción de un proceso concreto, cuando es un **metamodelo** que da el vocabulario mínimo —rol, producto de trabajo, tarea— para describir cualquier proceso. Confundir fases con disciplinas: las fases **ordenan el tiempo** y las disciplinas **agrupan el tipo de trabajo**. Y creer que RUP es una cascada porque tiene cuatro fases, cuando dentro de cada fase hay **iteraciones**.

Sobre cómo reconocer el área. Creer que el área la define el **tema** del enunciado, cuando la define la **actividad**: escalar una no conformidad sobre requerimientos es **PPQA**, no REQM. Distinguir PP de PMC por el tema de la reunión, cuando decide su **momento**: por delante se planifica, ya ocurrida se monitoriza. Creer que todo control de un artefacto es PPQA, cuando PPQA compara contra un **estándar de la organización**, VER contra **lo especificado** y CM controla las **convenciones del propio proyecto**. Creer que todo lo que hace el sector de SQA de una empresa es PPQA, cuando proponer mejoras a los procesos es **OPF** y guardarlas es **OPD**. Y no considerar **NINGUNA** cuando la opción combina una sigla con una práctica de otra área.

Sobre la planificación y la estimación. Creer que un Gantt es un plan de proyecto, cuando es sólo una vista del cronograma. Creer que cualquier desvío debe resolverse, cuando PMC actúa sobre los **significativos**. Ubicar **definir las actividades** en el grupo de Tiempo, cuando pertenece a **Alcance**. Sumar **duración** donde se pide **esfuerzo**: el esfuerzo se suma; la duración sale del camino crítico, y reordenar tareas cambia la duración, no el esfuerzo. Olvidar que los **módulos independientes arrancan en paralelo**. Confundir **tamaño** con esfuerzo: contar puntos función es **PP SP 1.2** y pasarlos a horas es **PP SP 1.4**. Y aplicar a lo eliminado en una mejora el factor de ajuste de después, cuando le corresponde el de **antes**.

Sobre los riesgos. Creer que el análisis va antes que la identificación, cuando el orden es **identificar, analizar, priorizar**. Creer que cualquier desvío dispara el tratamiento, cuando se actúa al superar un **umbral**. Confundir **mitigación**, que actúa antes para reducir probabilidad o impacto, con **contingencia**, que responde cuando el riesgo ya ocurrió. Creer que aceptar es siempre no hacer nada, cuando INTECO admite la aceptación **activa**, y que transferir significa lo mismo en CMMI (reasignar requerimientos) y en INTECO (pasar el impacto a un tercero). Y cargar el costo de una mitigación a la reserva, cuando va al **presupuesto**.

Sobre los ciclos de vida. Creer que en un incremental el análisis se hace sólo en el primer incremento, cuando **cada incremento es una cascada completa**. Confundir incremental con iterativo: el incremental agrega pedazos distintos; el iterativo **vuelve sobre los mismos** requisitos. Asignar al incremental la alta incertidumbre, que es propia del **espiral**, o elegir cascada con requisitos poco claros, cuando es **el menos adecuado**. Lo que decide

la elección es la **estabilidad de los requerimientos** y la necesidad de **salir antes a producción**, no el presupuesto ni el lenguaje.

Sobre los requerimientos y sus cambios. Creer que los no funcionales son opcionales, cuando son **tan importantes como los funcionales**. Tomar como requisito la solución que sugiere el cliente. Elegir la opción en la que el equipo **decide solo**, cuando RD exige confirmar con el cliente. Confundir RD SP 3.5, que valida **los requerimientos**, con VAL, que valida **el producto**. Atribuir a RD la trazabilidad, que es **REQM SP 1.4**. Elegir la opción que **implementa antes de evaluar el impacto**, aunque registre el cambio y actualice la matriz. Separar REQM de CM por quién pide el cambio, cuando decide si el cambio **altera un requisito** y en qué **momento** está: todavía evaluándose es REQM SP 1.3; ya aceptado y en trámite, **CM SP 2.1**. Y creer que las solicitudes de cambio tratan sólo requisitos, cuando también cubren **defectos**.

Sobre verificación y validación. Creer que las pruebas de sistema son validación porque se hacen sobre el producto terminado, cuando se contrastan contra la especificación y son **verificación**. Creer que toda técnica dinámica es validación y toda estática es verificación, cuando una prueba de integración es dinámica y es **VER**, y una revisión de requisitos con el cliente es estática y es **VAL**. Atribuir a VAL las **revisiones entre pares**, que existen sólo en VER. Y ubicar la prueba de aceptación en VER porque CMMI la nombra entre sus métodos, cuando en el examen se toma como **VAL**.

Sobre las pruebas. Creer que probar es demostrar que el programa funciona, cuando para Myers es **ejecutarlo con la intención de encontrar errores**. Creer que un sistema que pasó todas las pruebas está libre de errores, cuando sólo **pasó los casos ejecutados**. Creer que después de una corrección alcanza con re-ejecutar los casos que fallaron, cuando eso es **confirmación**: la **regresión** re-ejecuta también lo que ya andaba. Llamar alfa a cualquier prueba sobre un prototipo, cuando alfa y beta **requieren al cliente**. Y creer que las revisiones reemplazan a las pruebas o que un nivel de prueba reemplaza a otro, cuando son **complementarios**.

Sobre el diseño de los casos de prueba. Creer que partición y valores límite son alternativas, cuando la segunda **mejora** a la primera: una pide **un representante** por partición y la otra **los extremos**. Leer mal el enunciado: con enteros, "superior a 65" empieza en **66** y "más de 6" en **7**, y la inválida de "mayor a cero" es ≤ 0 . Juntar en una partición los valores de una lista que cambian el comportamiento, o dejar el **vacío** sin partición propia. Creer que al definir los casos influye si la prueba es alfa o beta, quién la ejecuta o el lenguaje, cuando el caso se define por **entradas, salidas y comportamiento esperados** y por la **técnica**. Contar toda condición de una tabla de decisión como binaria. Y creer que el 100 % de cobertura de sentencias garantiza que no hay defectos.

Sobre las revisiones. Creer que el walkthrough y la inspección los dirige la misma persona, cuando el primero lo conduce **el autor** y la segunda **un moderador que nunca es el autor**. Creer que la revisión técnica busca defectos, cuando busca **consenso técnico**. Creer que las inspecciones son un área o que sólo las hace Verificación, cuando son una **técnica** que también usa PPQA. Y clasificar el análisis estático como dinámico por ser automatizado, cuando **no ejecuta nada**.

Sobre la gestión de configuración. Creer que conservar todas las versiones de cada artefacto es hacer gestión de configuración, cuando sin **línea base** no se sabe qué versión de cada artefacto compone cada versión del producto. Creer que una línea base no se puede modificar, cuando se puede, pero **sólo por control de cambios**. Dejar afuera las **herramientas** con que se construyó el producto, o incluir algo que todavía no existía en el momento del enunciado. Calcular la supervivencia de un esquema de versión con el componente que cambia más seguido, cuando si el menor se reinicia con cada cambio del mayor, **limita el mayor**. Contar como release cada versión interna, cuando release es **lo que se hace público**: una en cascada, una por incremento. Llamar versión a una adaptación a otra plataforma, que es una **variante**. Y creer que CM exige un repositorio organizacional, que es de **OPD**.

Sobre el aseguramiento de la calidad. Creer que la auditoría termina con el informe, cuando termina cuando las no conformidades **se resuelven**. Creer que toda no conformidad se escala, cuando primero se intenta resolverla **en el proyecto**. Creer que el escalamiento es opcional, o que archivar el informe o sancionar al líder lo reemplaza. Creer que alcanza con escalar a un gerente "con quien hay buena comunicación", cuando si también manda sobre los líderes de proyecto es **juez y parte**. Creer que QA cumplió con informar porque el líder es responsable de corregir, cuando el **seguimiento hasta el cierre** sigue siendo de QA. Y creer que PPQA verifica que el producto cumpla los requerimientos, cuando evalúa la **adherencia a procesos y estándares**.

Sobre la medición. Creer que toda métrica que menciona fases o defectos es de proceso, cuando el cumplimiento de plazos compara plan contra real y es de **proyecto**, y los defectos en producción son de **producto**: es de proceso la que mide cómo funciona una actividad. Creer que conviene empezar a recolectar datos cuanto antes, cuando primero se **alinea** con los objetivos y recién después se mide. Creer que medir la productividad de cada desarrollador para premiarlo o sancionarlo es buena práctica, cuando CMMI lo cita como **uso inapropiado**. Usar métrica, medida e indicador como sinónimos. Y creer que replanificar con las métricas es MA, cuando MA mide y **PMC** actúa.

Sobre las pericias. Creer que el perito hace un backup del disco, cuando hace una **imagen forense bit a bit**. Recolectar primero el disco o apagar el equipo, cuando el orden va de **mayor a menor volatilidad**. Aplicar una sola regla al equipo encendido en cualquier contexto. Creer que la cadena de custodia empieza cuando el material llega al perito, cuando arranca en el **primer contacto** con la evidencia. Creer que el MD5 garantiza la integridad sin discusión, o que un bloqueador de escritura congela un SSD con TRIM. Y creer que el perito hace el allanamiento, que es tarea de la **policía con el fiscal**.

Sobre las preguntas BP Creer que alcanza con acertar el sí o el no, cuando se corrige la **razón**. Combinar opciones del "Sí" con opciones del "No", que la consigna prohíbe. Creer que una sanción, la buena comunicación o los manuales de la empresa reemplazan el mecanismo que falta. Y marcar como problema lo que el modelo pide: testers dedicados, intentar resolver la no conformidad primero en el proyecto, o que el líder de proyecto sea responsable de corregirla.